Recent progress on the butterfly factorization

Samuel F. Potter

Courant Institute of Mathematical Sciences
New York University

What's the first part of this talk about?

A new algorithm:

A fast algorithm based on the butterfly factorization of the "manifold harmonic transform" (linear transform into spectrum of discretized Laplace-Beltrami operator)...

Why do we care?

There's no fast Fourier transform for an unstructured domain. Lots of applications could take advantage of one in one way or another.

What's the second part of this talk about?

A new implementation:

As part of a long-term project on fast direct solvers for high-frequency scattering problems, we developed an efficient implementation of the multilevel butterfly factorization (Michielssen's "MLMDA"—the original butterfly algorithm).

Thoroughgoing numerical experiments:

Experiments show that this butterfly-accelerated forward operator is competitive with the state of the art, the high-frequency FMM (as implemented in Flatiron/CCM's `fmm2d`).

What's the third part of this talk about?

A new software library:

The throughline of this work is a new C library called `butterfly` implementing all of the butterfly algorithms discussed. We discuss the motivation for this library and what our plans are for it.

Butterfly-accelerated manifold harmonic transforms

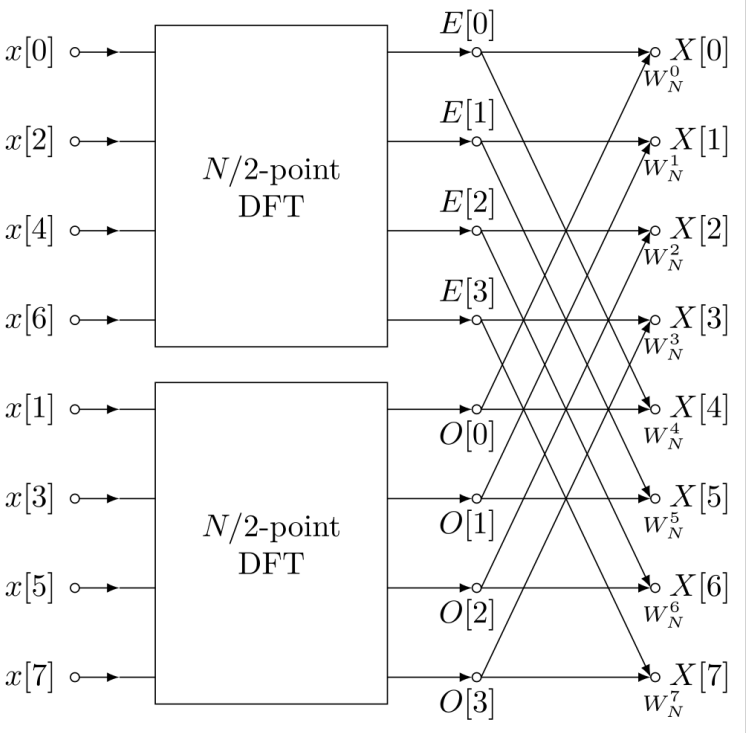
History of the butterfly factorization

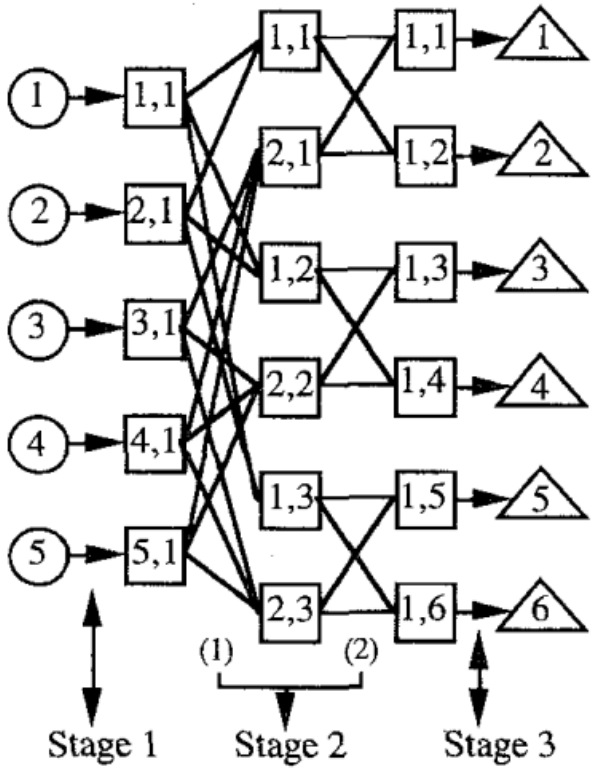
- **1996:** Michielssen and Boag introduce the butterfly factorization for problems in electromagnetic scattering, which turns out to be of broader applicability.
- **2007–10:** O'Neil, Rokhlin, et al. use the butterfly factorization to systematically and rigorously develop fast special function transforms.
- **2010–now:** Extension to higher (3!) dimensions, application to Fourier integral operators, fast direct solvers, etc.

Authors involved: Candes, Demanet, Engquist, Michielssen, Poulson, Ho, Yang, Ying, others.

What is the butterfly factorization?

- Linear-algebraic generalization of the FFT
- Can be used to compress other special function transforms and oscillatory kernel matrices





Left: Cooley-Tukey FFT algorithm - Wikipedia.

Right: Michielssen et al. “A multilevel matrix decomposition algorithm for analyzing scattering from large structures.”

When can we butterfly a matrix?

. . . when it satisfies the **butterfly property**:

An $n \times n$ matrix A can be butterfly factorized into a product of $O(\log n)$ factors with $O(n)$ nonzero entries each if the rows and columns can be permuted such that the rank of contiguous subblocks is proportional to the product of the "area" of the rows and the "area" of the columns (the "space-frequency product").

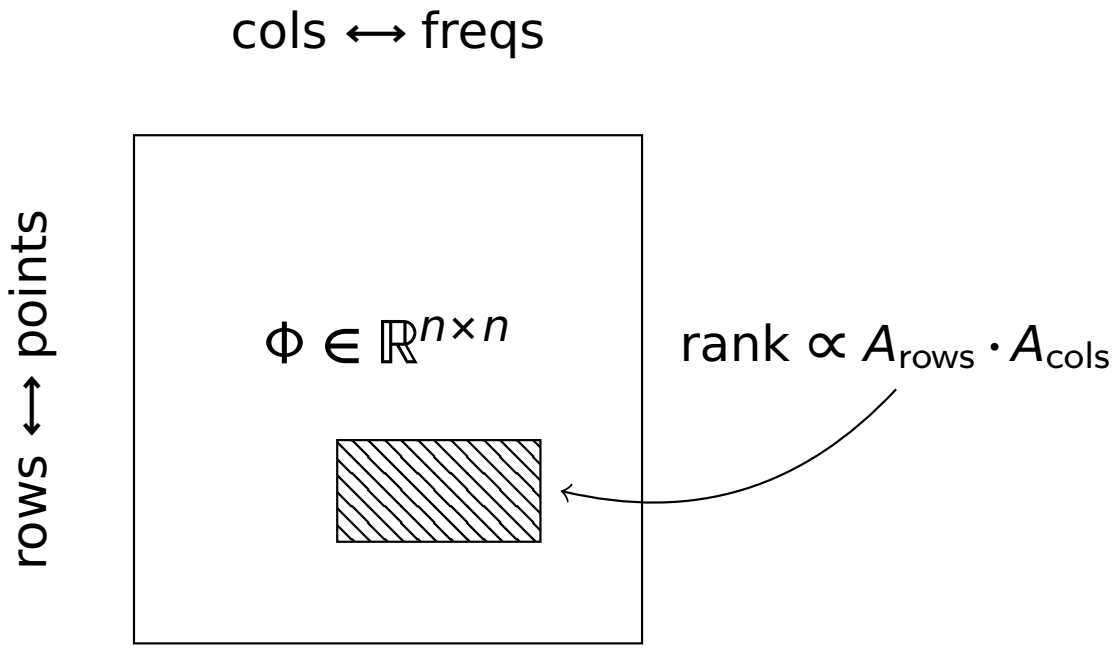
When can we butterfly a matrix?

. . . when it satisfies the **butterfly property**:

> An $n \times n$ matrix A can be butterfly factorized into a product of $O(\log n)$ factors with $O(n)$ nonzero entries each if the rows and columns can be permuted such that the rank of contiguous subblocks is proportional to the product of the "area" of the rows and the "area" of the columns (the "space-frequency product").

???

When can we butterfly a matrix?

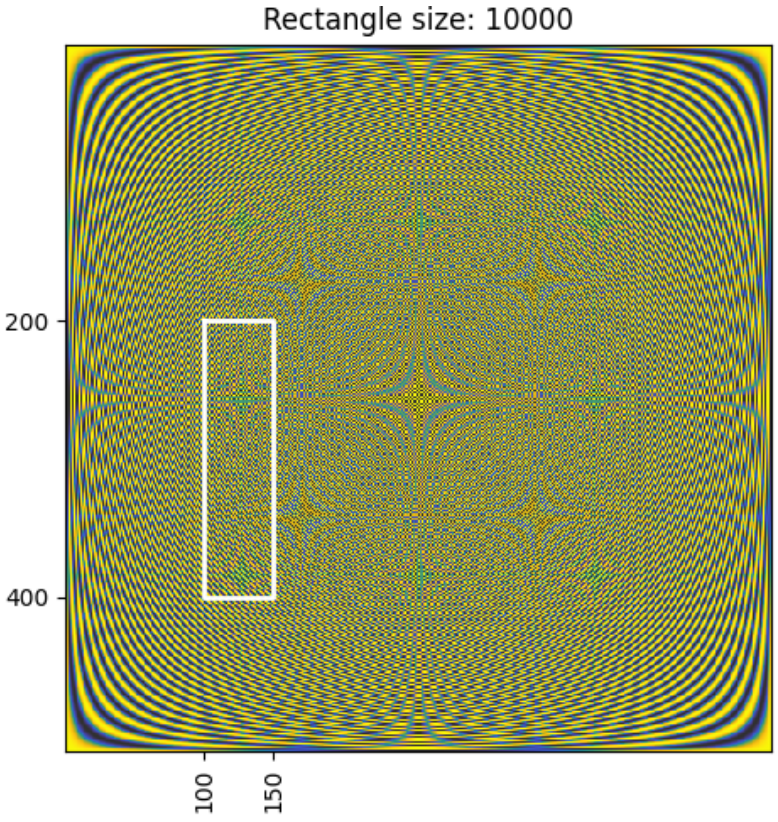


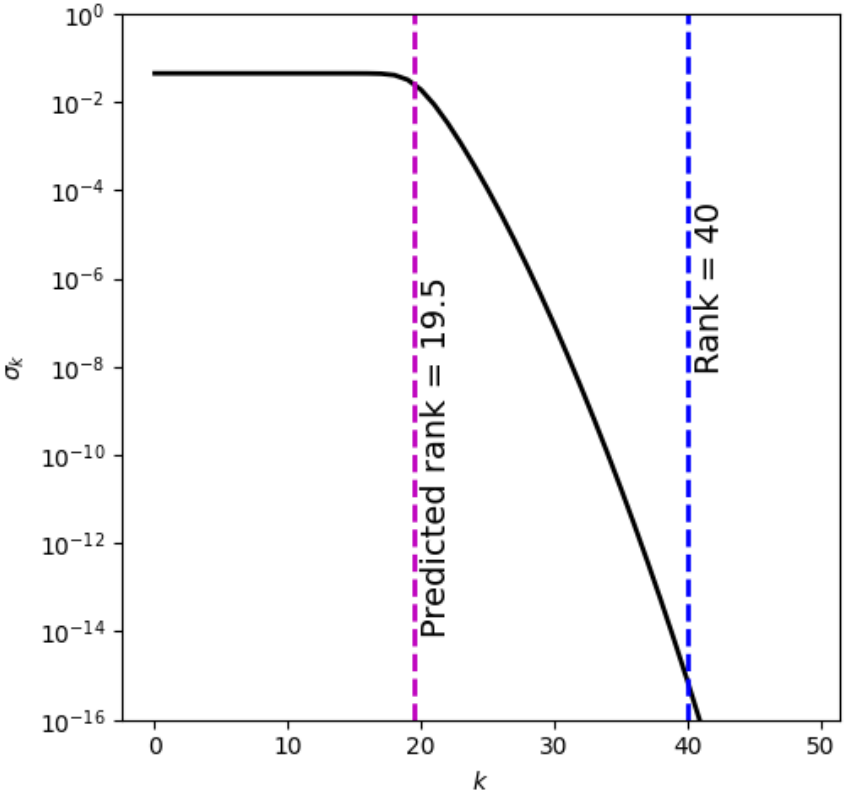
Goal: construct space and frequency trees so that as we move up and down them simultaneously we keep the space-frequency product constant

Example: the discrete Fourier transform

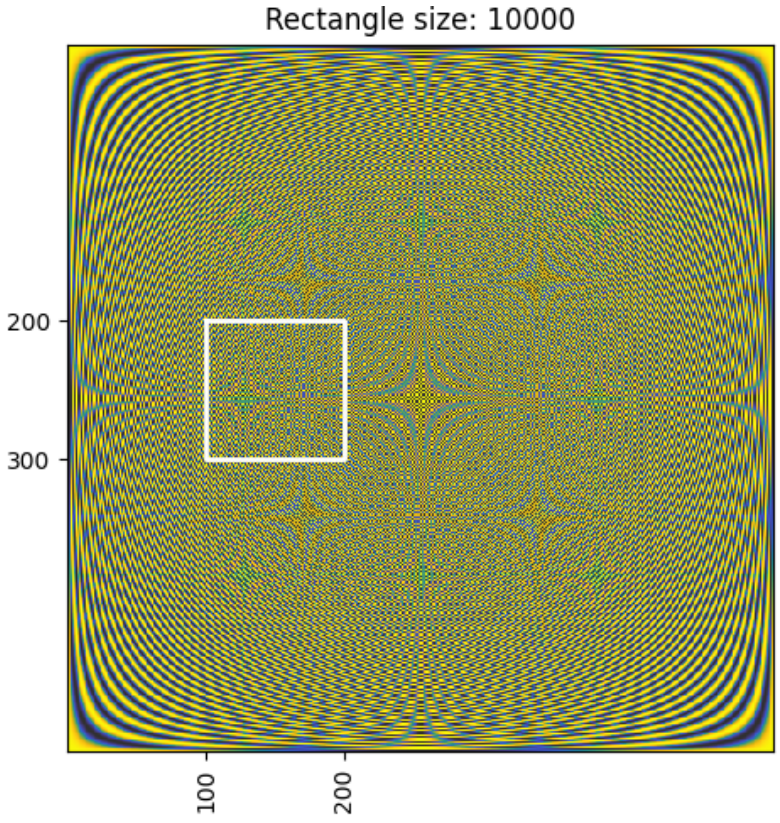
- The DFT: $X \in \mathbb{C}^{N \times N}$, where $X_{kn} = e^{i2\pi kn/N}$
- Easy to prove that the DFT satisfies the butterfly property
- The fast Fourier transform: use algebraic properties of the DFT to apply it in $O(N \log N)$ operations
- Can view the FFT as a sparse matrix factorization of the DFT into $O(\log N)$ sparse factors—so, interpret it as a butterfly factorization
- Can also compute "the" butterfly factorization of the DFT, but very inefficient! (Constant is ~300x worse)

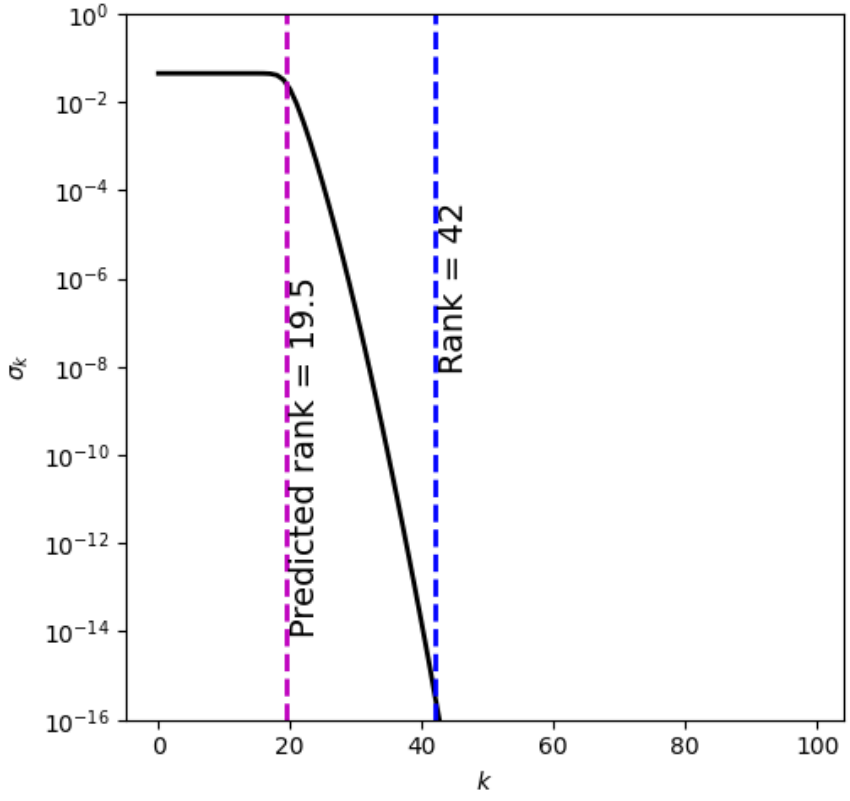
The DFT can be “butterflied”



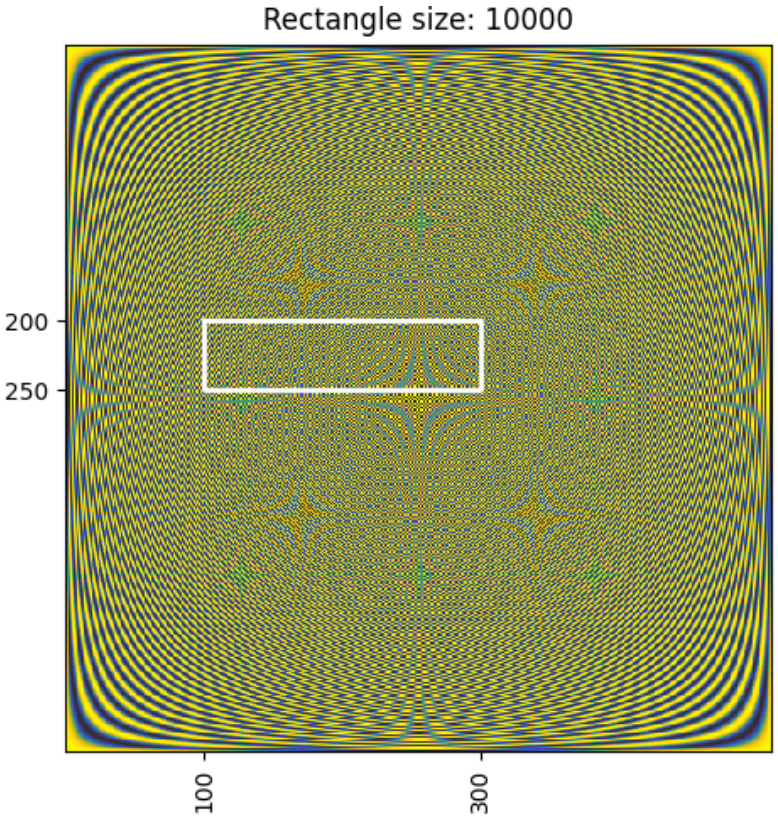


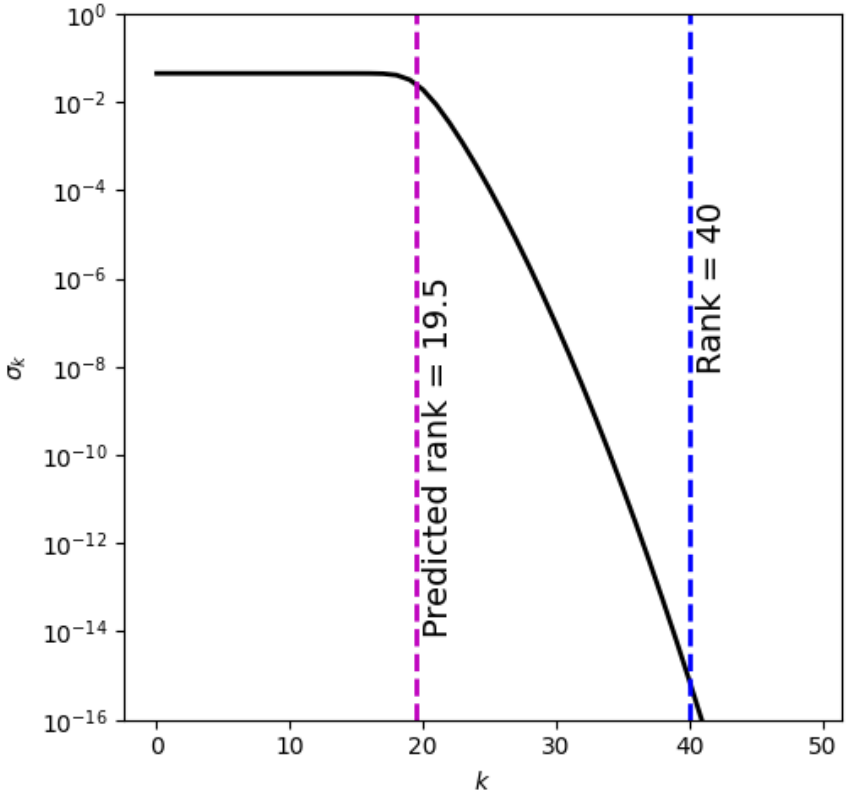
The DFT can be “butterflied”



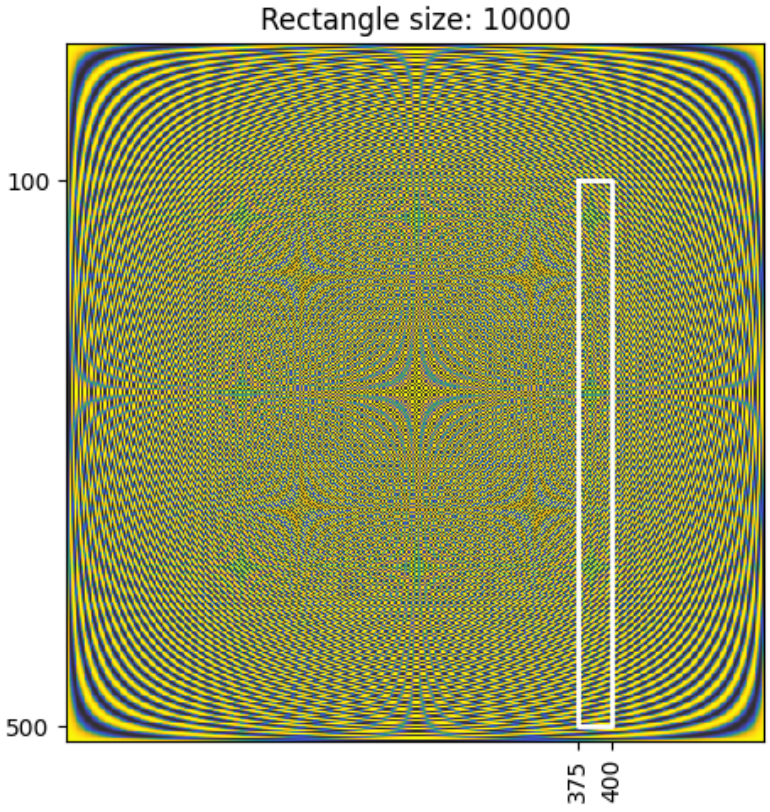


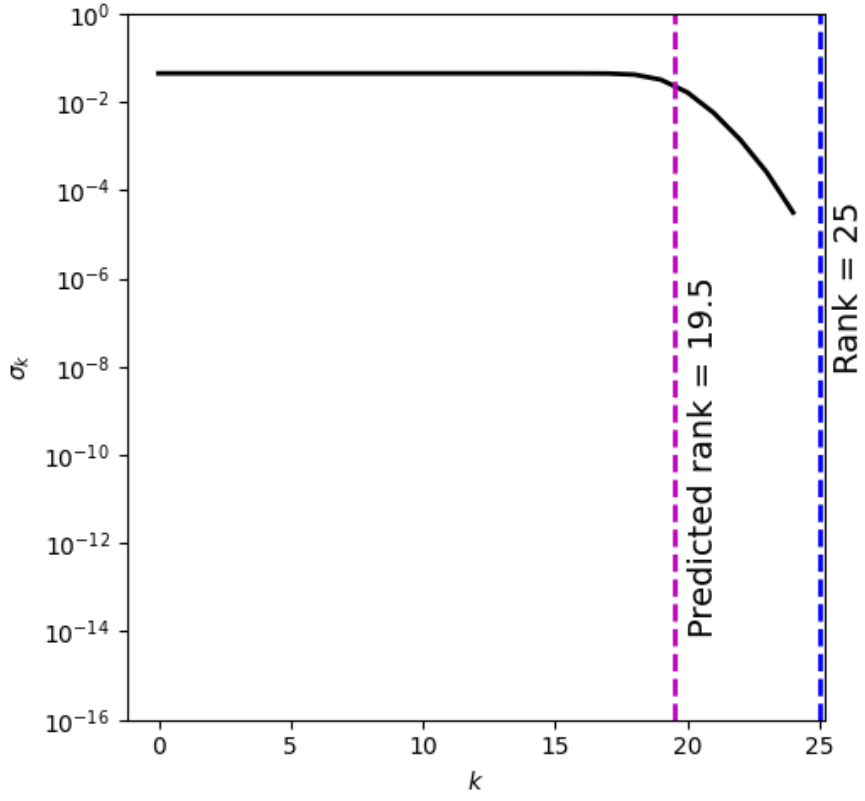
The DFT can be “butterflied”



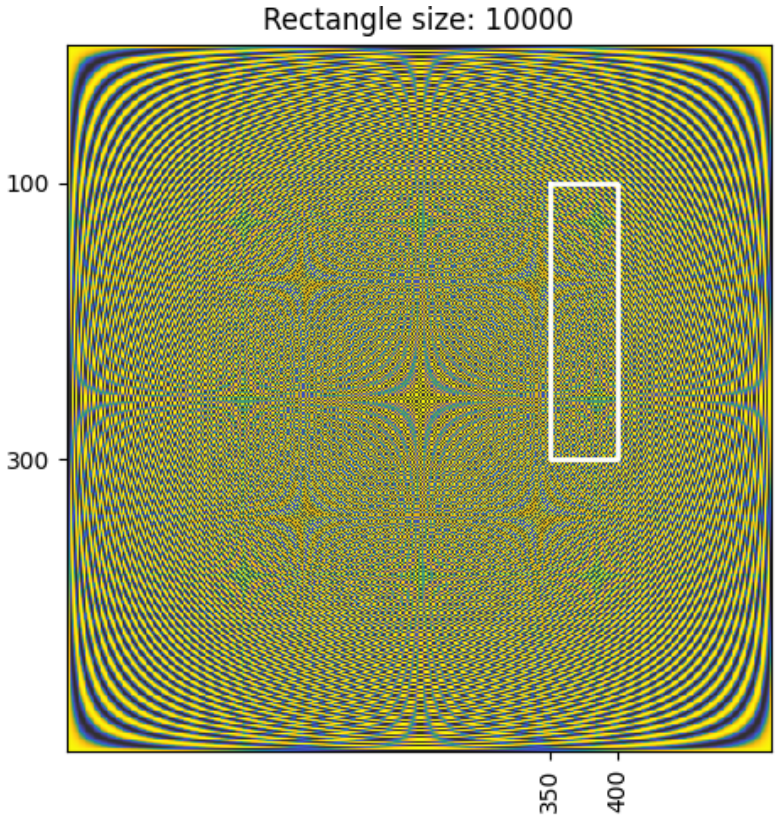


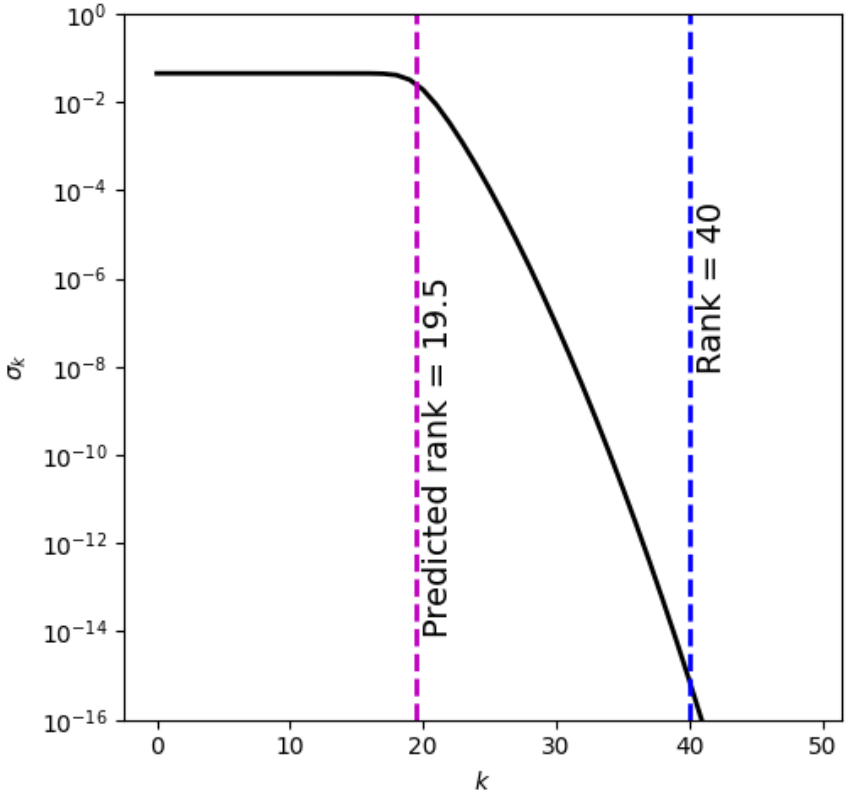
The DFT can be “butterflied”



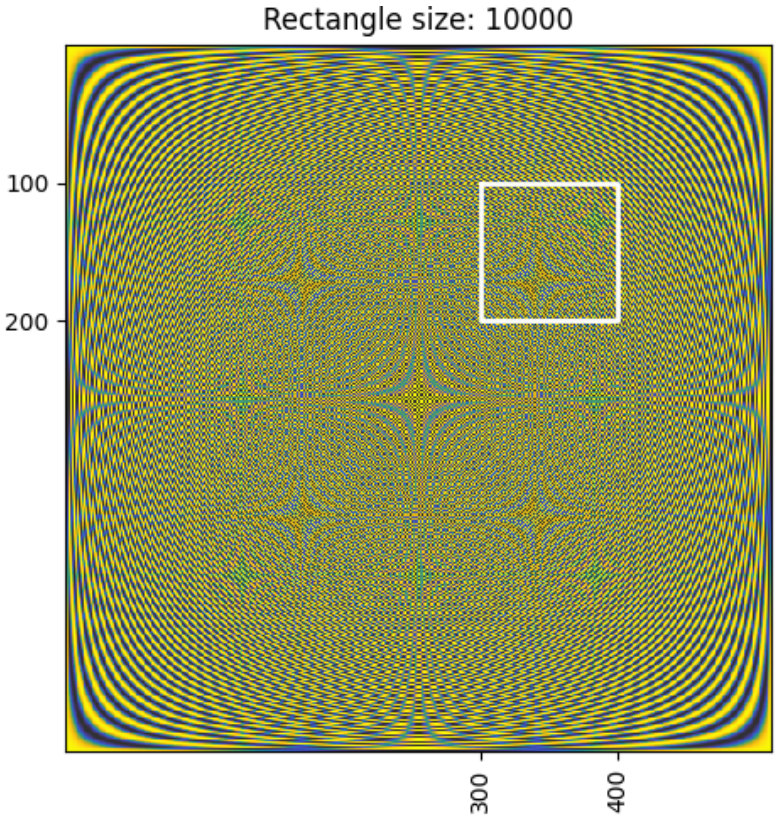


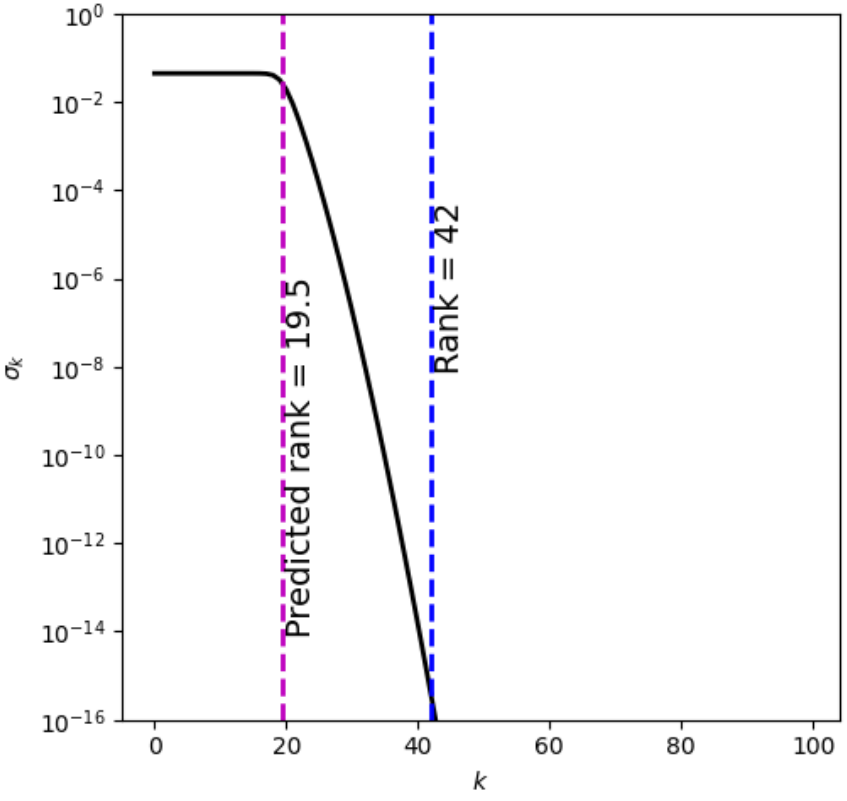
The DFT can be “butterflied”



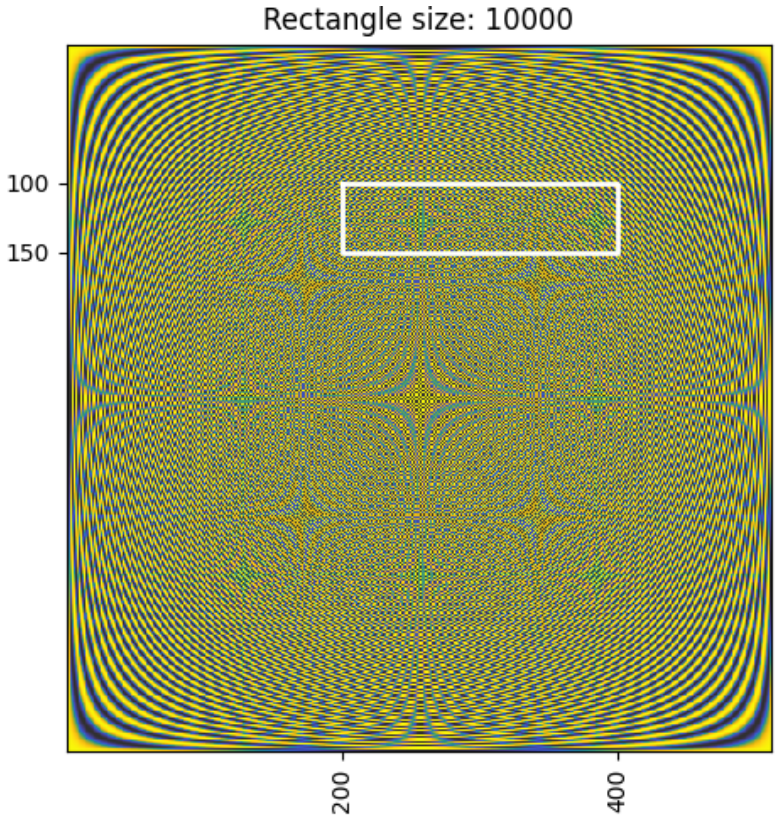


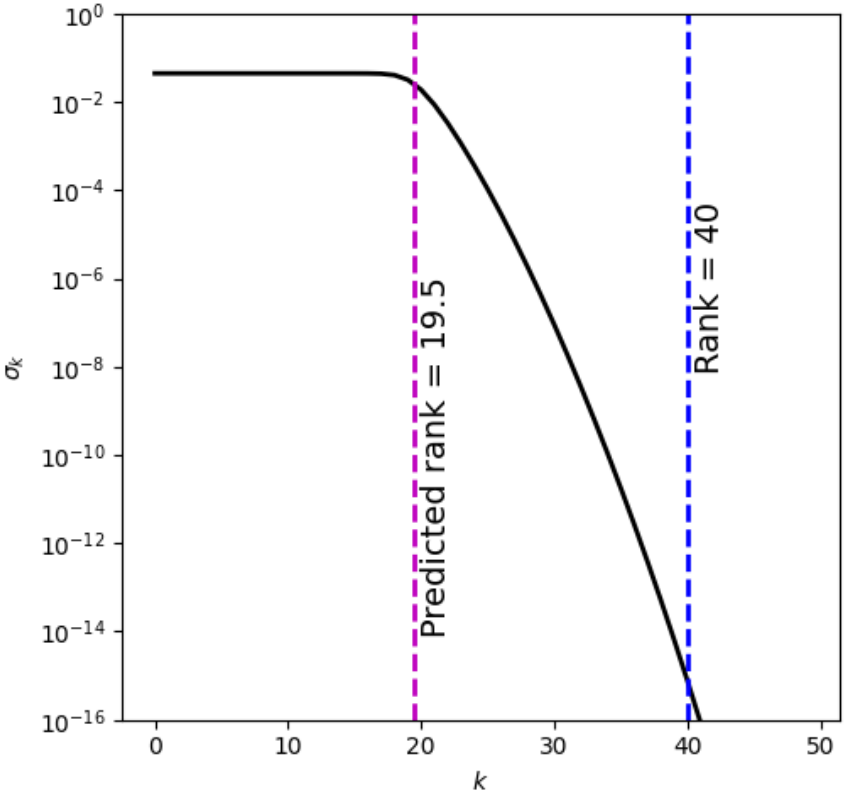
The DFT can be “butterflied”



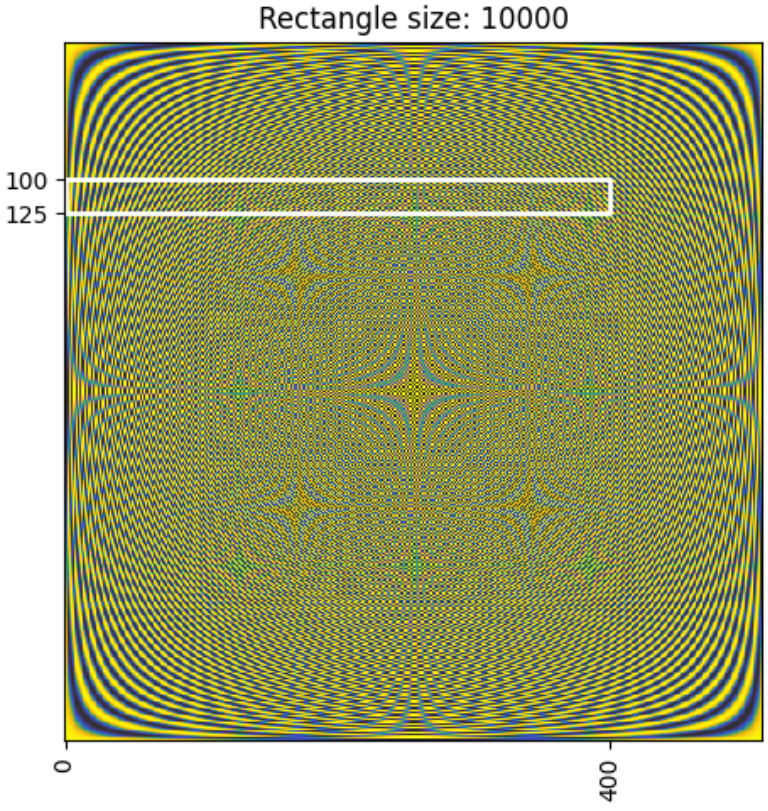


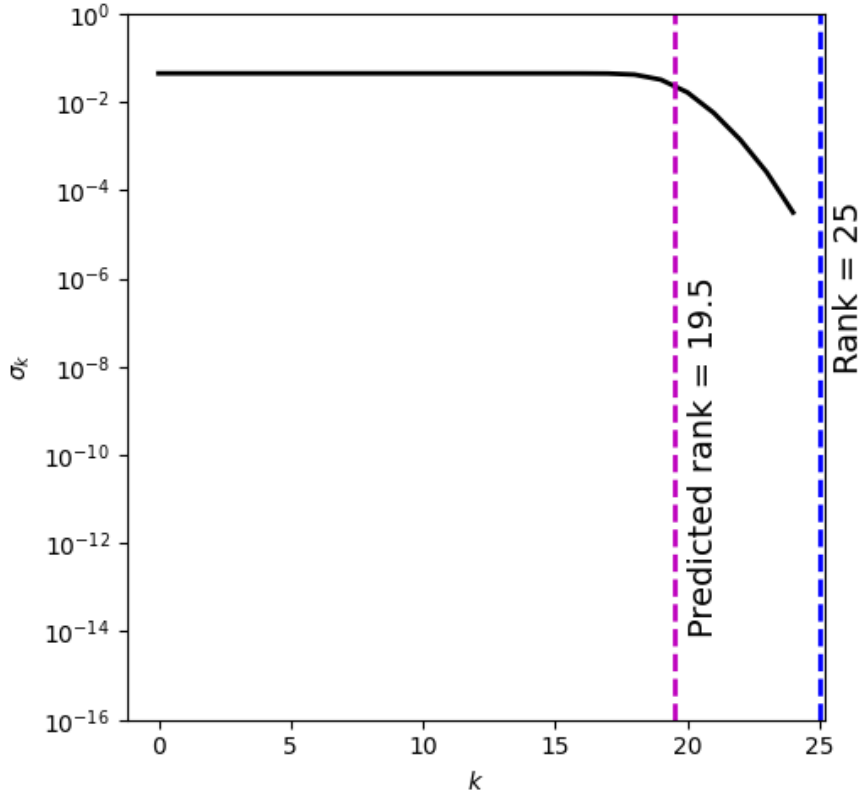
The DFT can be “butterflied”





The DFT can be “butterflied”





What are “manifold harmonics”?

- Let M be a compact Riemannian manifold. The **Laplace-Beltrami operator** is the Laplacian on M:

$$\Delta_M = \nabla_M \cdot \nabla_M$$

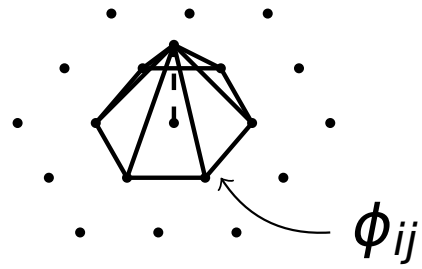
- A **manifold harmonic** is an eigenfunction of the Laplace-Beltrami eigenproblem on M:

$$\Delta_M u = \lambda u, \qquad u \in L^2(M)$$

- Eigenfunctions of Δ_M give an orthonormal basis for $L^2(M)$ (they have *manifold* applications!)

Our focus: genus 0 surfaces $S \subseteq \mathbb{R}^3$.

How do we compute manifold harmonics?



- Discretize $S \rightarrow$ tri. mesh T with vertices $V = \{x_i\}_{i \in I}$
- Set up basis of linear finite elements $\{\phi_i\}_{i \in I}$:

$$\phi_i(x_j) = \delta_{ij}, \qquad x_j \in V, \qquad \sum_i u_i \phi_i(x) \in C^0(T)$$

- Leads to generalized eigenvalue problem:

$$Lu = \lambda M u$$

  where L and M are stiffness and mass matrices.
- Solve using standard sparse eigenvalue package.

How do we compute manifold harmonics?

"Asian dragon" mesh: 56.4K vertices

Source: Stanford 3D Scanning Repository

How do we compute manifold harmonics?

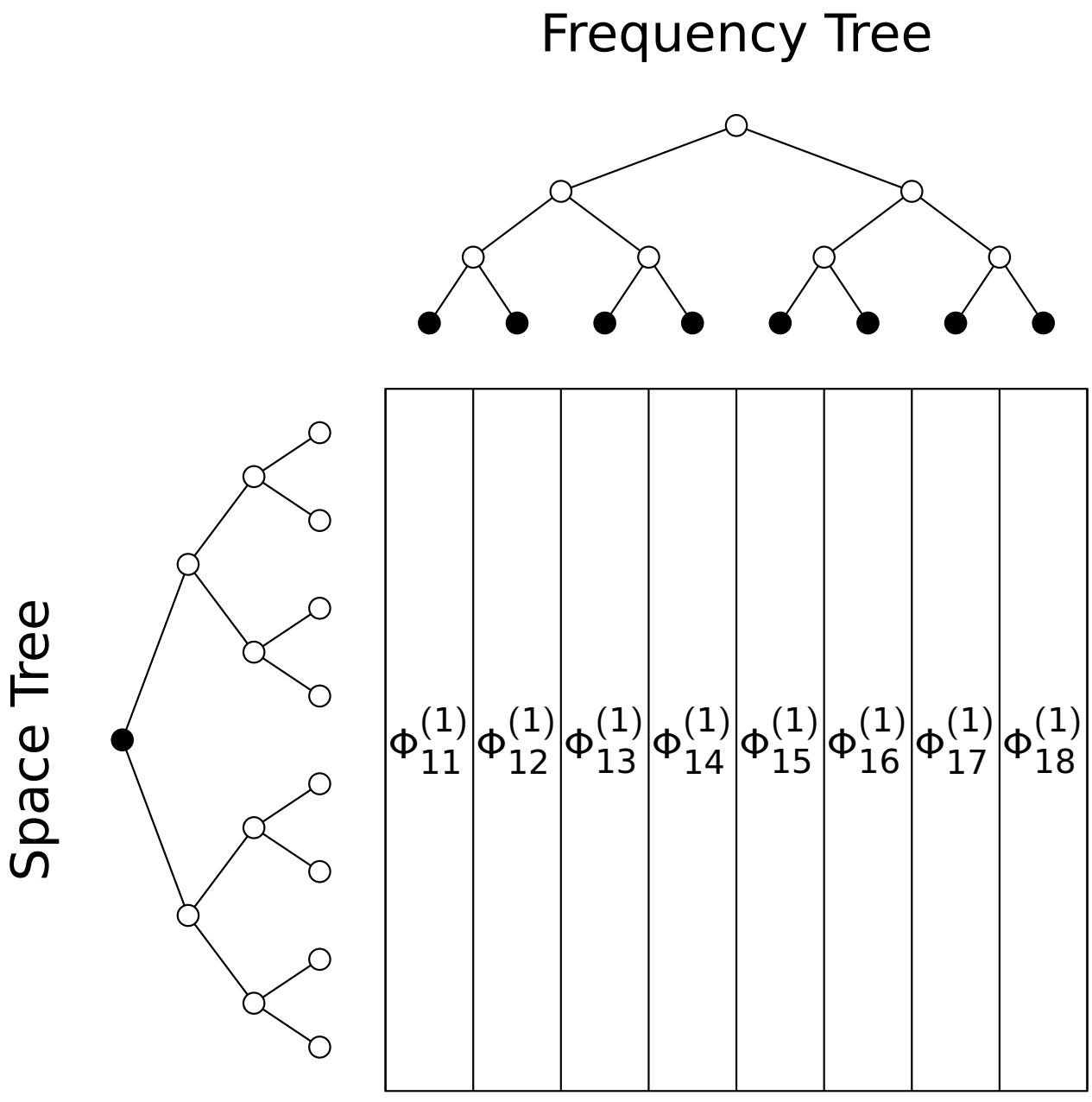
A harmonic ($\lambda \approx 0.848$) on the “Asian dragon” mesh

Source: Stanford 3D Scanning Repository

Spatial partitioning

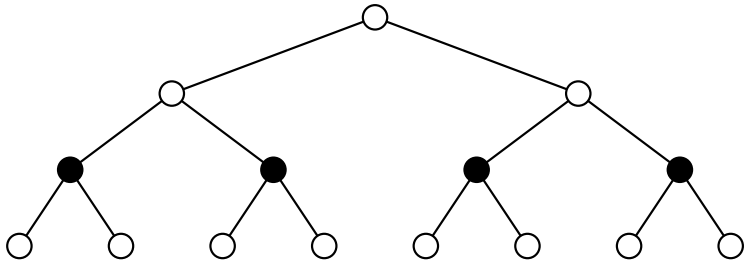
"Asian dragon" with **octree ordering** on vertices

The butterfly property

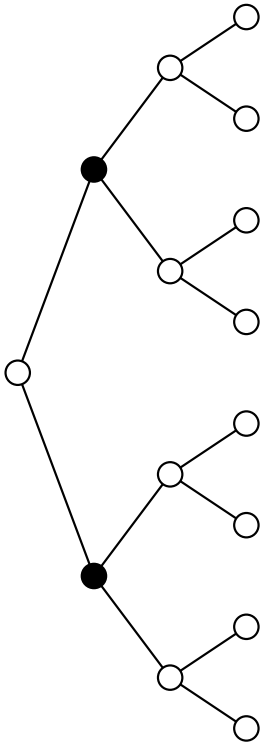


The butterfly property

Frequency Tree



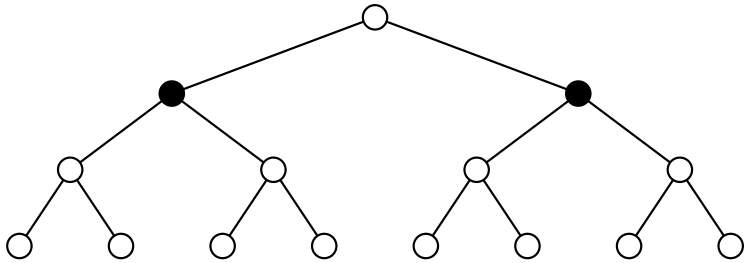
Space Tree



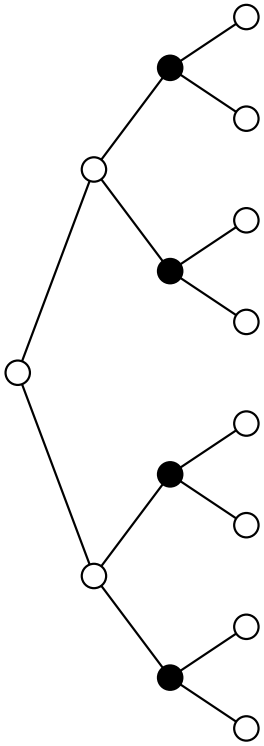
$\Phi_{11}^{(2)}$	$\Phi_{12}^{(2)}$	$\Phi_{13}^{(2)}$	$\Phi_{14}^{(2)}$
$\Phi_{21}^{(2)}$	$\Phi_{22}^{(2)}$	$\Phi_{23}^{(2)}$	$\Phi_{24}^{(2)}$

The butterfly property

Frequency Tree



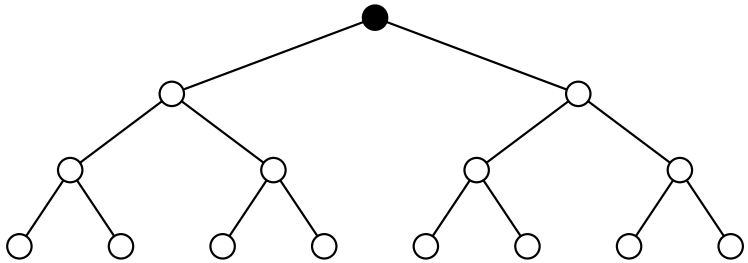
Space Tree



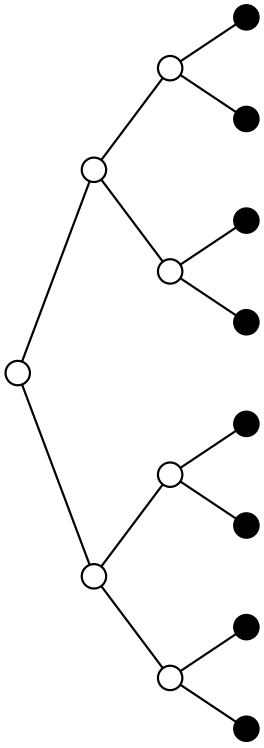
$\Phi_{11}^{(3)}$	$\Phi_{12}^{(3)}$
$\Phi_{21}^{(3)}$	$\Phi_{22}^{(3)}$
$\Phi_{31}^{(3)}$	$\Phi_{32}^{(3)}$
$\Phi_{41}^{(3)}$	$\Phi_{42}^{(3)}$

The butterfly property

Frequency Tree

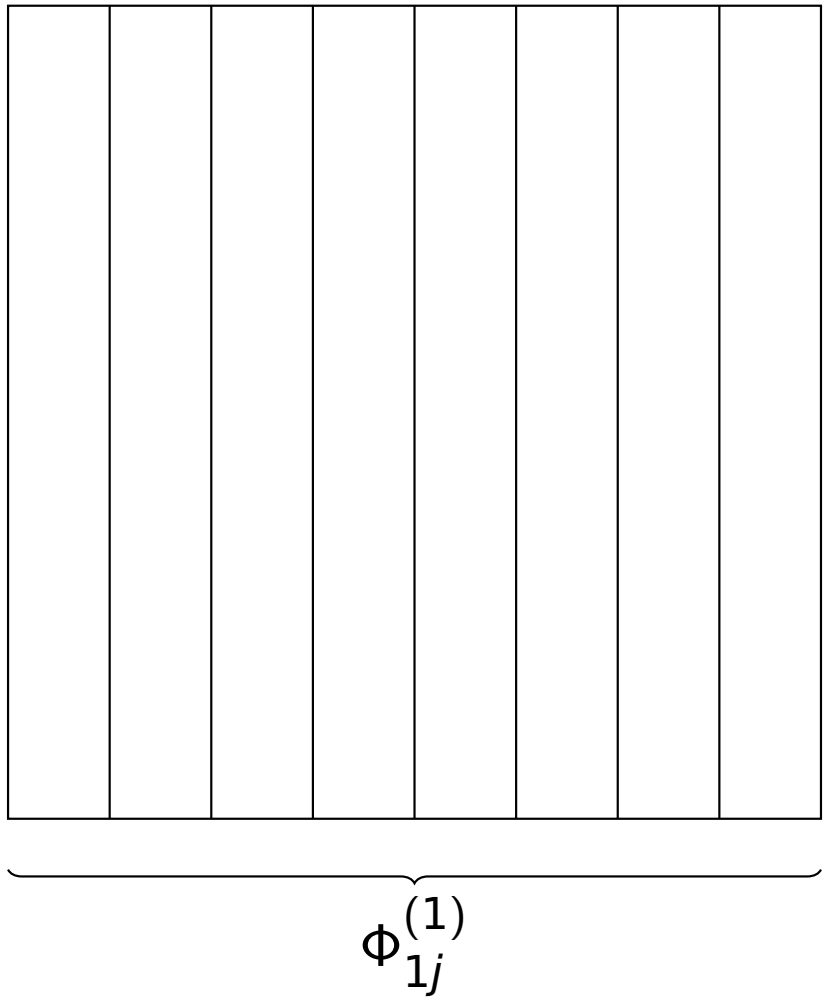


Space Tree

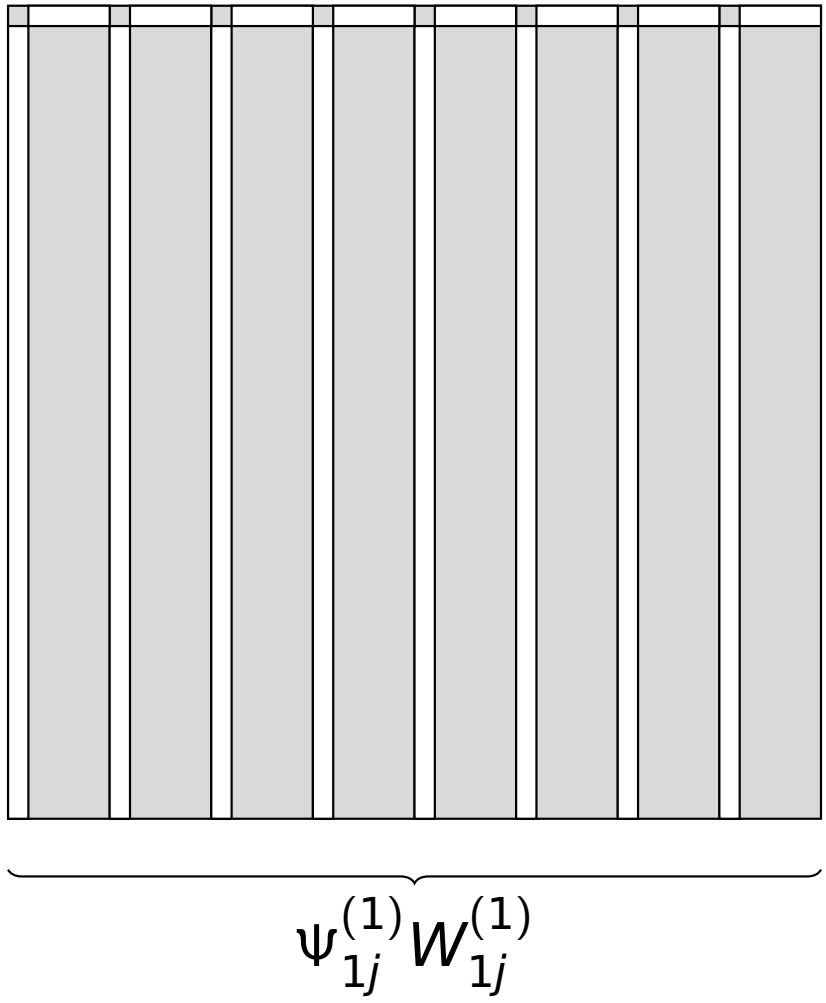


$\Phi_{11}^{(3)}$
$\Phi_{21}^{(3)}$
$\Phi_{31}^{(3)}$
$\Phi_{41}^{(3)}$
$\Phi_{51}^{(3)}$
$\Phi_{61}^{(3)}$
$\Phi_{71}^{(3)}$
$\Phi_{81}^{(3)}$

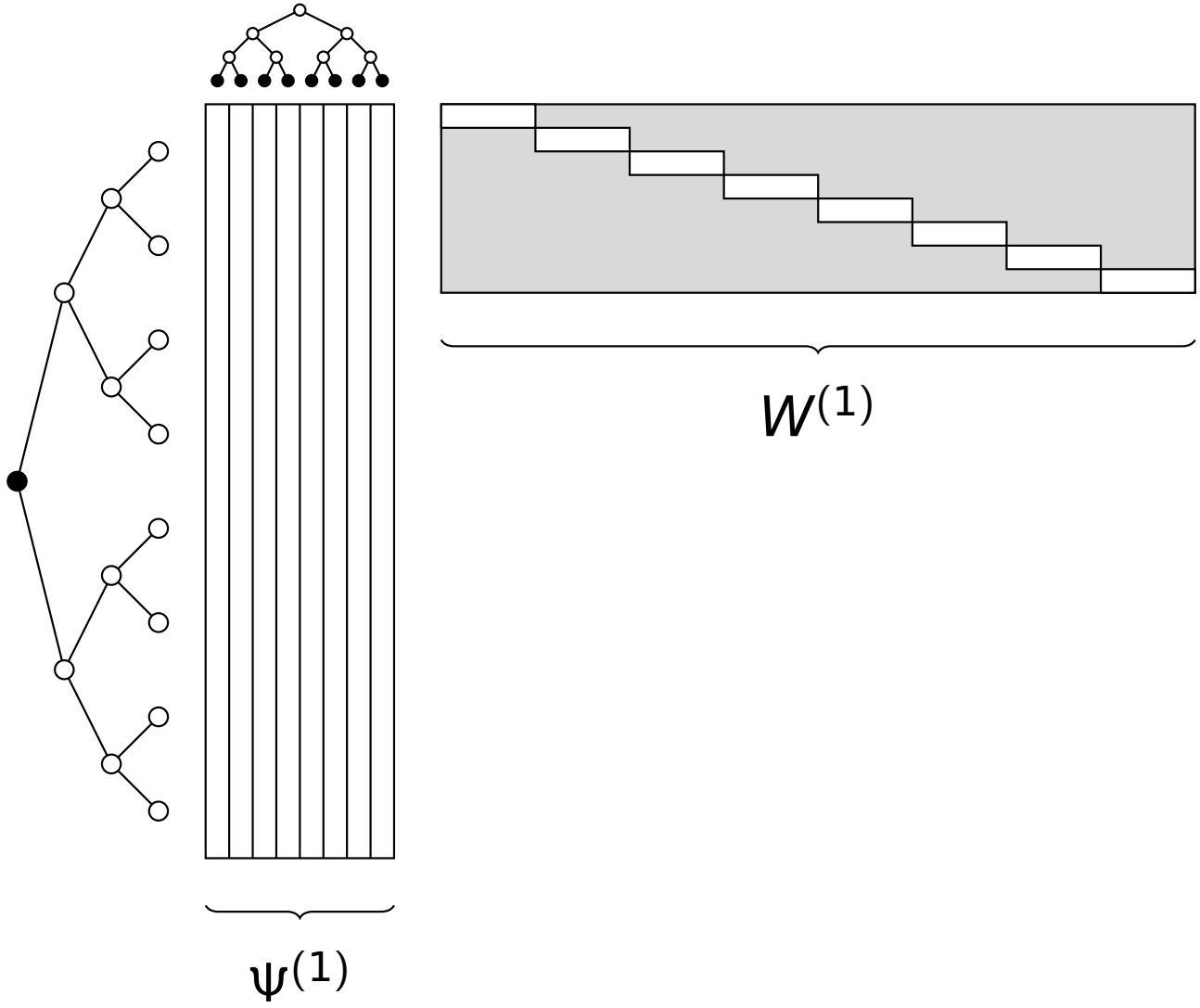
How to compute a column-wise BF



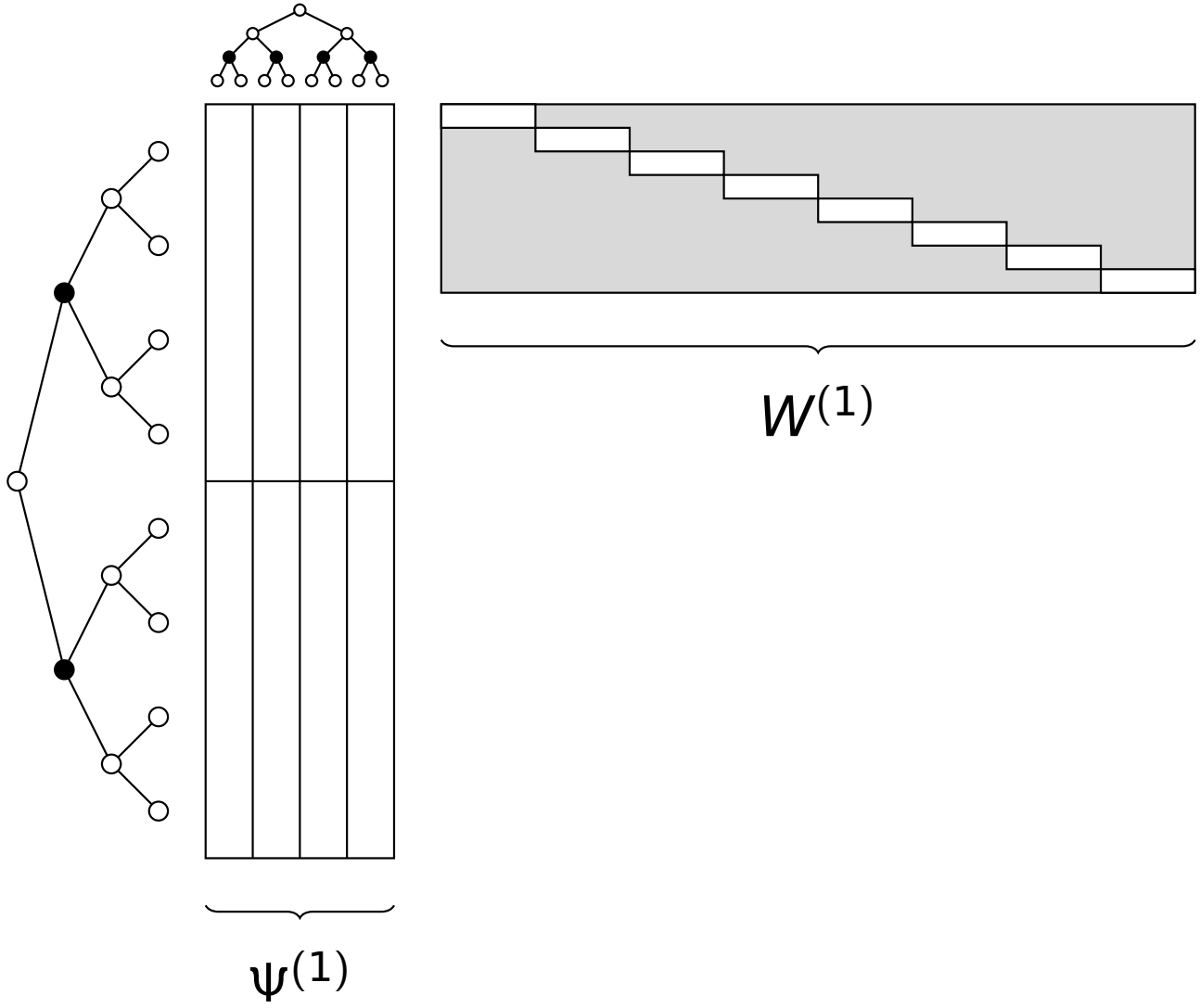
How to compute a column-wise BF



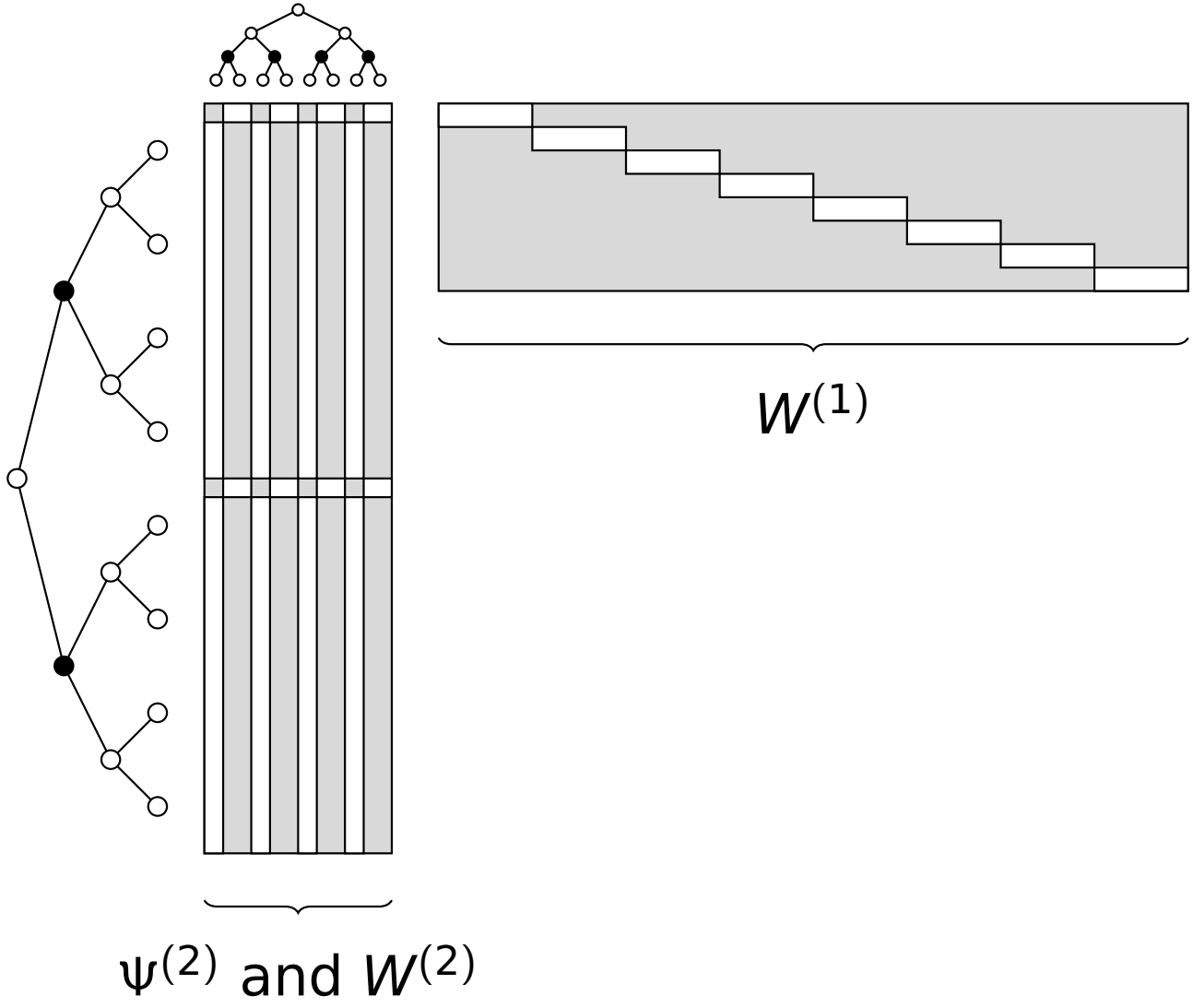
How to compute a column-wise BF



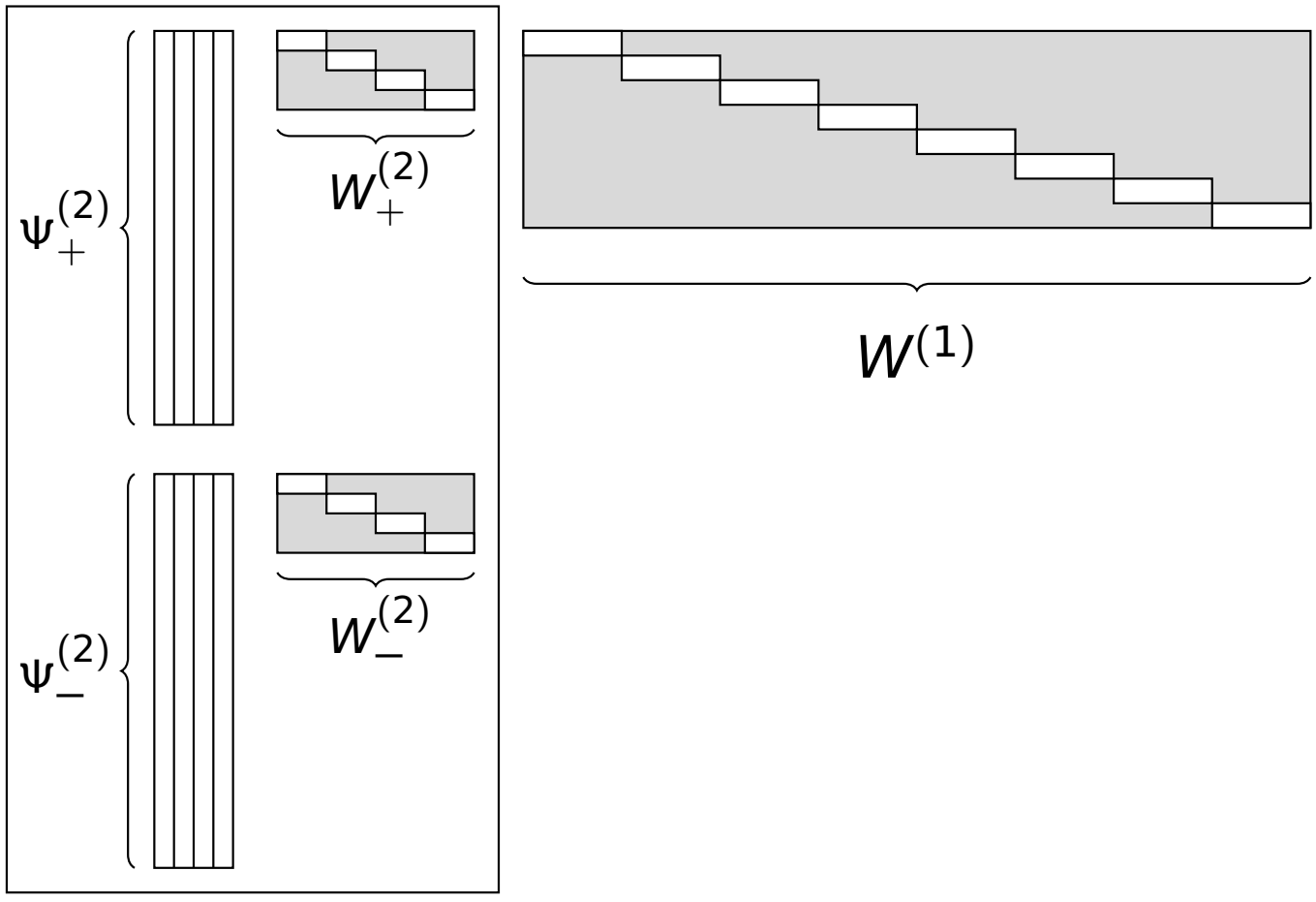
How to compute a column-wise BF



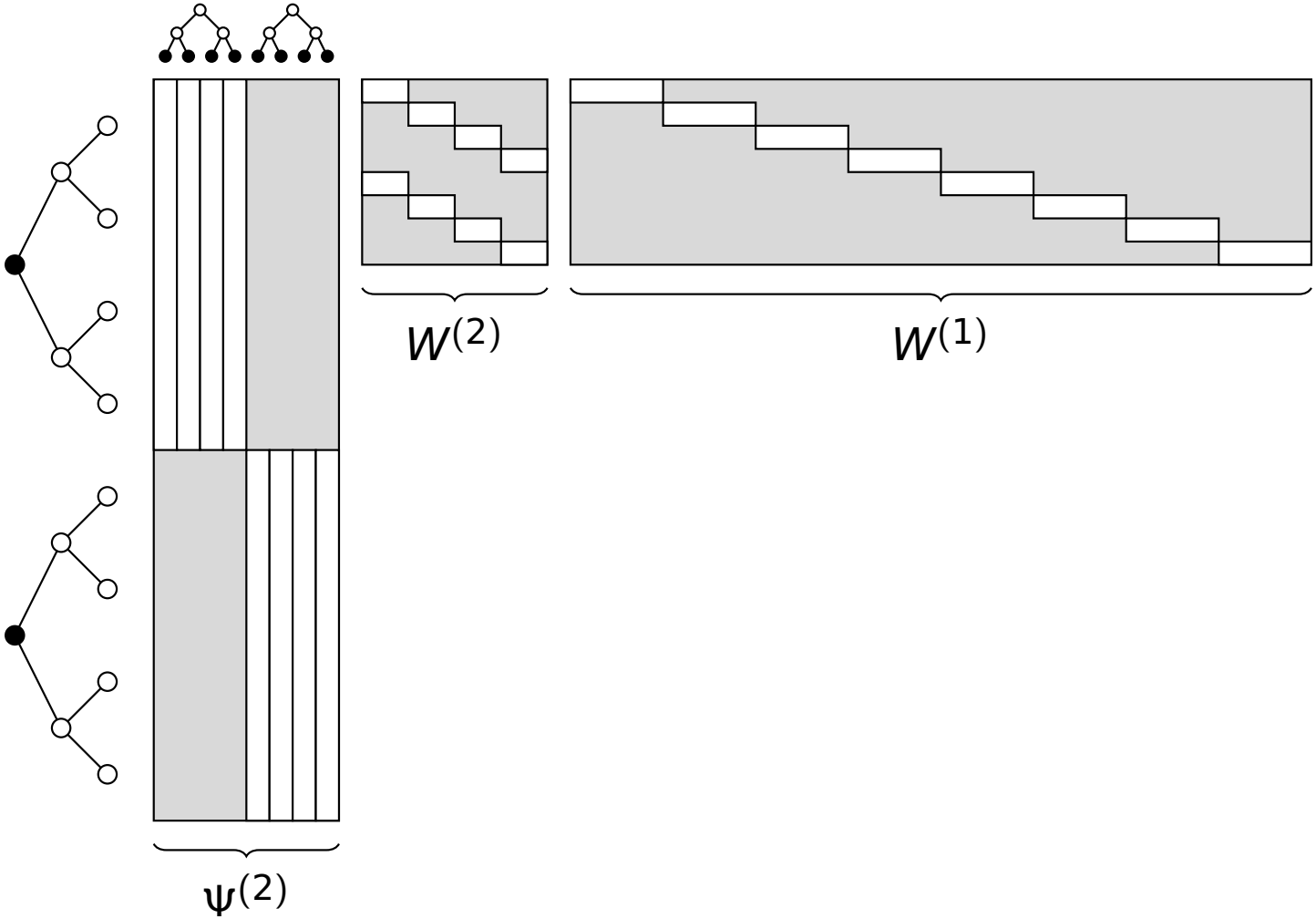
How to compute a column-wise BF



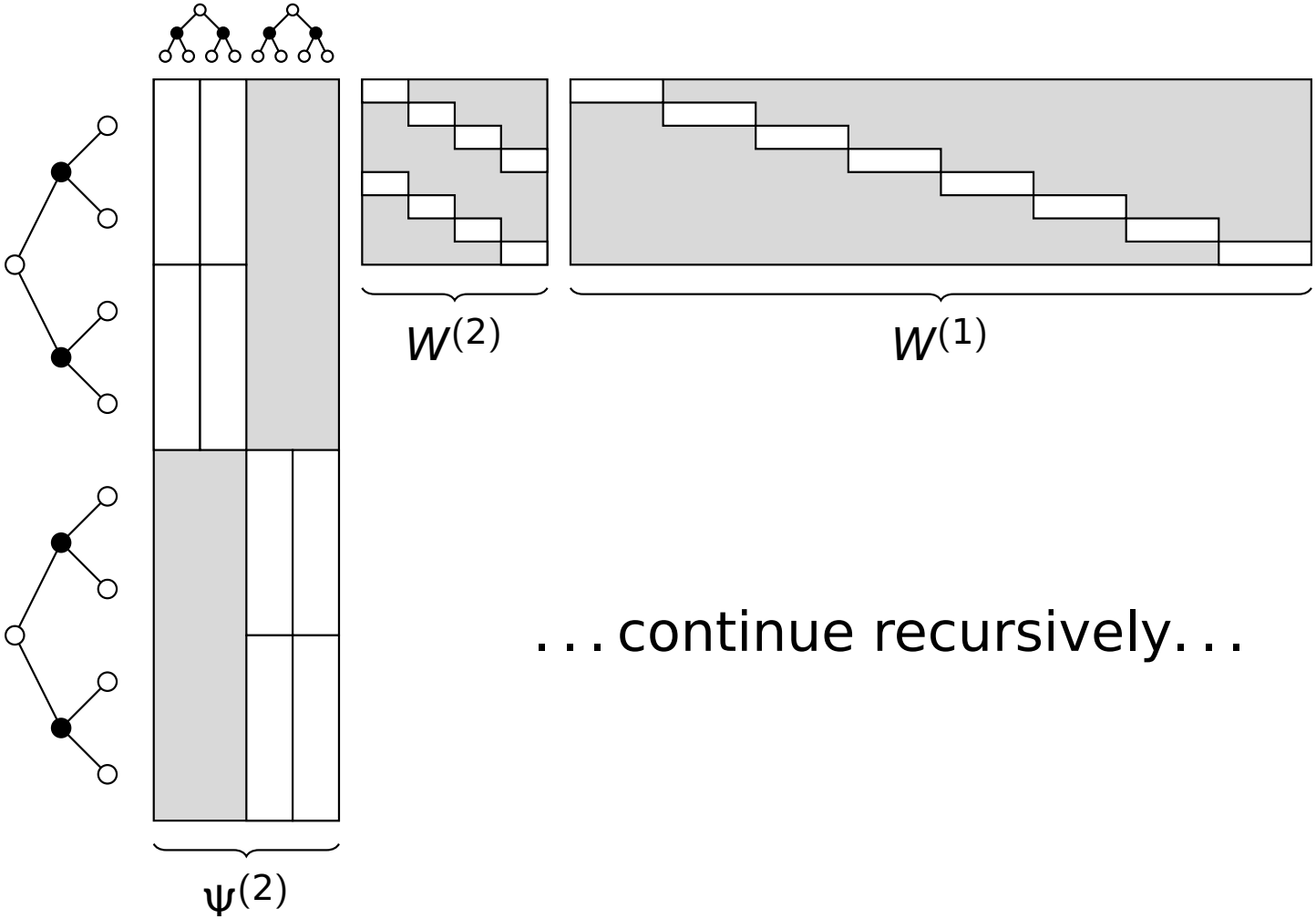
How to compute a column-wise BF



How to compute a column-wise BF

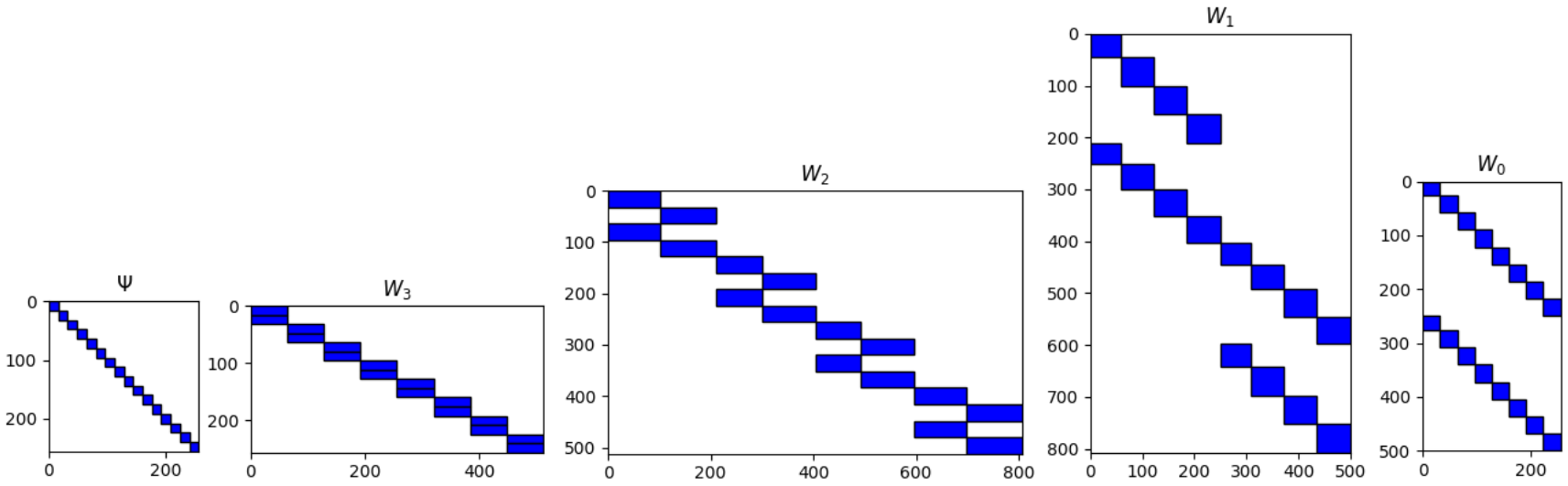


How to compute a column-wise BF



. . . continue recursively. . .

Example of a butterfly factorization



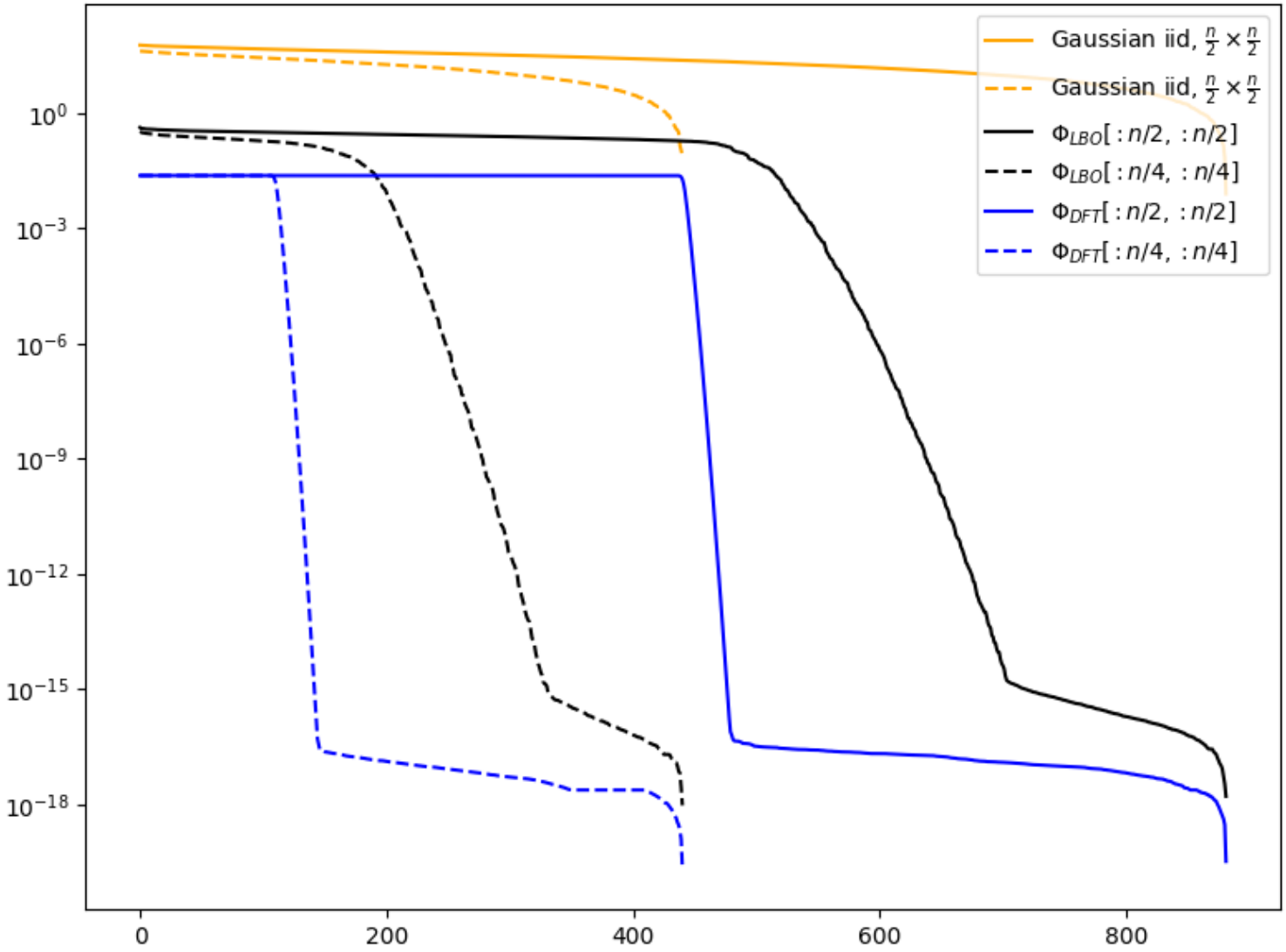
Butterfly factorization of 128×128 discrete Legendre transform:

$$\Phi \approx \Psi^{(3)} W^{(3)} W^{(2)} W^{(1)} W^{(0)}, \qquad \Phi_{ij} = P_j(x_i)$$

Bad choice here because of inner factor size blow-up—butterfly compressed matrix is ~2 larger than original. . .

Can we butterfly surface harmonics?

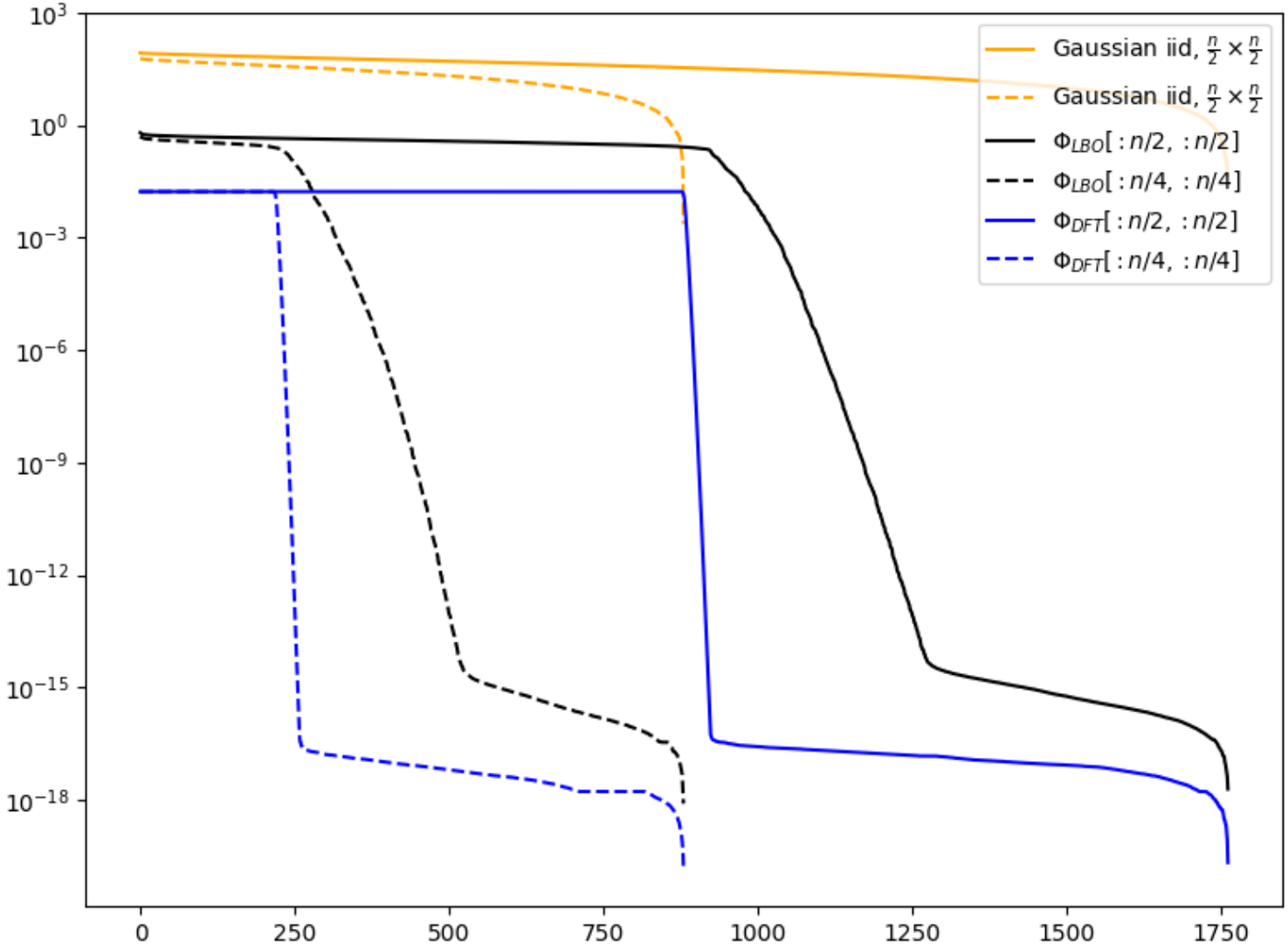
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **1,764** vertices)

Can we butterfly surface harmonics?

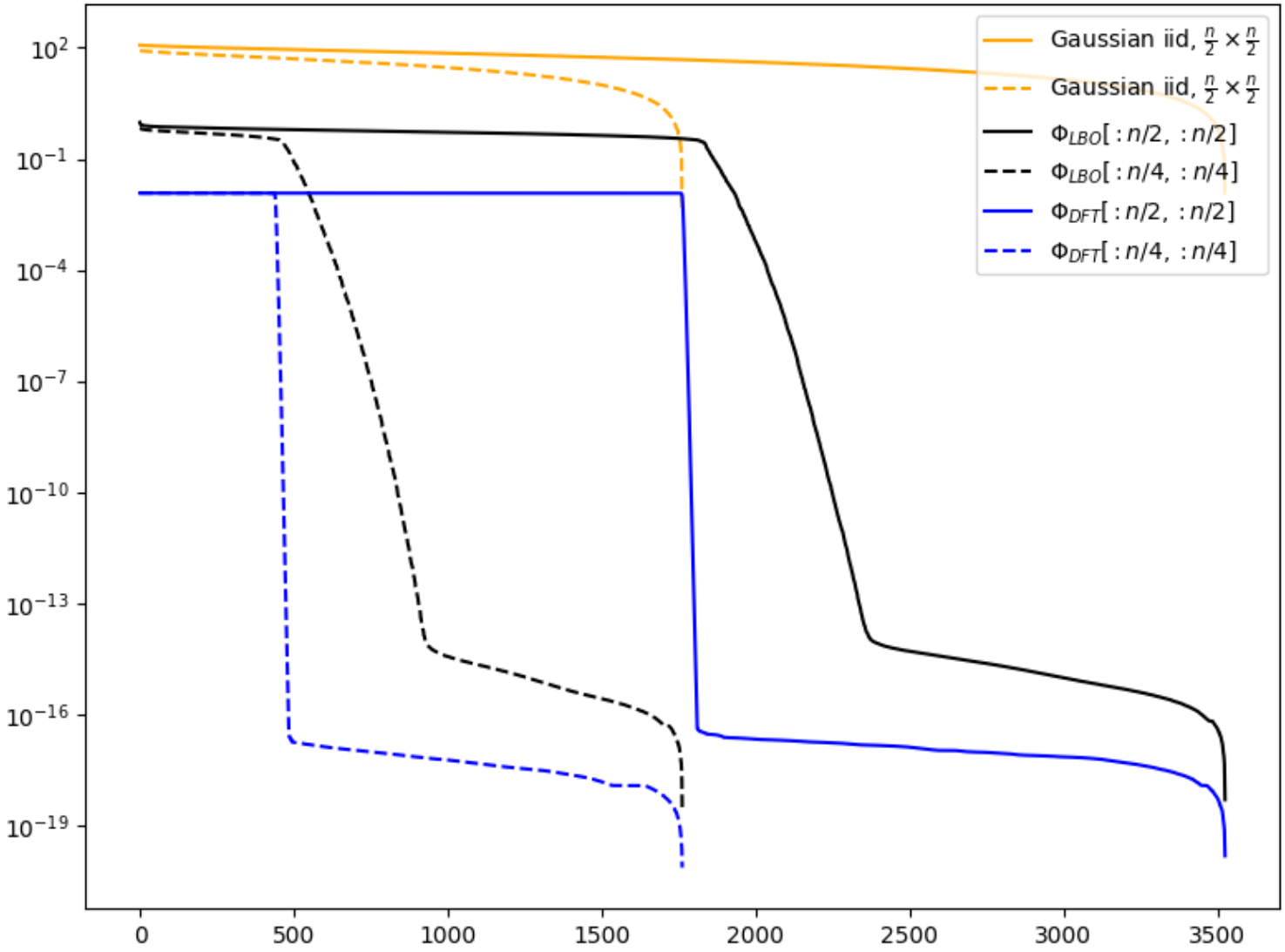
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **3,526** vertices)

Can we butterfly surface harmonics?

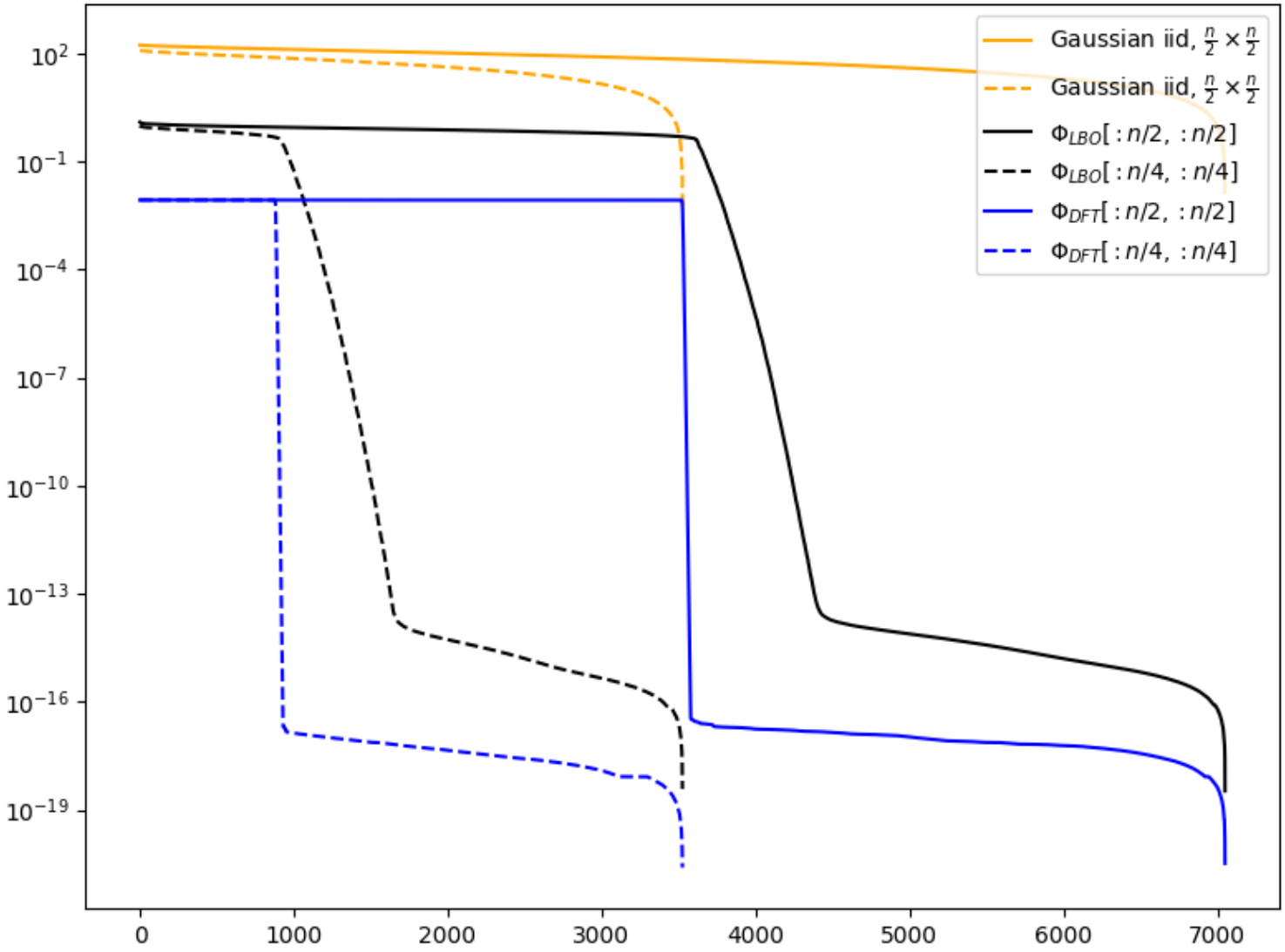
Surface harmonics are *weakly* butterfliable



(“Asian dragon” mesh w/ **7,051** vertices)

Can we butterfly surface harmonics?

Surface harmonics are *weakly* butterfliable



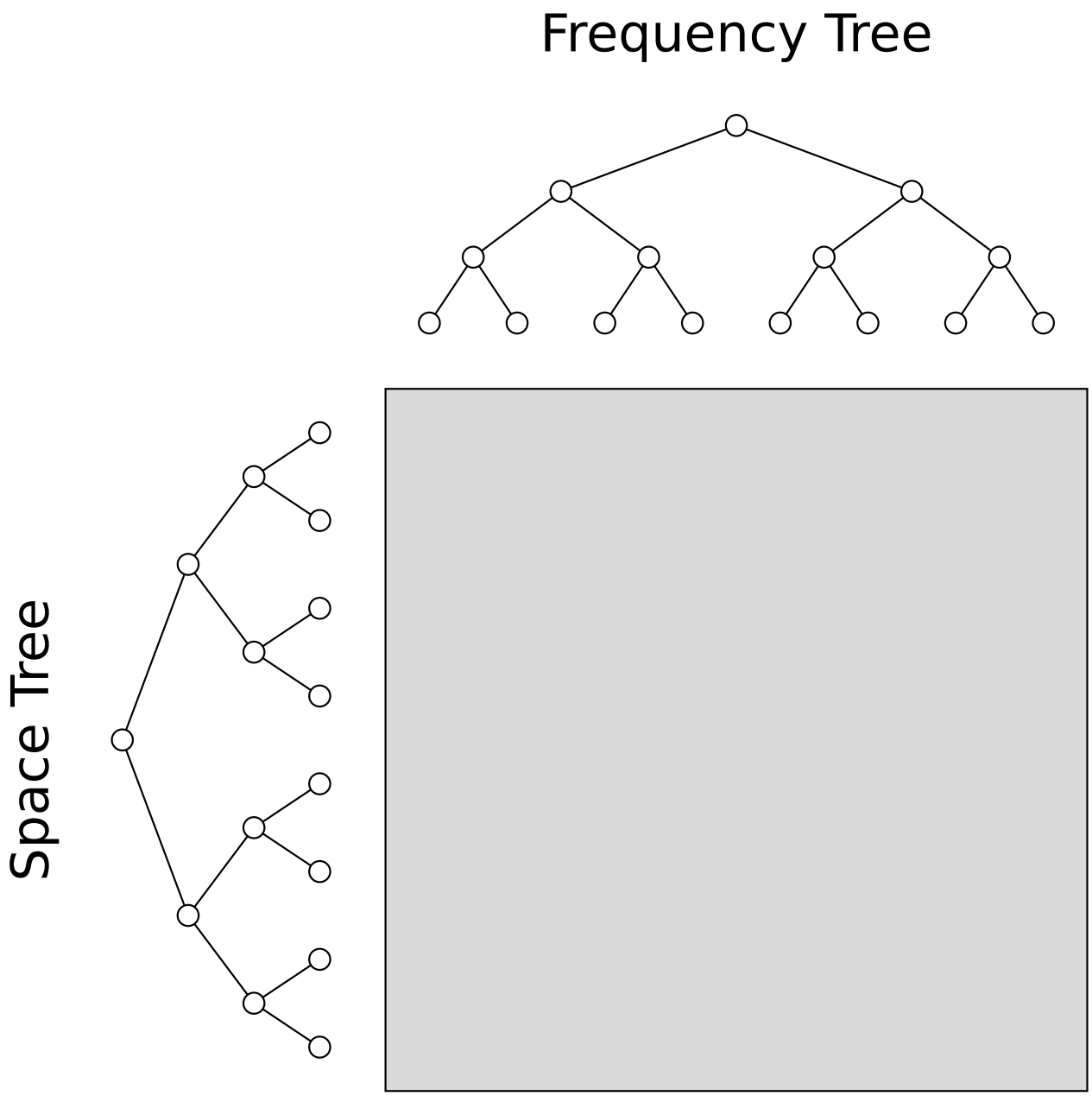
("Asian dragon" mesh w/ **14,101** vertices)

Can we butterfly surface harmonics?

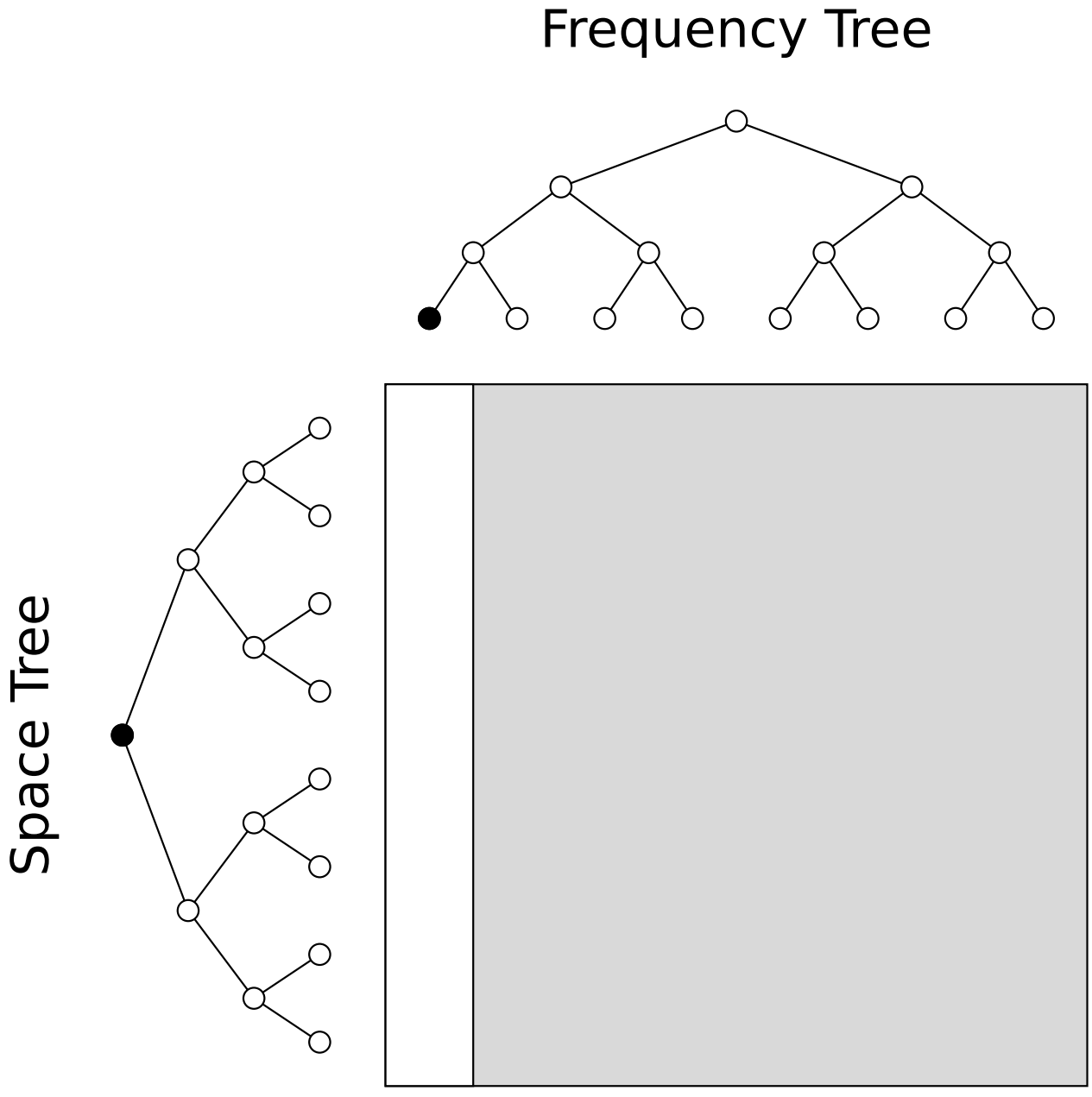
- **Note**: $\omega^2 = \lambda$
- Build tree on uniform grid in frequency—this is a **nonuniform** tree on the eigenvalues
- **Recall**: eigenfunctions of Laplacian on unit sphere are the degree p and order m spherical harmonics Y_p^m and corresponding eigenvalues are $-p(p+1)$
- There are $2p+1$ spherical harmonics of order p...
- More and more eigenfunctions per freq. cell...
- Behavior for LBO on genus-0 surface is similar

Yes—but we can't expect $O(n \log n)$ complexity because of Weyl law...

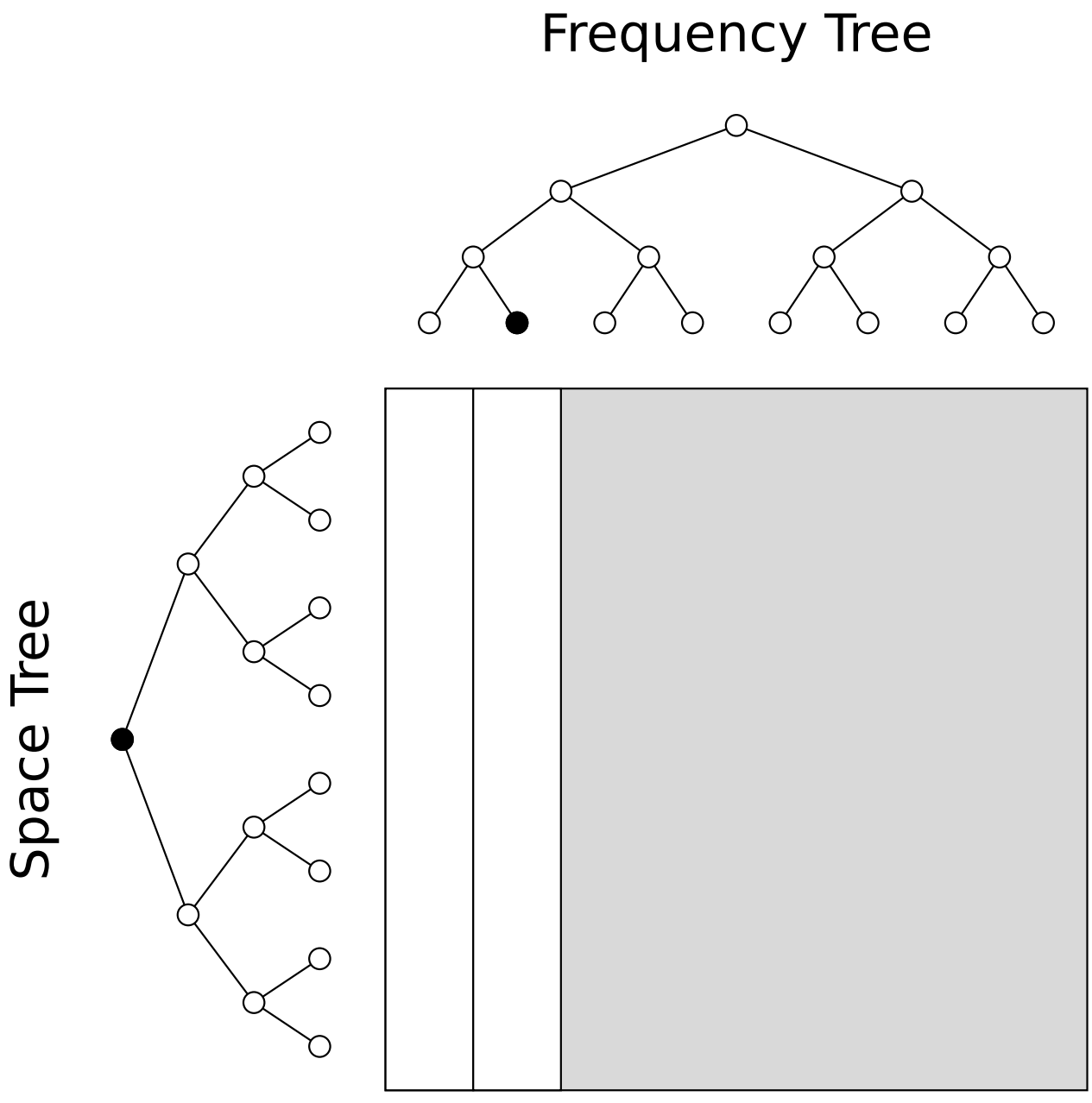
Streaming a butterfly factorization



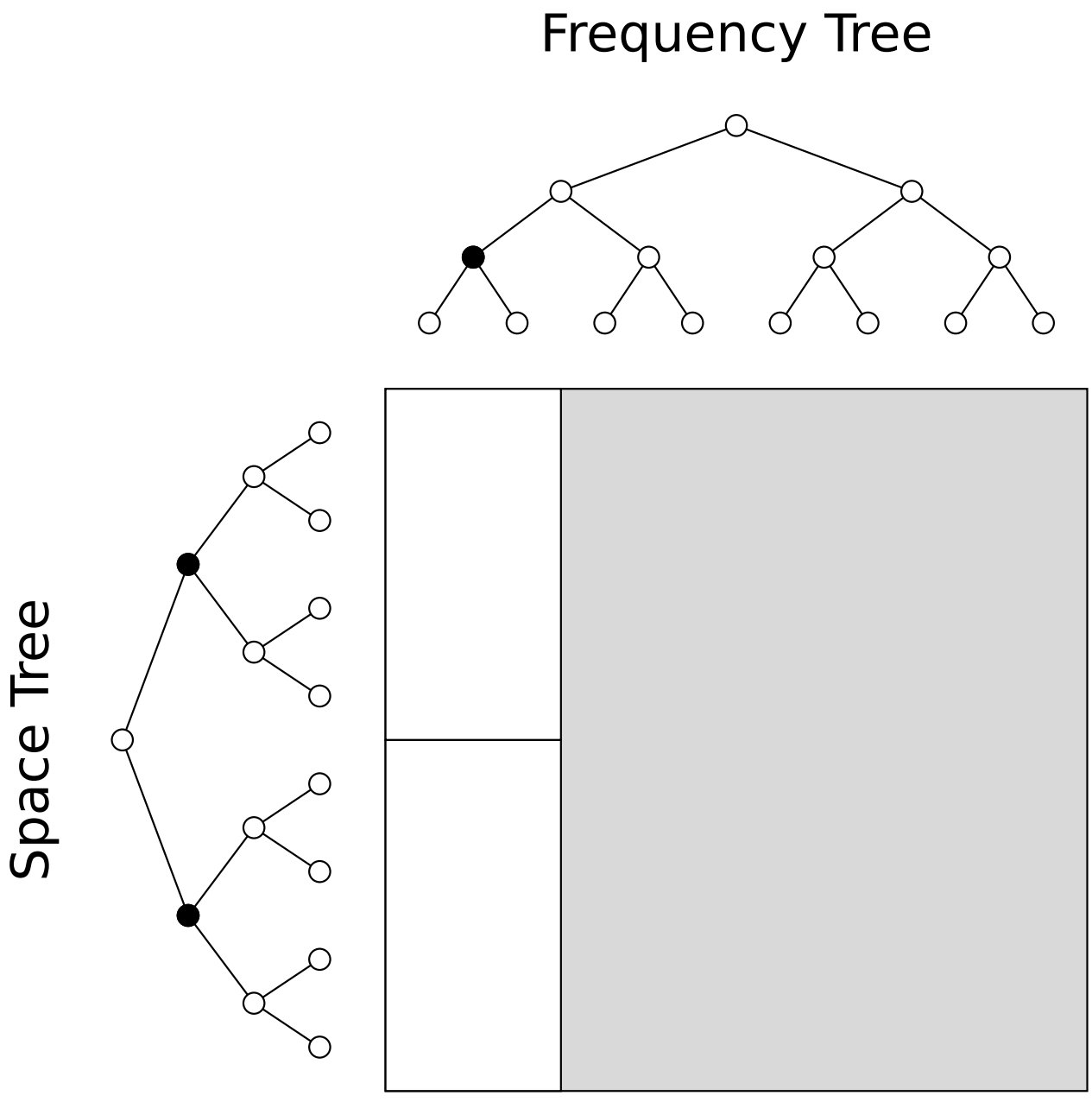
Streaming a butterfly factorization



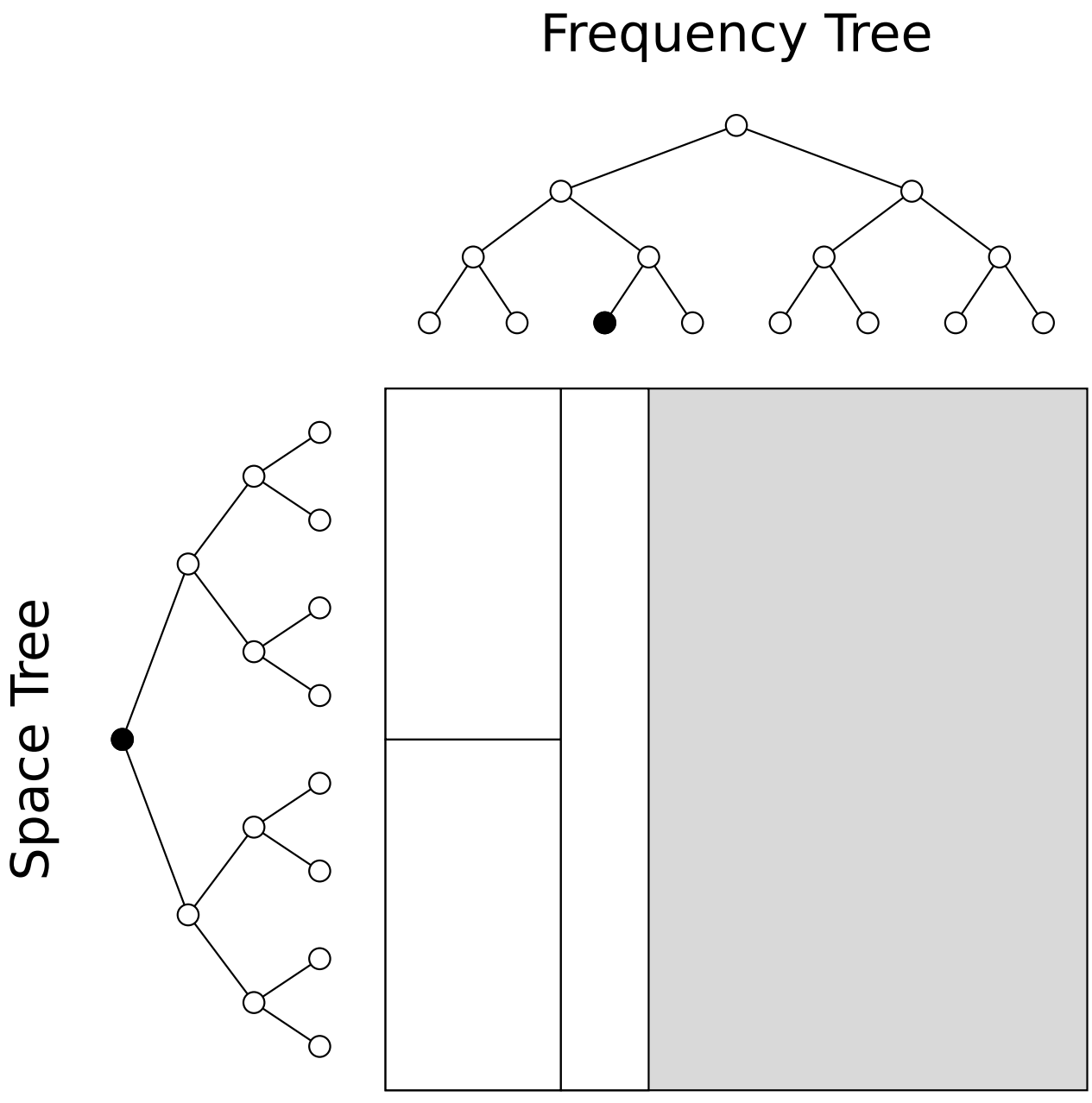
Streaming a butterfly factorization



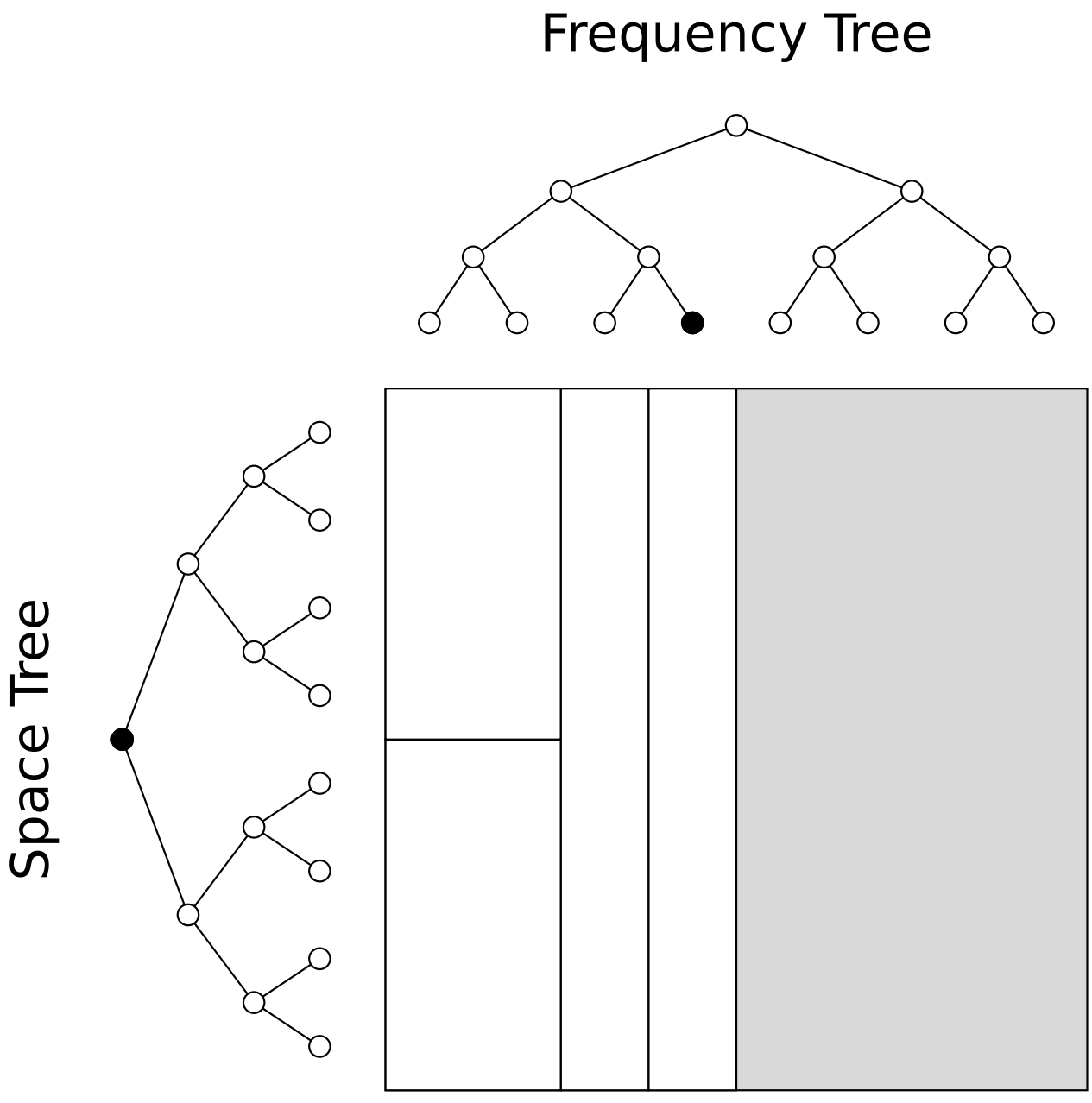
Streaming a butterfly factorization



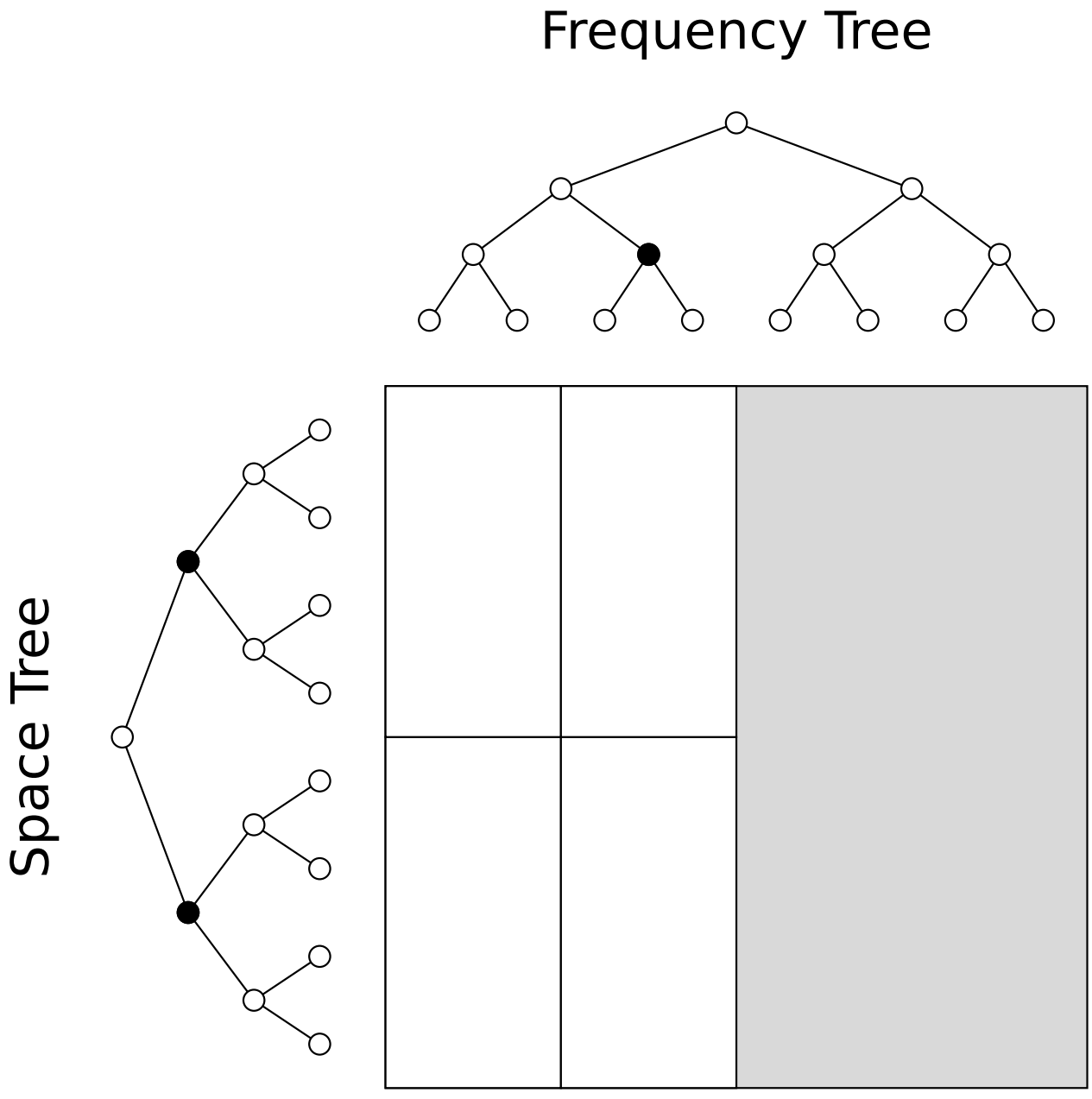
Streaming a butterfly factorization



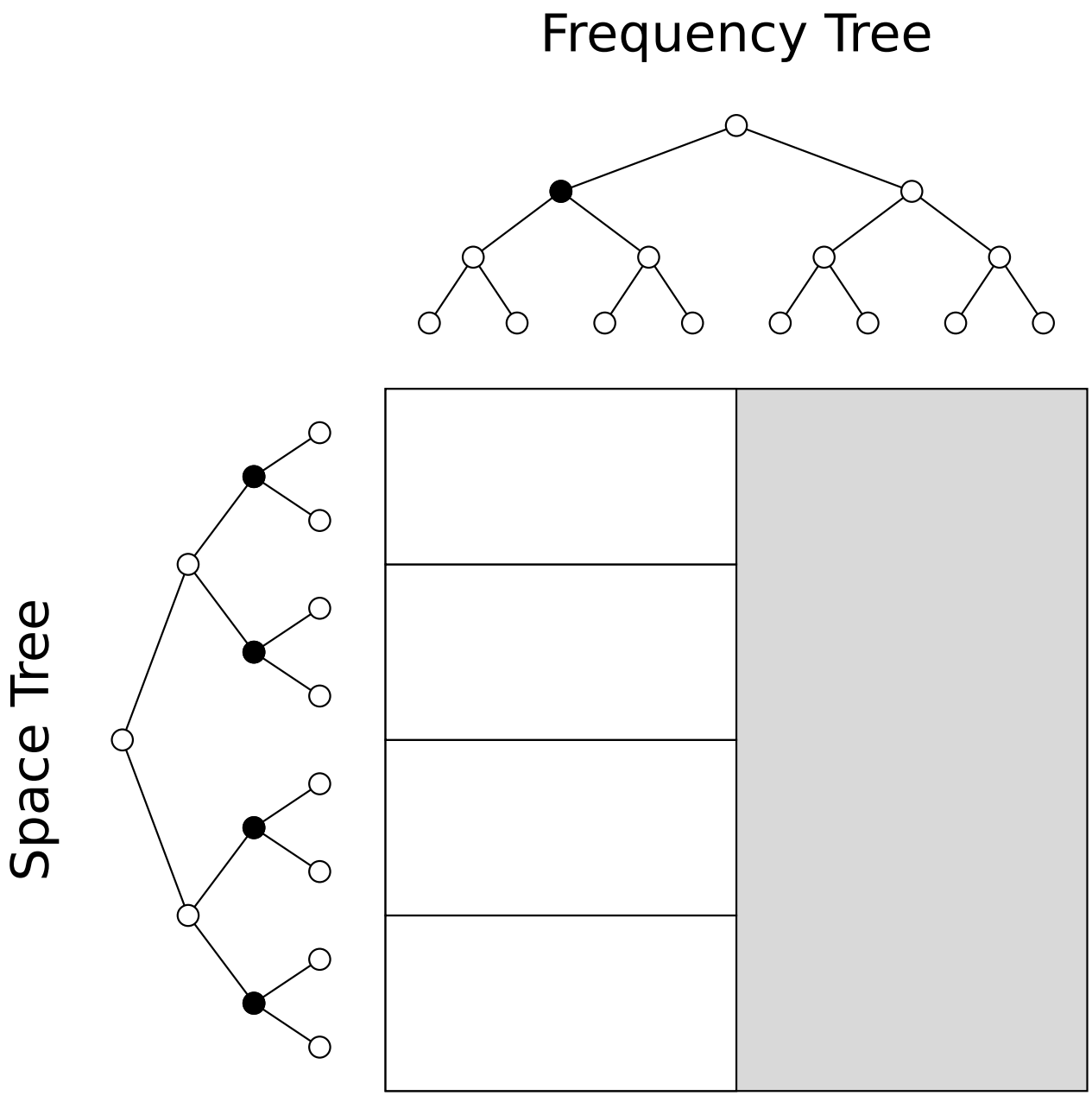
Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming a butterfly factorization



Streaming eigenbands

- Cheap to compute $\lambda_{\min}$ and $\lambda_{\max}$.
- Solve shifted generalized eigenvalue problem:

$$(L - \mu M)u = (\lambda - \mu)Mu$$

  to recover eigenvalues nearest μ first.
- Set up grid of eigenvalues:

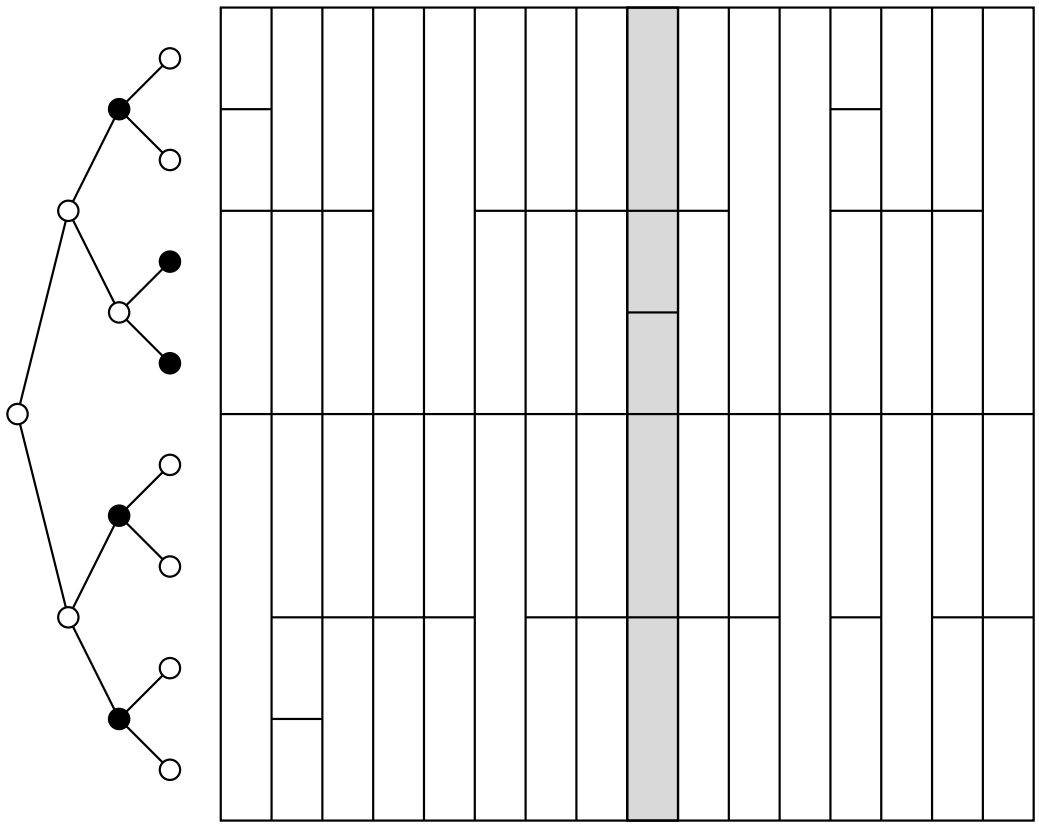
$$\tilde{\lambda}_0 = \lambda_{\min} < \tilde{\lambda}_1 < \cdots < \tilde{\lambda}_{N-1} < \tilde{\lambda}_N = \lambda_{\max}$$

- **Algorithm:** for each interval $[\tilde{\lambda}_{j-1}, \tilde{\lambda}_j)$, guess number of eigs. in band: $k > 0$. Compute first k eigenpairs. Finished if eigenvalues cover the band. Otherwise, set $k \to 2k$ and try again.

Compute p eigenpairs in roughly $O(np)$ time. Can be used to stream bands one at a time.

Adaptive butterfly factorization

For each column, find the highest cut through the spatial tree such that each subblock can be truncated with the desired precision.



Adaptive butterfly factorization

For each column, find the highest cut through the spatial tree such that each subblock can be truncated with the desired precision.

$$\begin{bmatrix} \Phi_1 W_1 & \Phi_2 W_2 \\ & \Phi_3 W_3 \end{bmatrix}$$

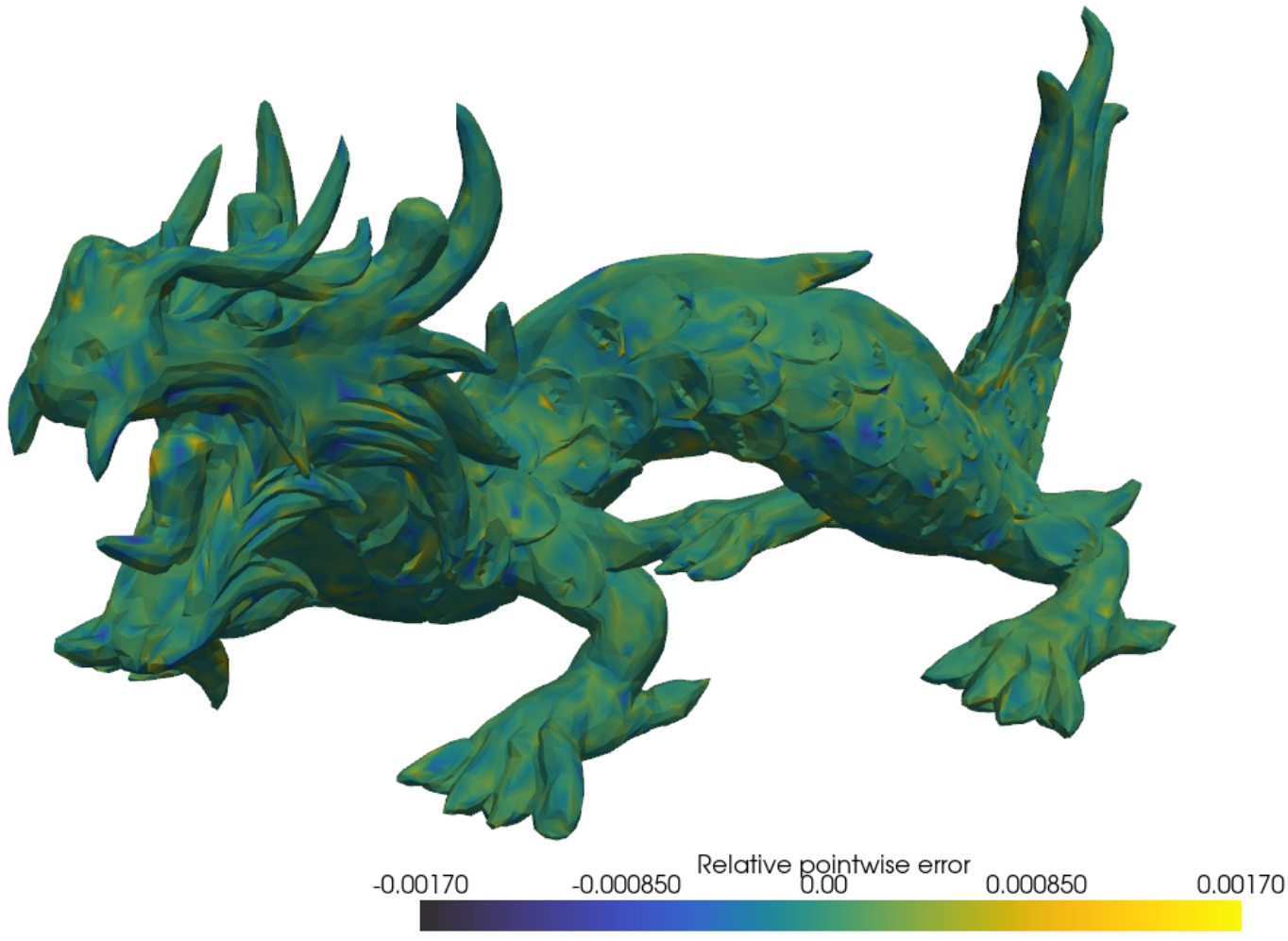
$$= \left[\begin{bmatrix} \Phi_1^+ \\ \Phi_1^- \end{bmatrix} W_1 \quad \begin{bmatrix} \Phi_2 & \\ & \Phi_3 \end{bmatrix} \begin{bmatrix} W_2 \\ W_3 \end{bmatrix} \right]$$

$$= \begin{bmatrix} \Phi_1^+ & \Phi_2 & \\ \Phi_1^- & & \Phi_3 \end{bmatrix} \begin{bmatrix} W_1 & & \\ & W_2 & \\ & & W_3 \end{bmatrix}$$

Need to take extra care to preserve sparsity with adaptive factorizations. (Foreshadowing: our library makes this is easier...)

Numerical results

For low-rank factorizations, use truncated SVD, keeping singular vectors where $\sigma_k \geq \epsilon \sigma_1$.



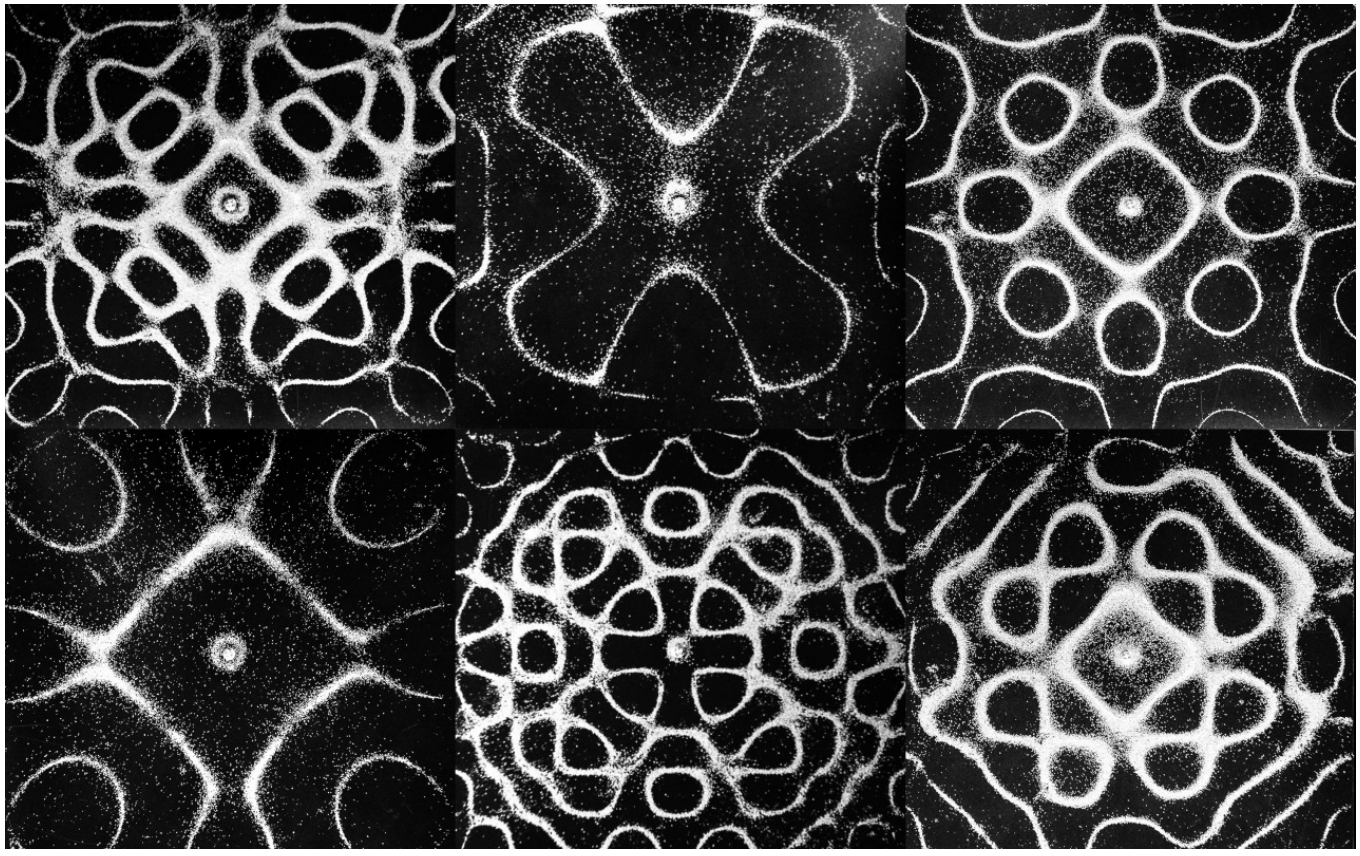
Numerical results

Results for adaptive BF with relative error $\epsilon = 10^{-3}$, computing lowest $\frac{1}{16}$th of spectrum.

Verts	Eigs	Orig	BF	Ratio	Order
1,764	110	1.5 MB	0.9 MB	1.67	
3,526	220	5.9 MB	3.0 MB	1.97	1.74
7,051	440	23.7 MB	9.7 MB	2.44	1.69
14.1K	881	94.8 MB	30.2 MB	3.14	1.64
28.2K	1,762	379.1 MB	92.8 MB	4.09	1.62
56.4K	3,524	1.5 GB	278.7 MB	5.44	**1.59**
113K	7,049	6.0 GB	863.7 MB	7.02	1.63
225K	14.1K	**24.2 GB**	**2.5 GB**	9.42	**1.58**

Complexity possibly $O(n^{\log_2(3)})$ ($\log_2(3) \approx 1.585\ldots$)?

Current work: the Fiedler tree



Courant nodal domain theorem: zero level set of kth eigenfunction of Laplace-Beltrami operator partitions Ω into at most k pieces ("nodal domains").

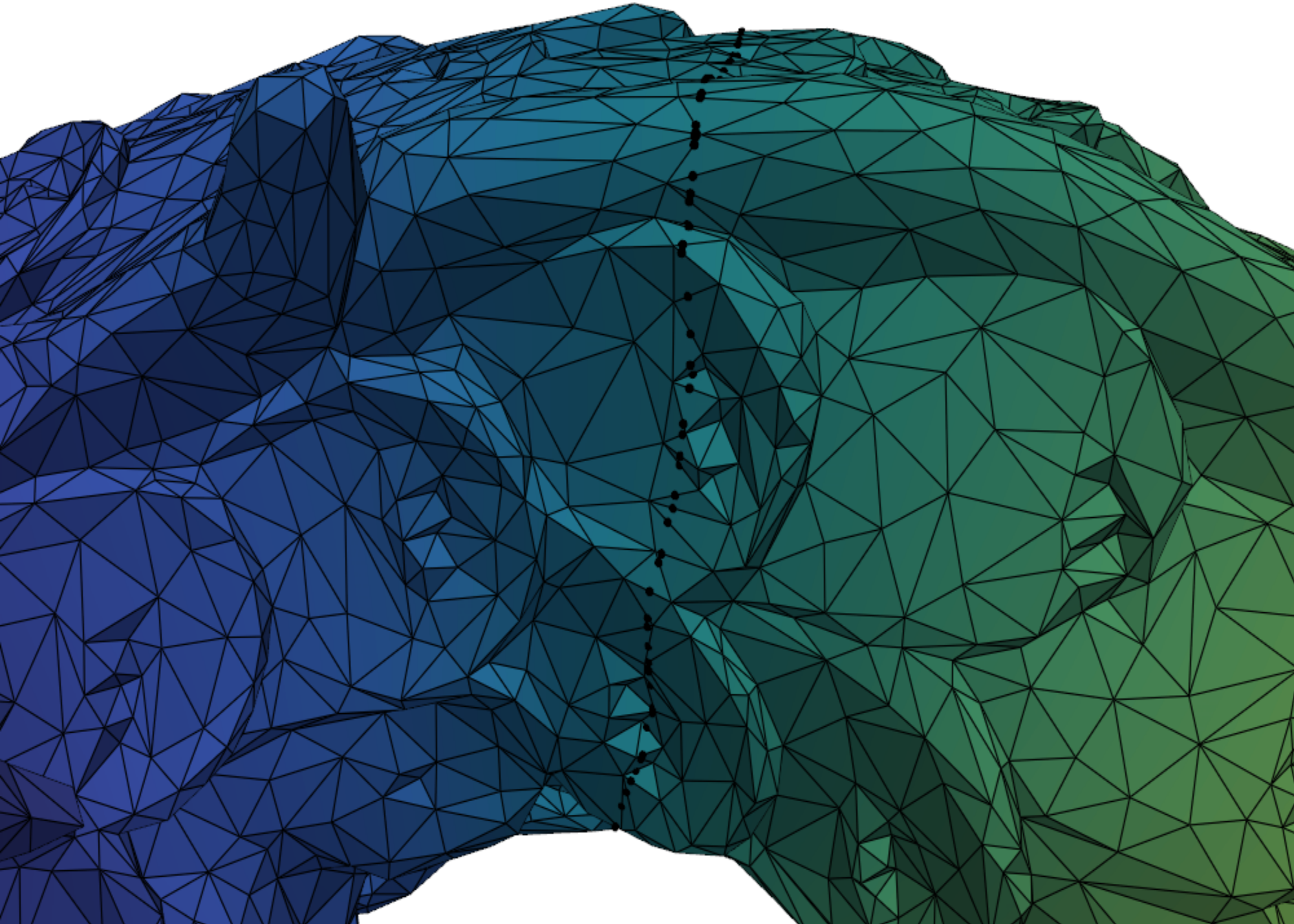
Source: http://dataphys.org/list/chladni-plates/

Current work: the Fiedler tree

Fiedler vector is the eigenvector for first nonzero eig.
Zero level set separates into two nodal domains.

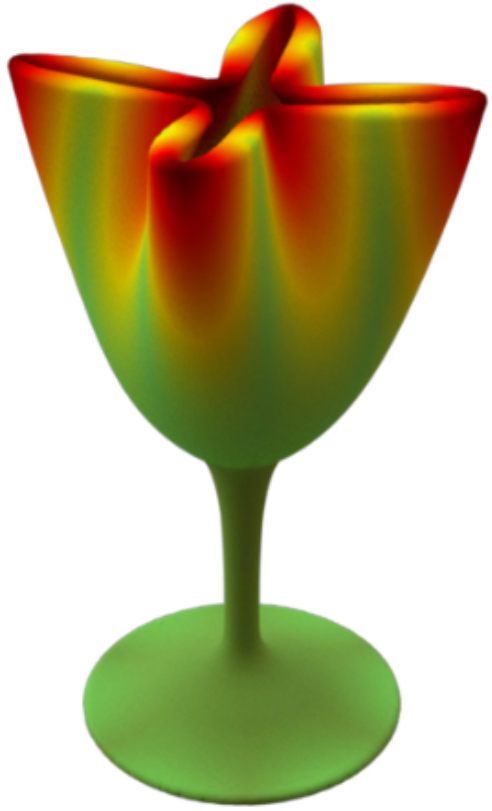
Current work: the Fiedler tree

So far have observed better compression than with octree, but need to insert new vertices into the mesh.



Application: acoustic mode compression

Mode 3, 1.9 kHz



Mode 15, 9.3 kHz

Mode 33, 16 kHz

Source: Langlois et al., “Eigenmode Compression for Modal Sound Models”

Application: eigenfluids

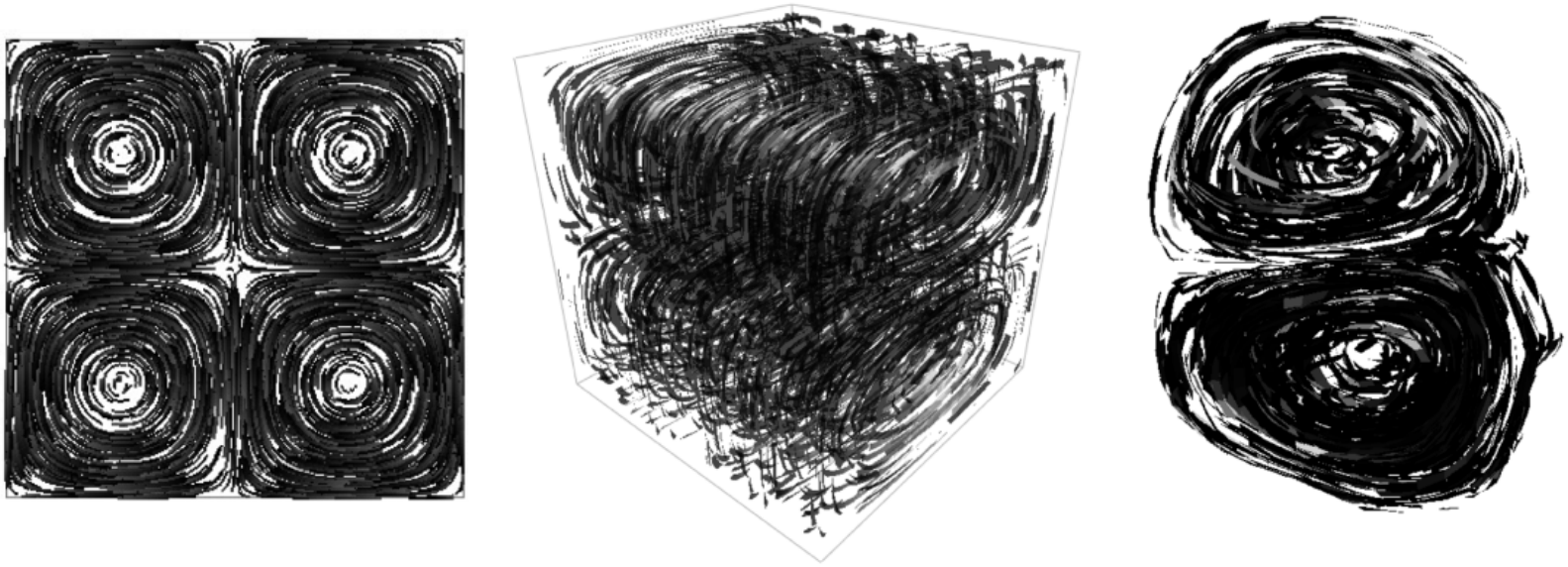


Fig. 1. Examples of Laplacian eigenfunction velocity basis fields for various domains. They are stationary solutions to the Navier-Stokes equations. For simple domains Laplacian eigenfunctions have closed-form expressions while for general meshes they are defined through eigendecomposition of a discrete Laplace operator.

Source: De Witt et al., “Fluid Simulation Using Laplacian Eigenfunctions”

Prospectus

- How to fully exploit the Courant nodal domain theorem?
- How to design optimal space trees for higher genus and higher dimension manifolds?
- The butterfly factorization is memory hungry—how to mitigate?
- Other applications: fractional PDEs, Gaussian processes

Butterfly-accelerated multiple scattering

Sound-soft acoustic scattering in 2D

Solve Helmholtz equation with Dirichlet (sound-soft) BCs on $\Omega \subseteq \mathbb{R}^2$:

$$\begin{cases} \Delta u(x) + k^2 u(x) = 0, & x \in \Omega \\ u(x) = g(x), & x \in \partial\Omega \end{cases} \tag{1}$$

Seek solution of the form:

$$u = u_{\text{in}} + u_{\text{scat}}, \tag{2}$$

where u_{in} is an incident field which satisfies the Helmholtz equation.

Solution by CFIE representation

Green's function for 2D Helmholtz is:

$$G(x,y) = \frac{i}{4} H_0^{(1)}(k\|x-y\|). \tag{3}$$

Use the combined-field integral equation representation. Let solution be of the form:

$$u_{\text{scat}}(x) = \int_{\partial\Omega} \left[\alpha G(x,y) + \beta \frac{\partial G(x,y)}{\partial \nu(y)} \right] \sigma(y) dy, \tag{4}$$

where $\alpha = -ik$ and $\beta = -1$.

CFIE makes sound-soft scattering problem uniquely solvable and well-conditioned for all k.

Solution using CFIE representation

Satisfies boundary integral equation of the form:

$$\frac{1}{2}\sigma(x) + \int_{\partial\Omega} \left[\alpha G(x,y) + \beta \frac{\partial G(x,y)}{\partial \nu(y)}\right]\sigma(y)dy = -u_{\text{in}}(x). \quad (5)$$

Discretizing and solving this leads to a dense linear system of form:

$$\left(\frac{1}{2}I + K\right)\sigma = -u_{\text{in}}. \quad (6)$$

Solving this system iteratively using GMRES requires a fast multiply with K.

Normally, use the (HF-)FMM. We will present a multi-level BF and show numerical studies.

Motivation

- Can solve this problem using an iterative method, but probably need preconditioning.
- Alternatively, use a **fast direct solver**: build hierarchical matrix factorization of $(I/2 + K)^{-1}$.
- Many different approaches, but must have an efficient matrix factorization of blocks of K.
- As k becomes large, simple low-rank decomposition (e.g. SVD) of K is no longer optimal.

Use the butterfly factorization instead...

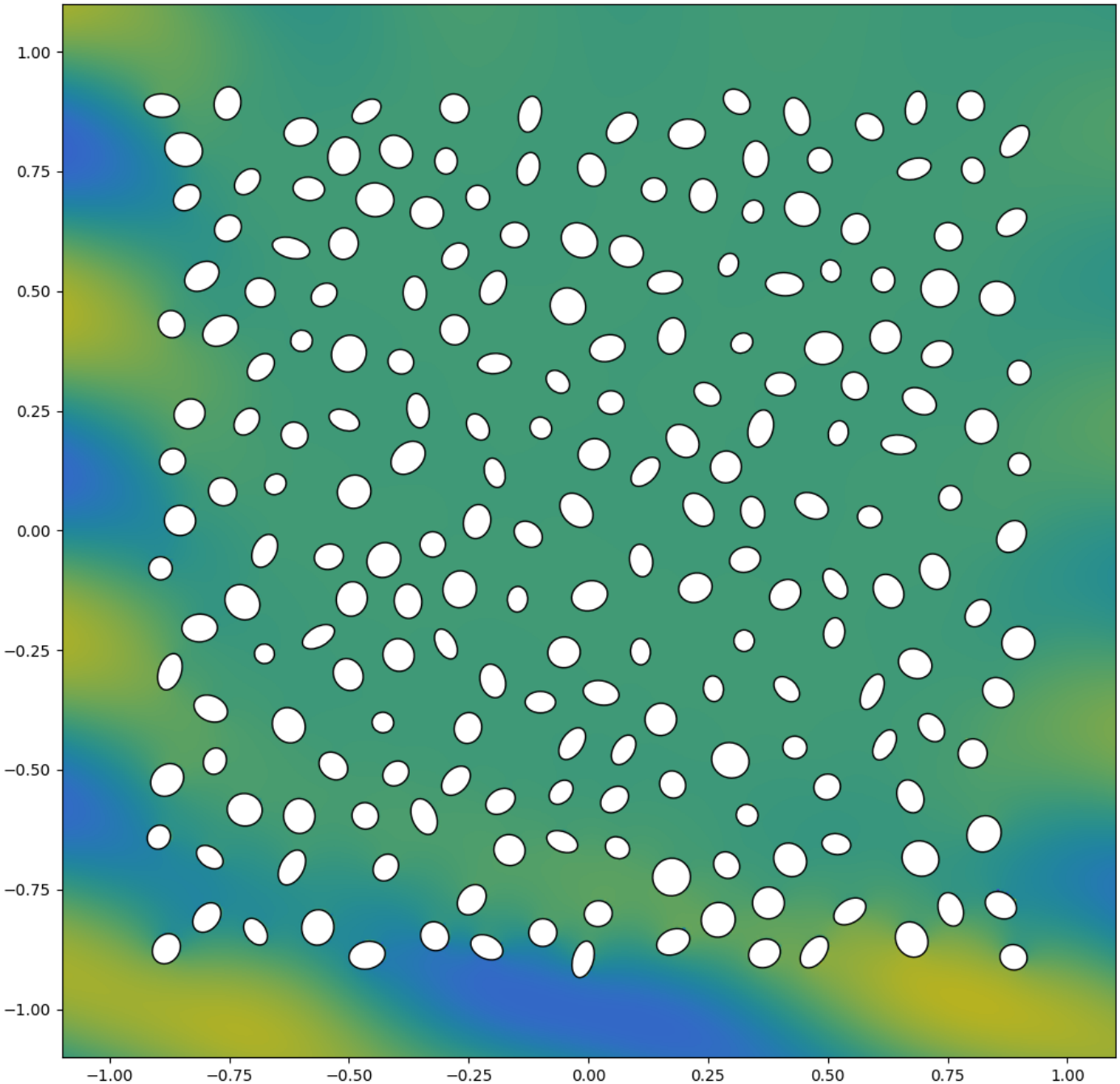
This talk: focus on the forward operator K.

Test problem

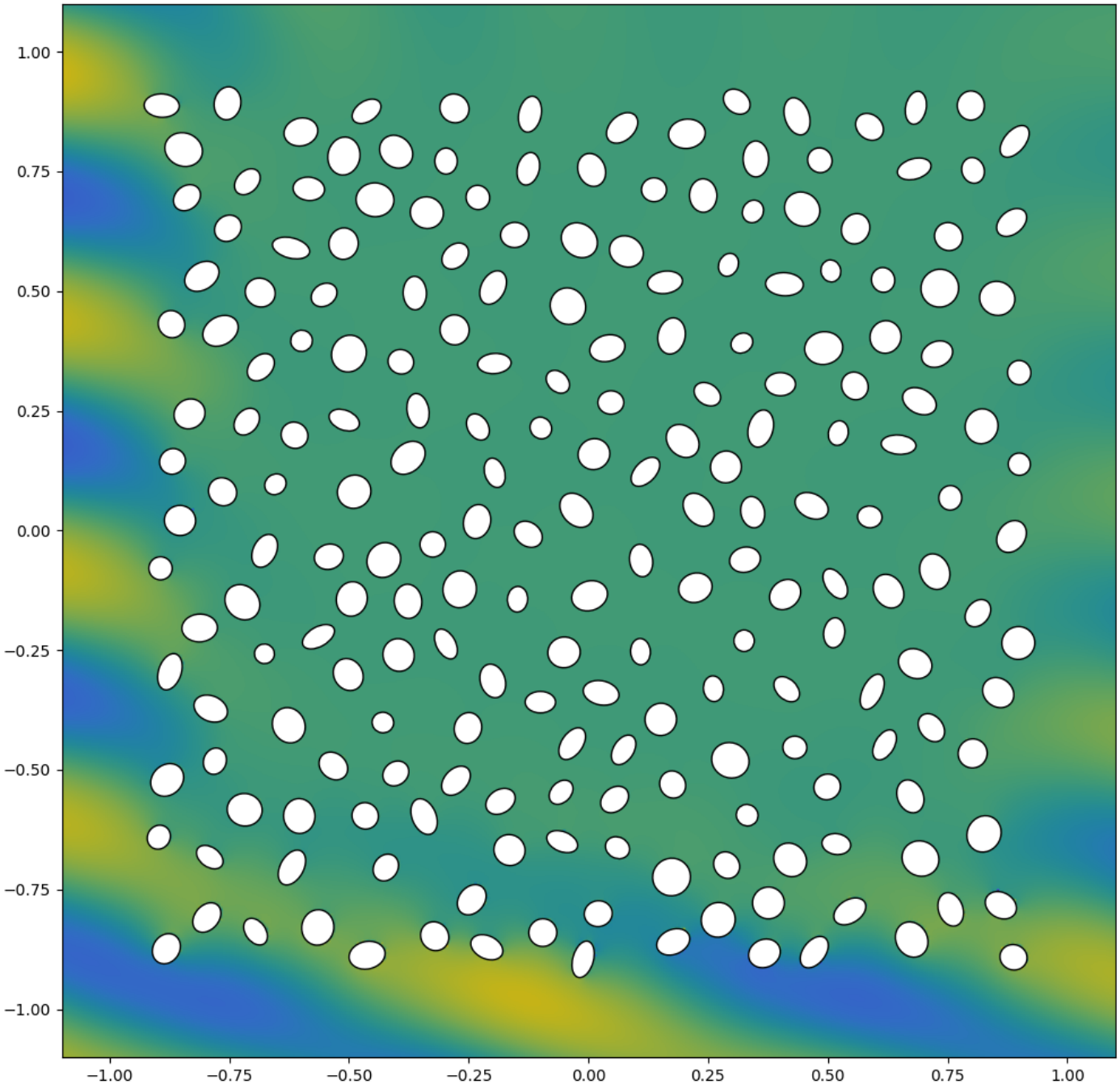
- Randomly sample nonoverlapping ellipses in the box $[-1,1] \times [-1,1]$ using Poisson disk sampling.
- Set $u_{\text{in}} = e^{ik\hat{d}\cdot x}$ for some choice of $\hat{d}$.
- Discretization: trapezoidal rule with 6th order Kapur-Rokhlin correction.
- Use a block Jacobi preconditioner to reduce number of GMRES iterations: for each ellipse, compute LU decomposition of corresponding block of K.

Our focus is on the relative **performance** (speed, memory) of the algorithms. **Not** the accuracy of solving this particular problem.

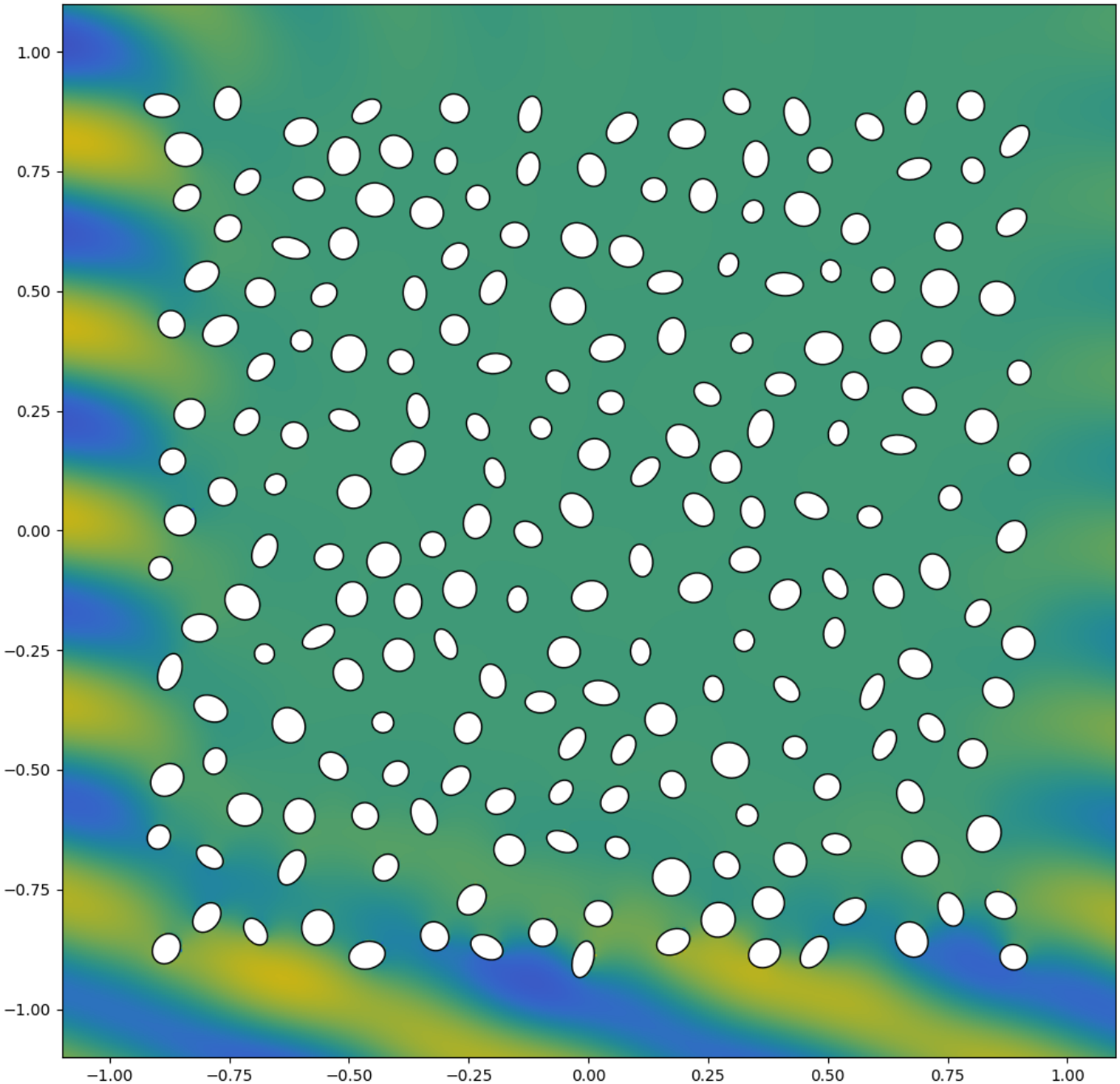
$k = 10$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

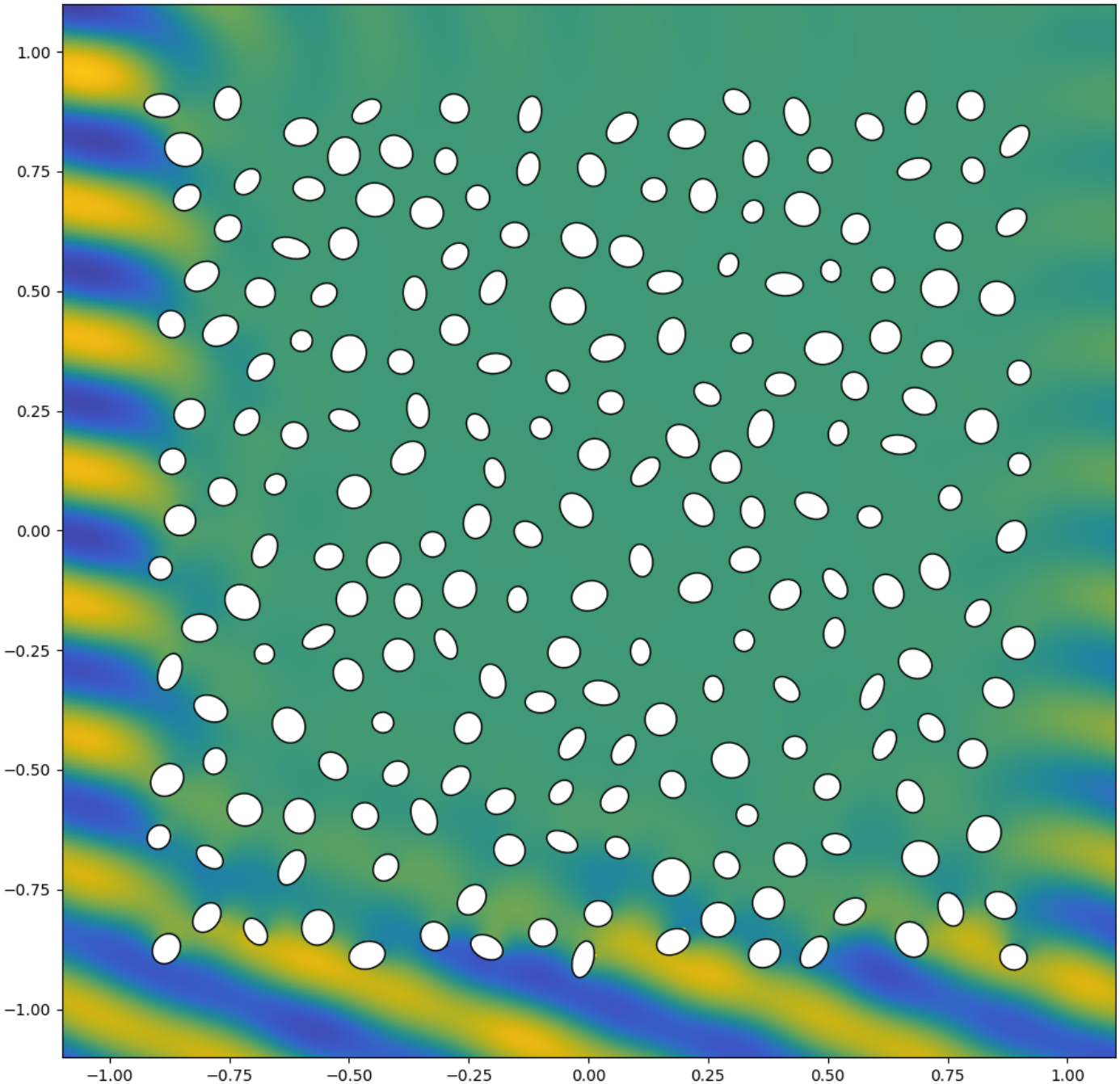
$k = 13$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

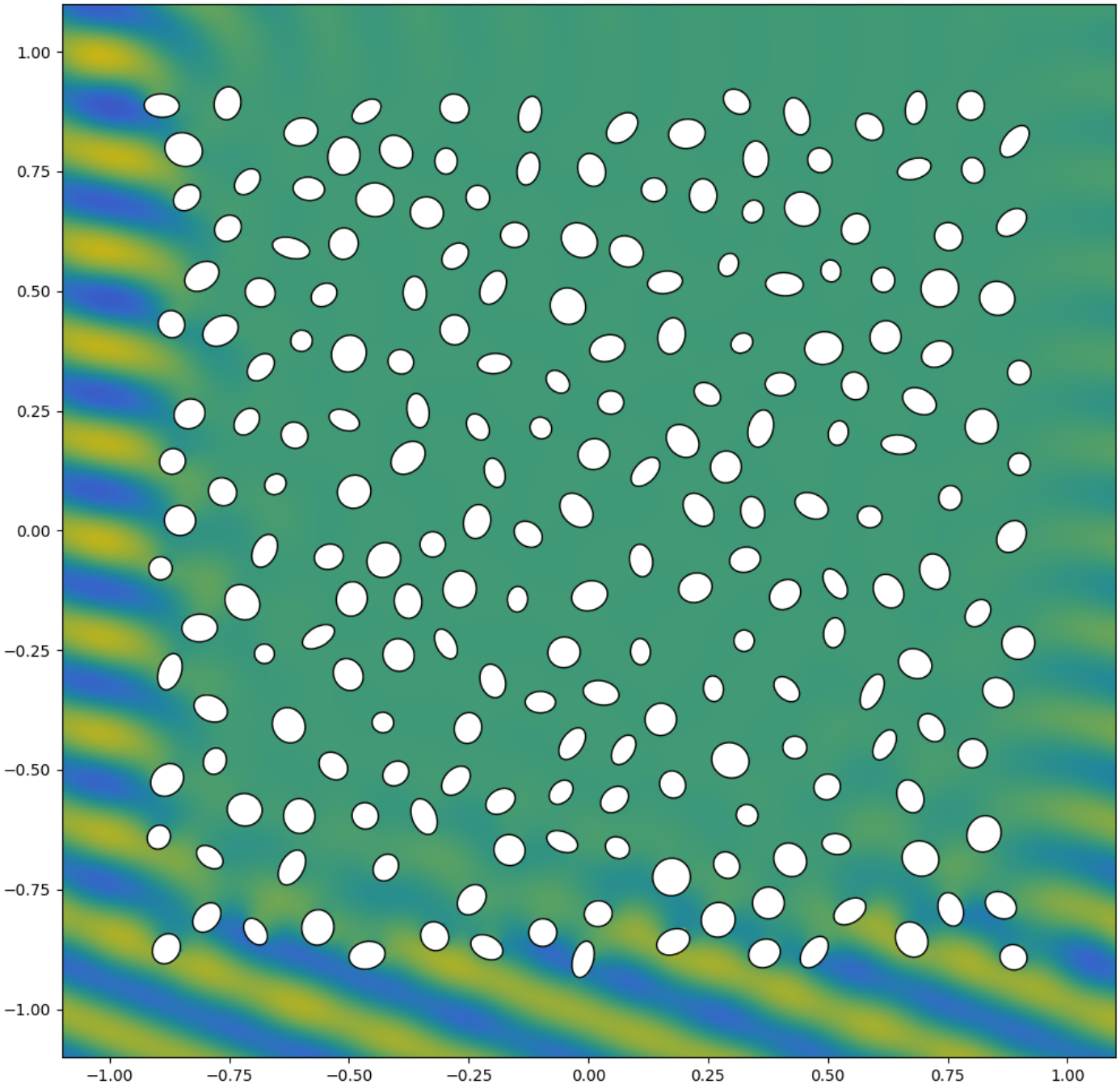
$k = 17$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

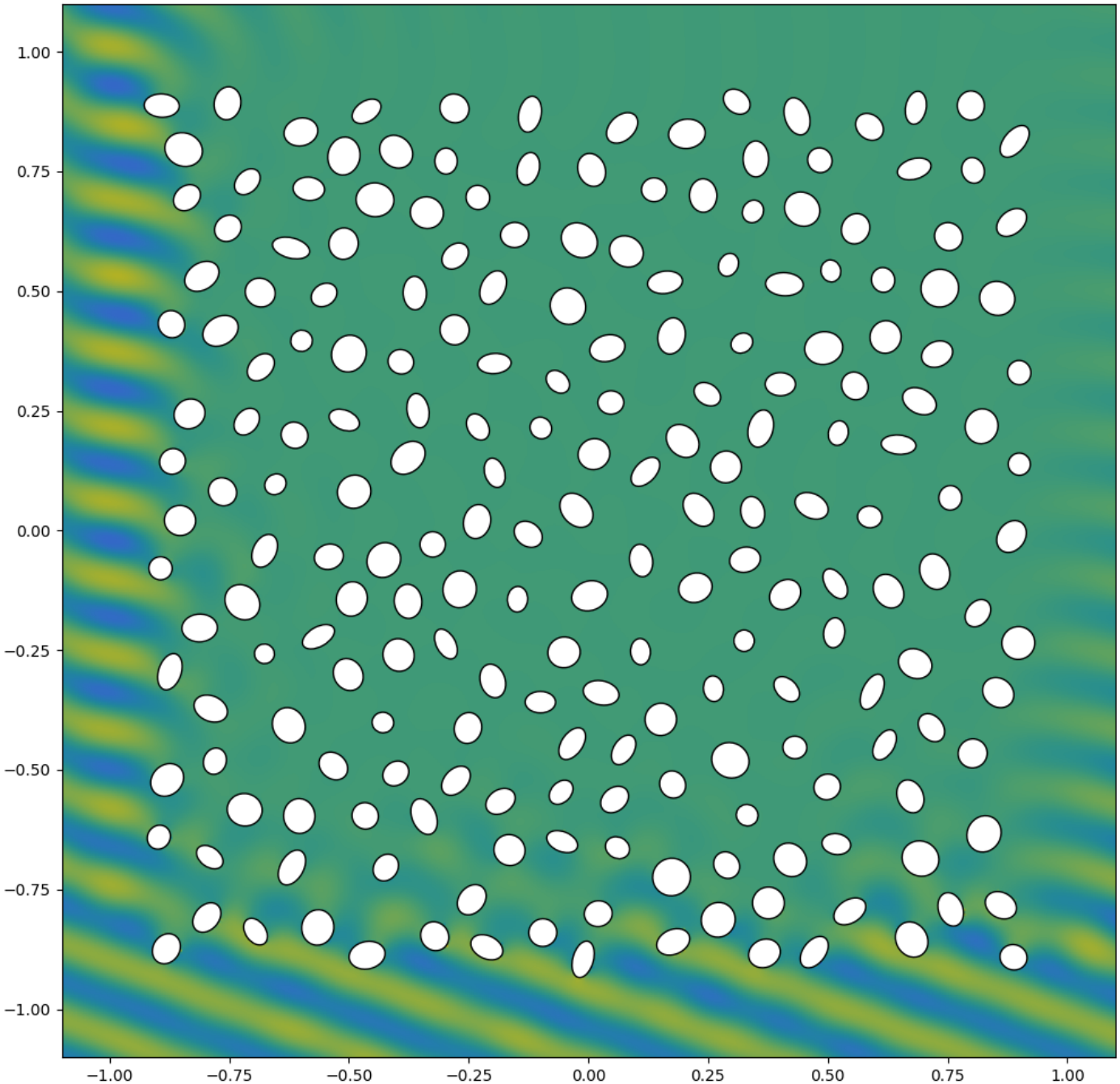
$k = 24$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

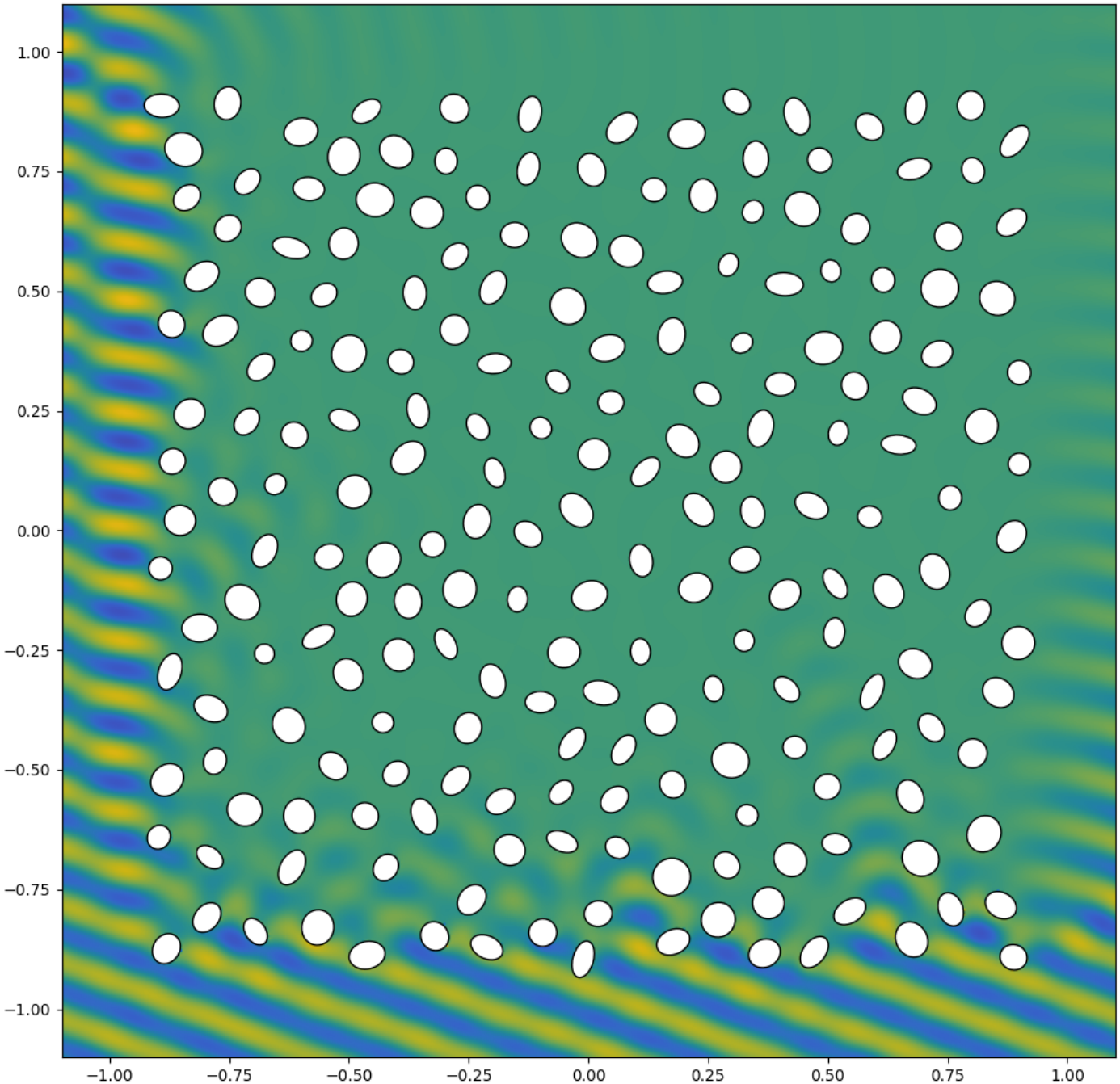
$k = 33$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

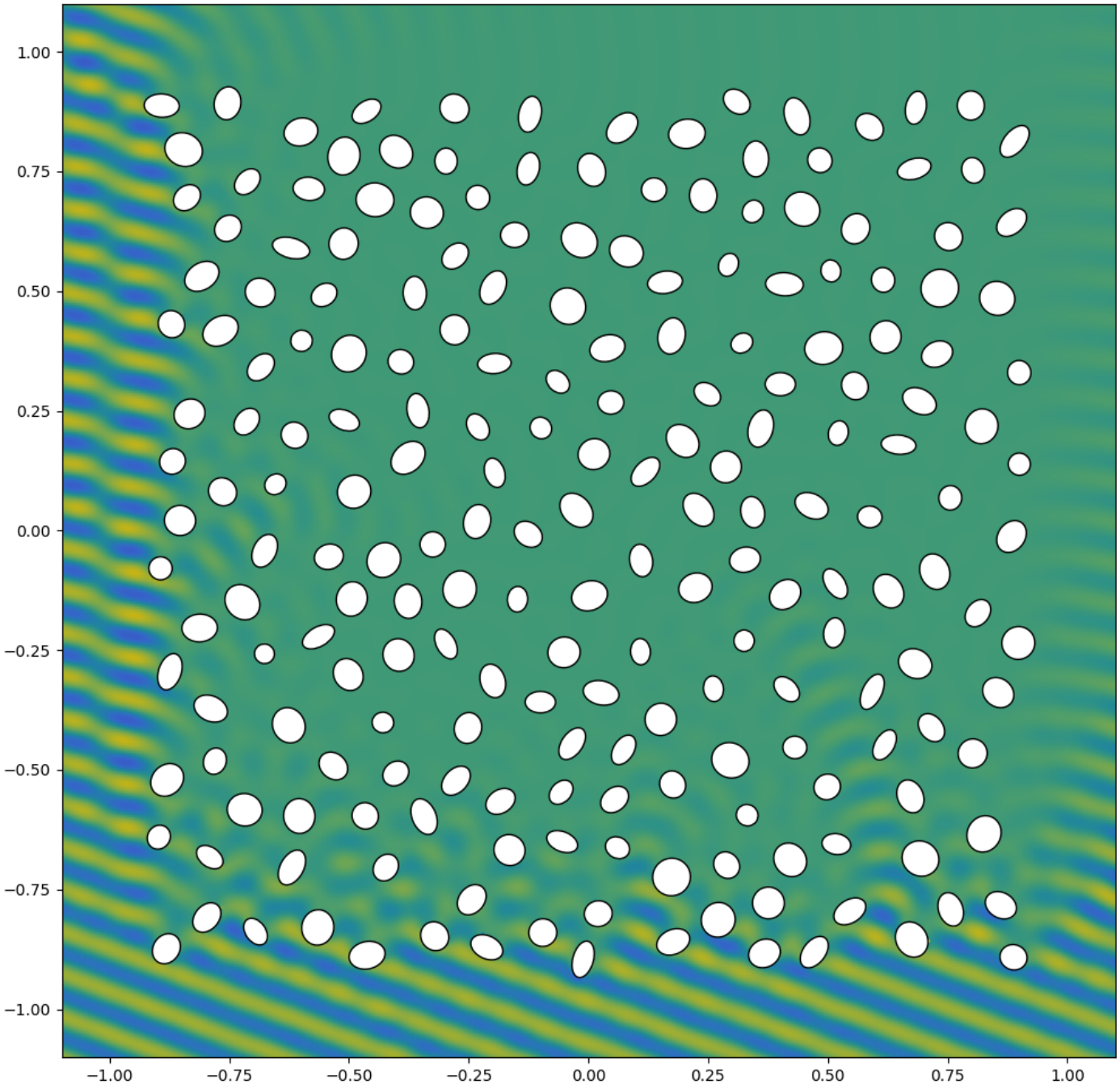
$k = 42$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

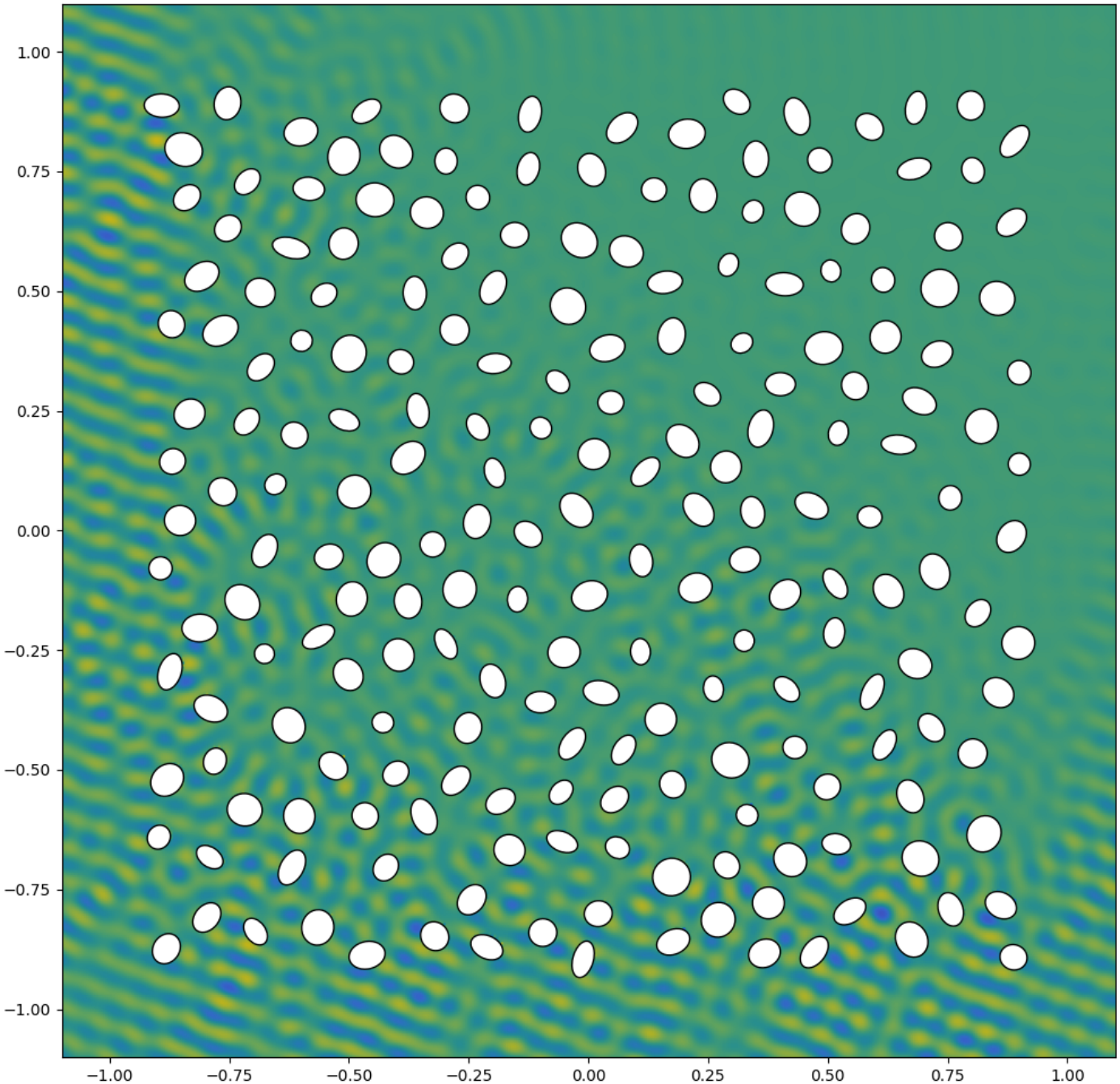
$k = 56$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

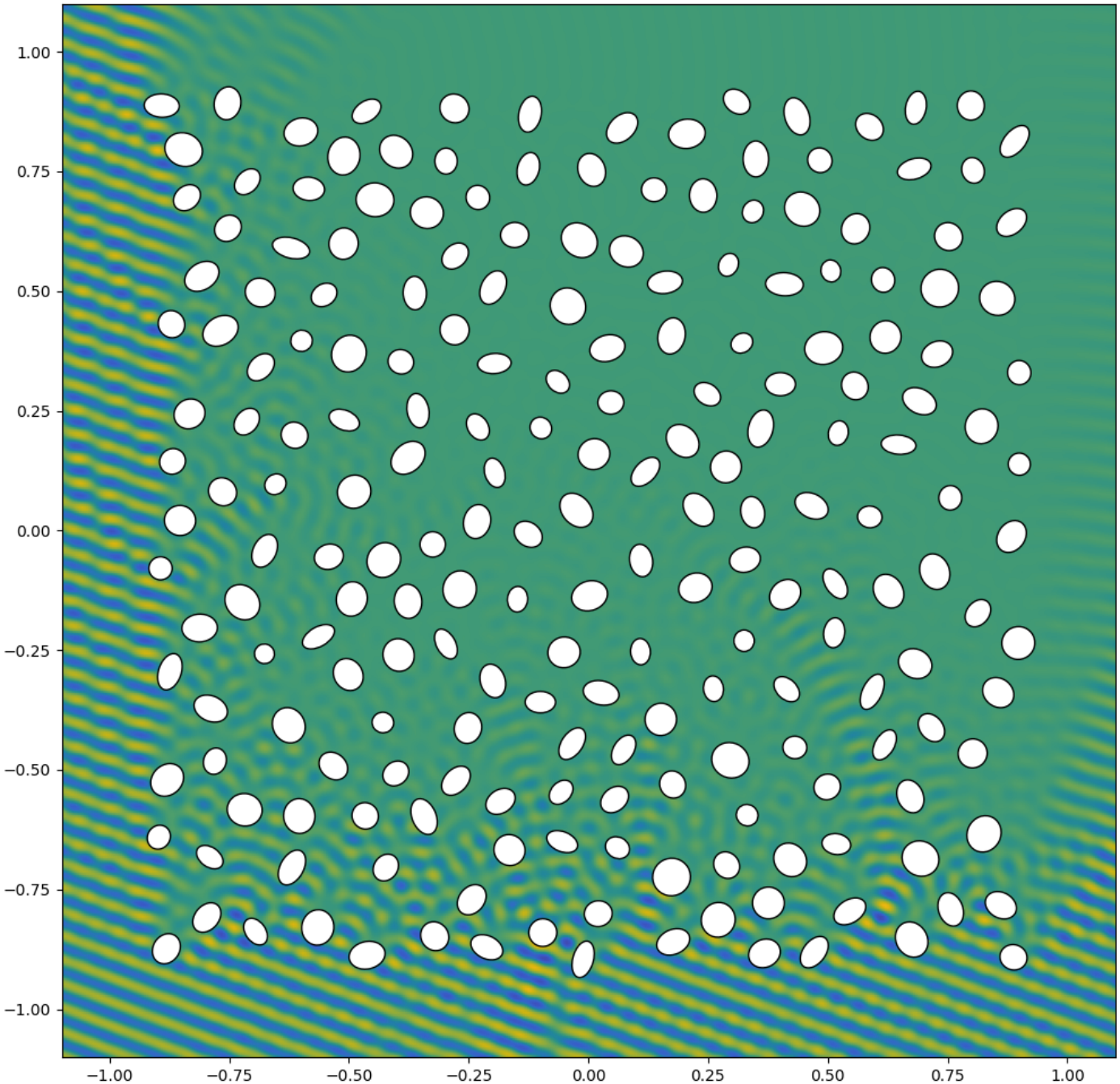
$k = 75$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

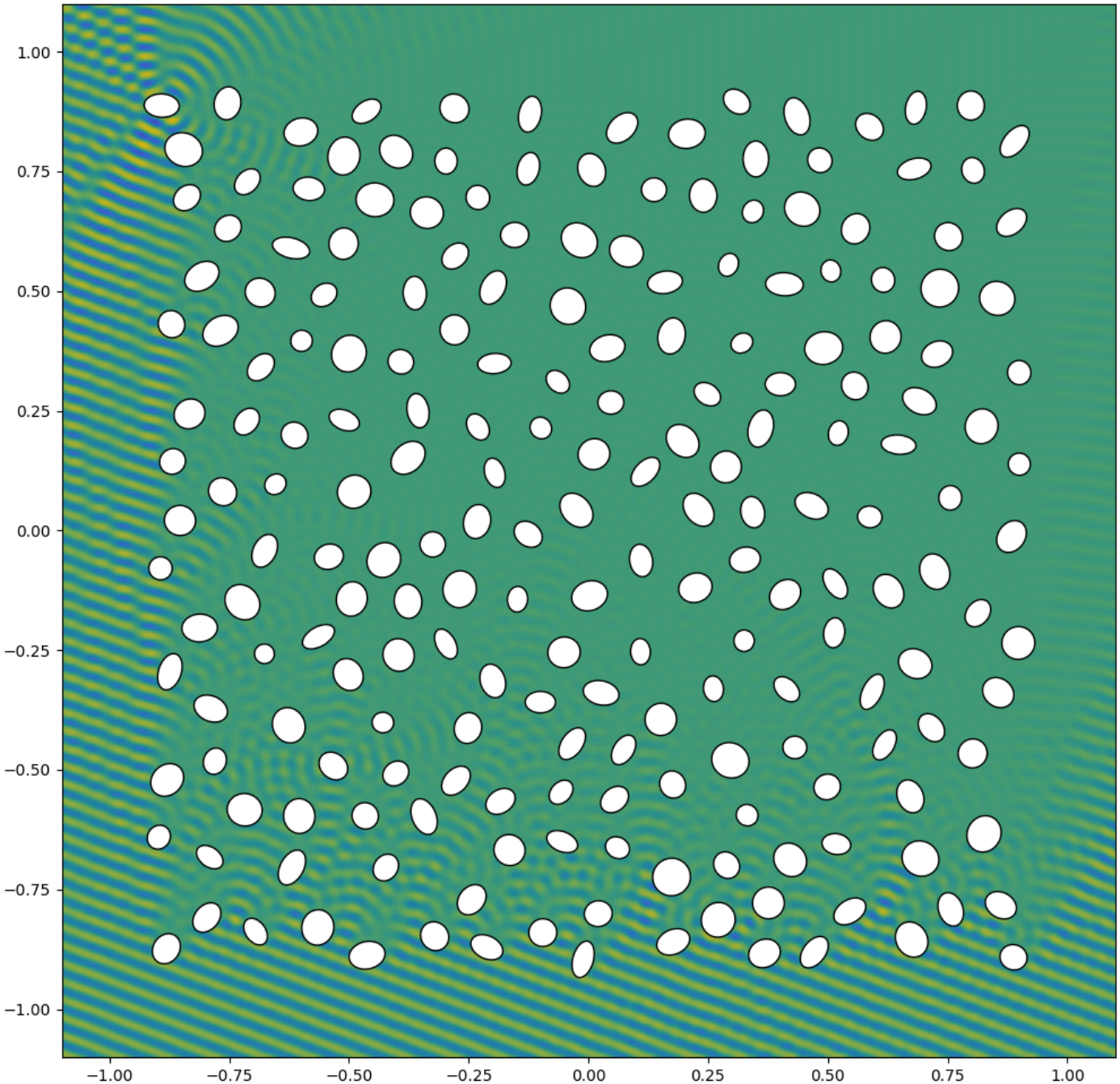
$k = 100$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

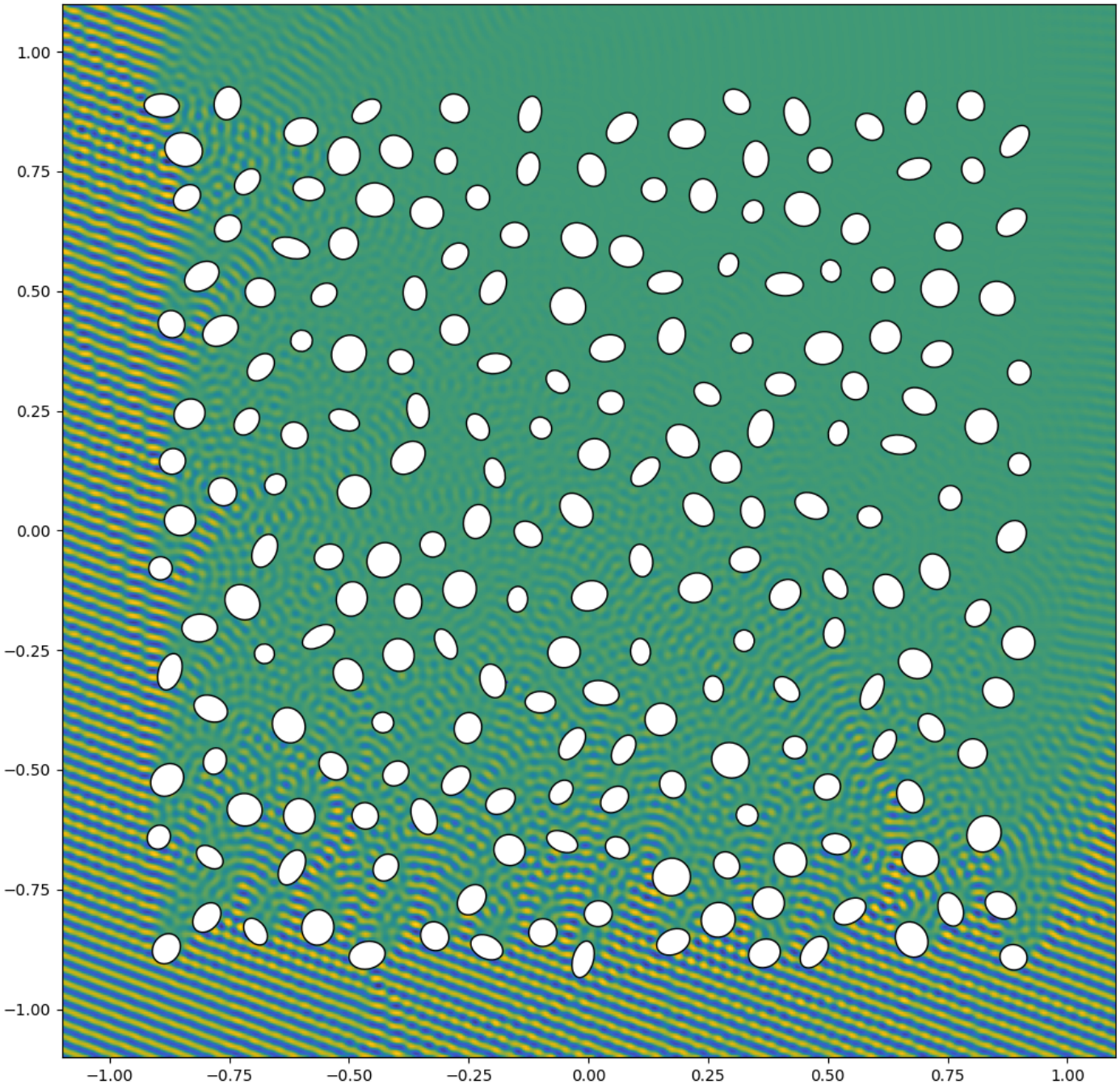
$k = 133$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

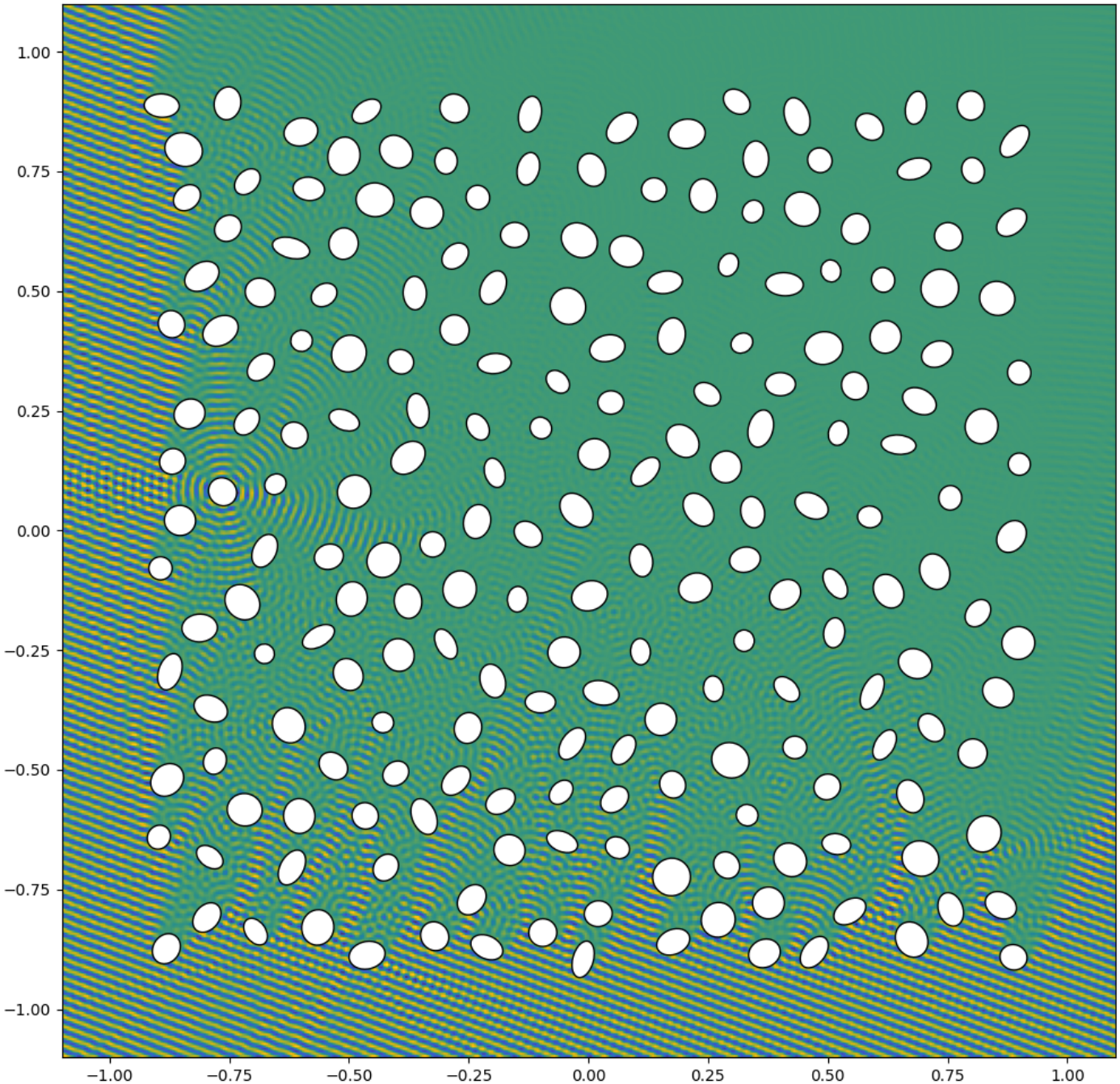
$k = 178$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

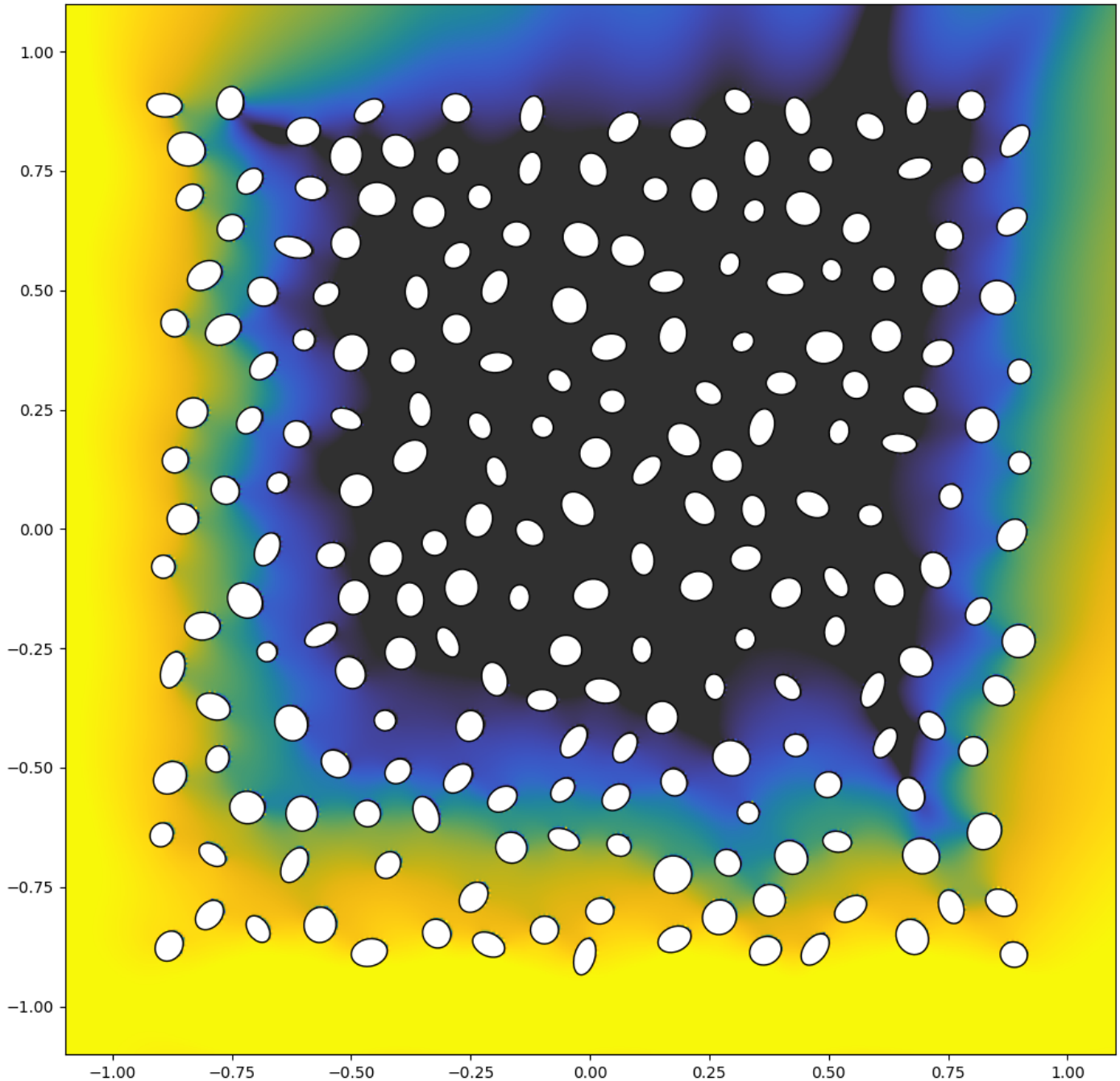
$k = 237$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

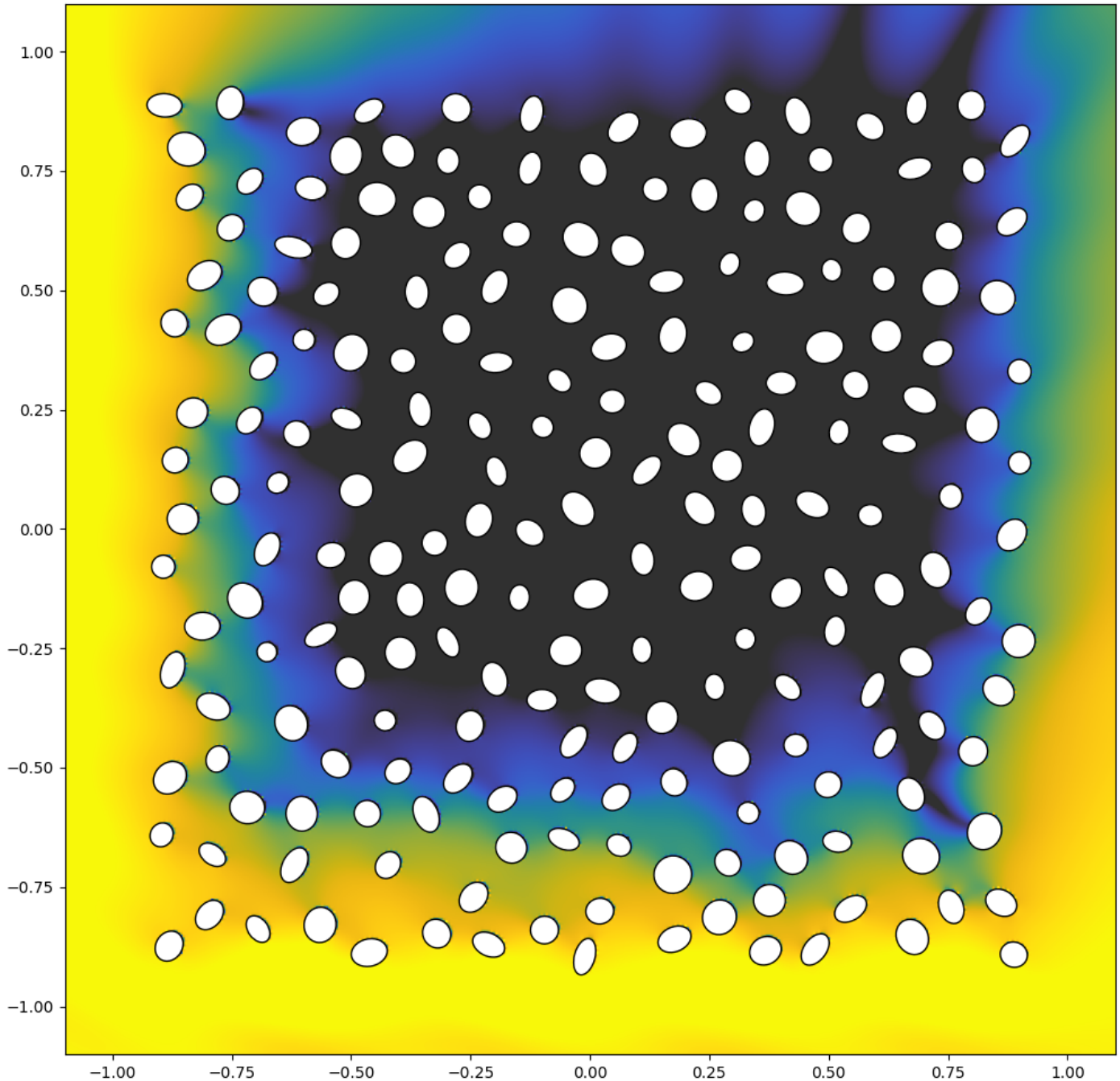
$k = 333$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

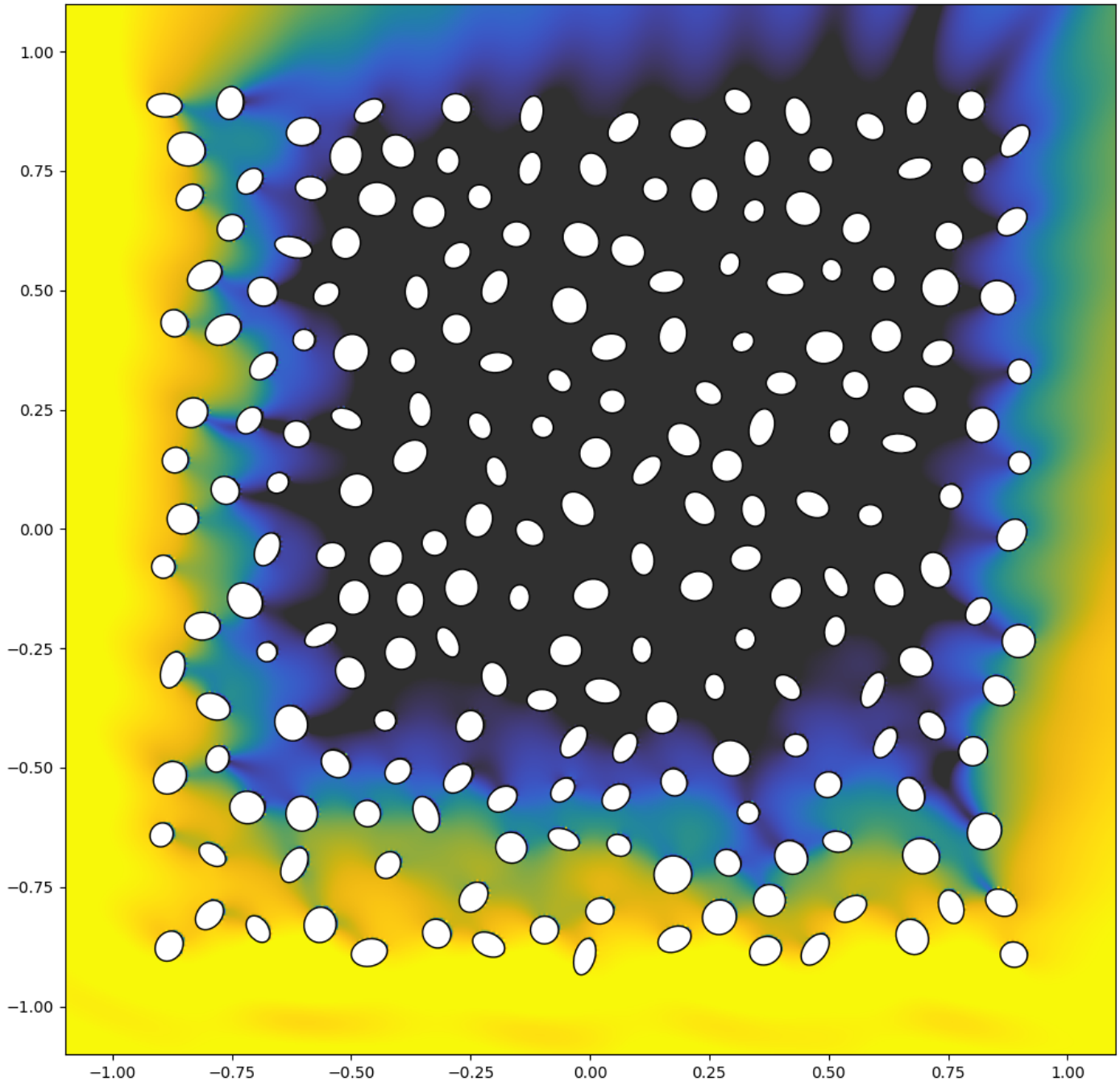
$k = 10$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

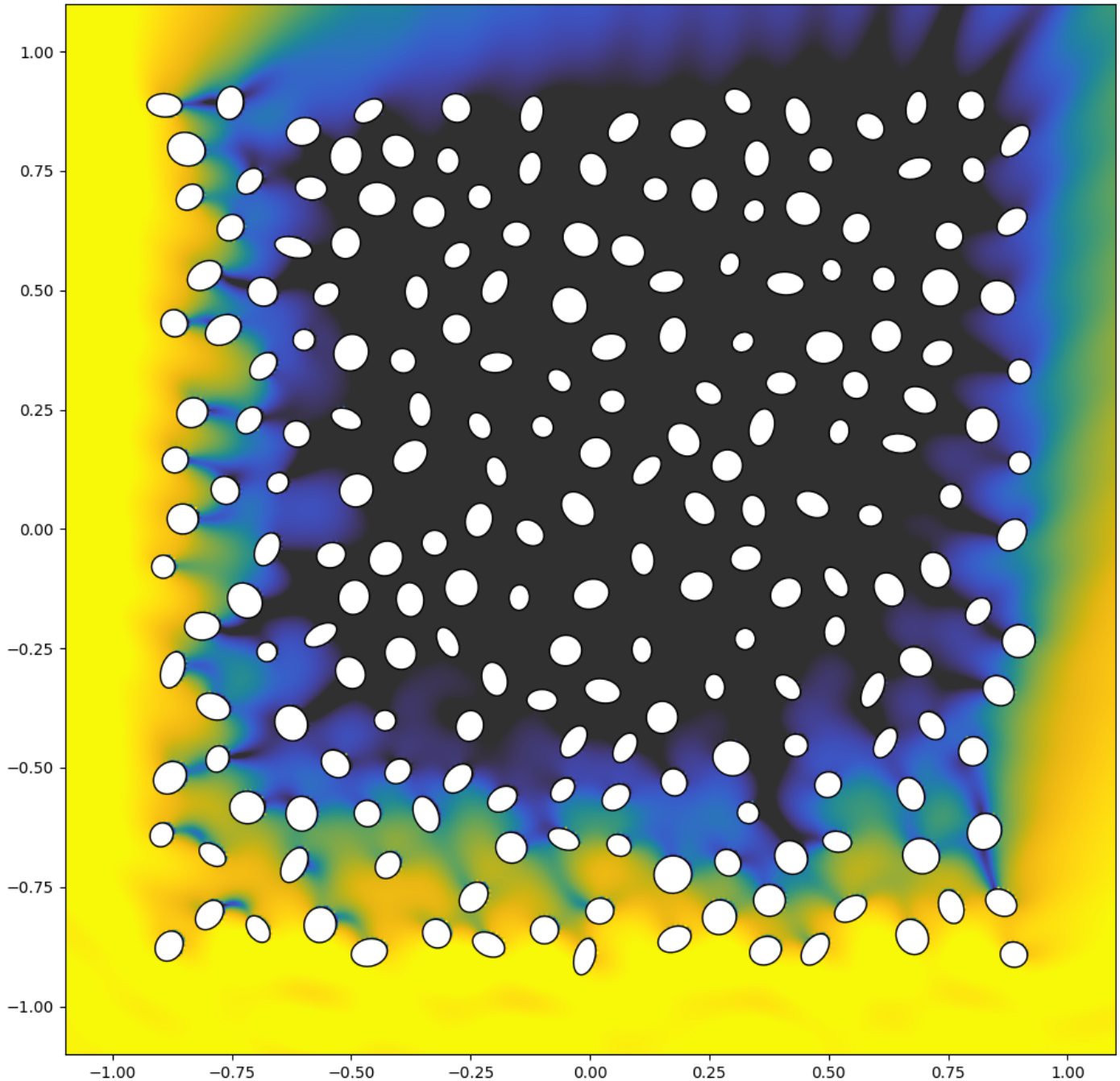
$k = 13$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

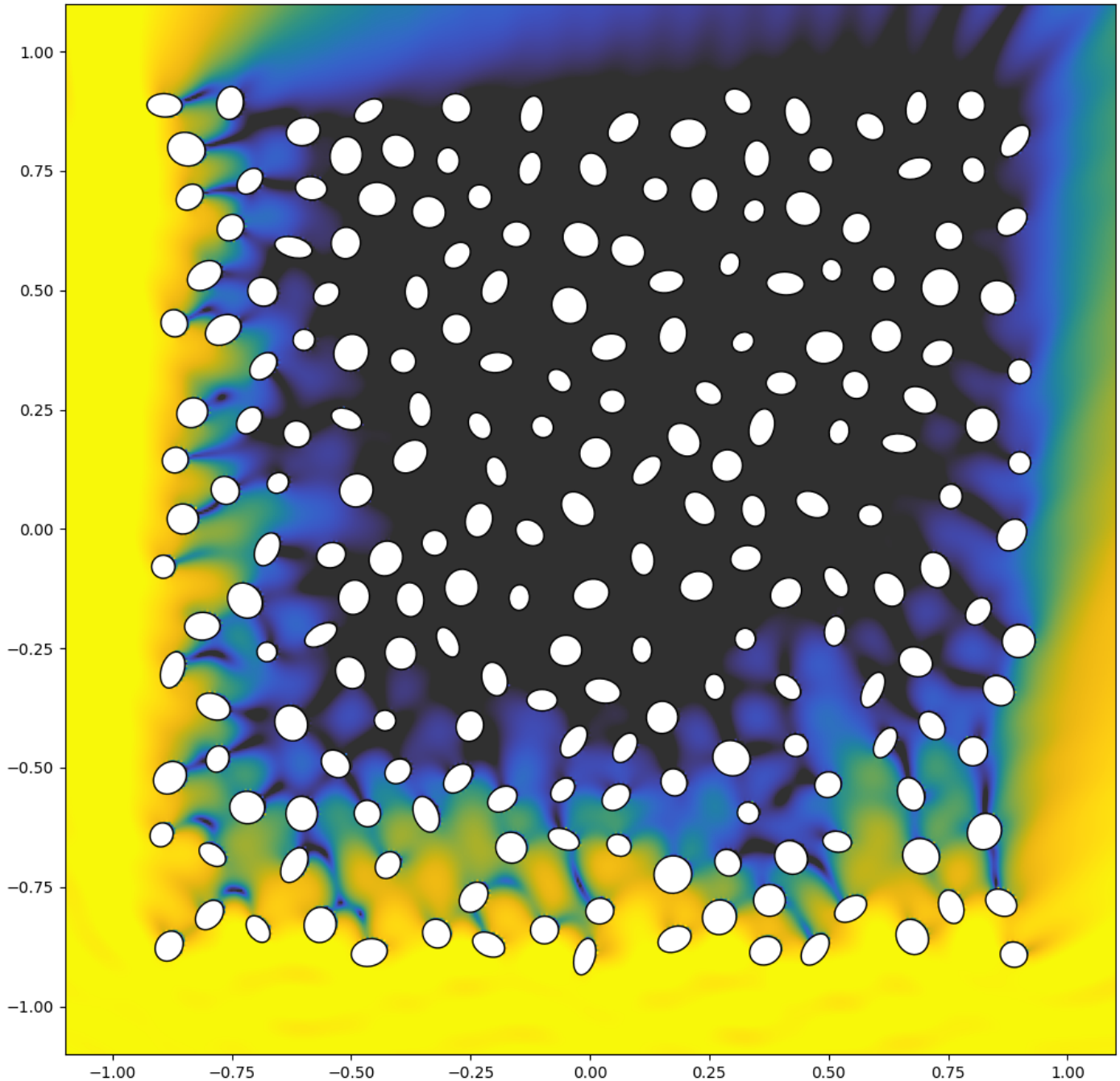
$k = 17$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

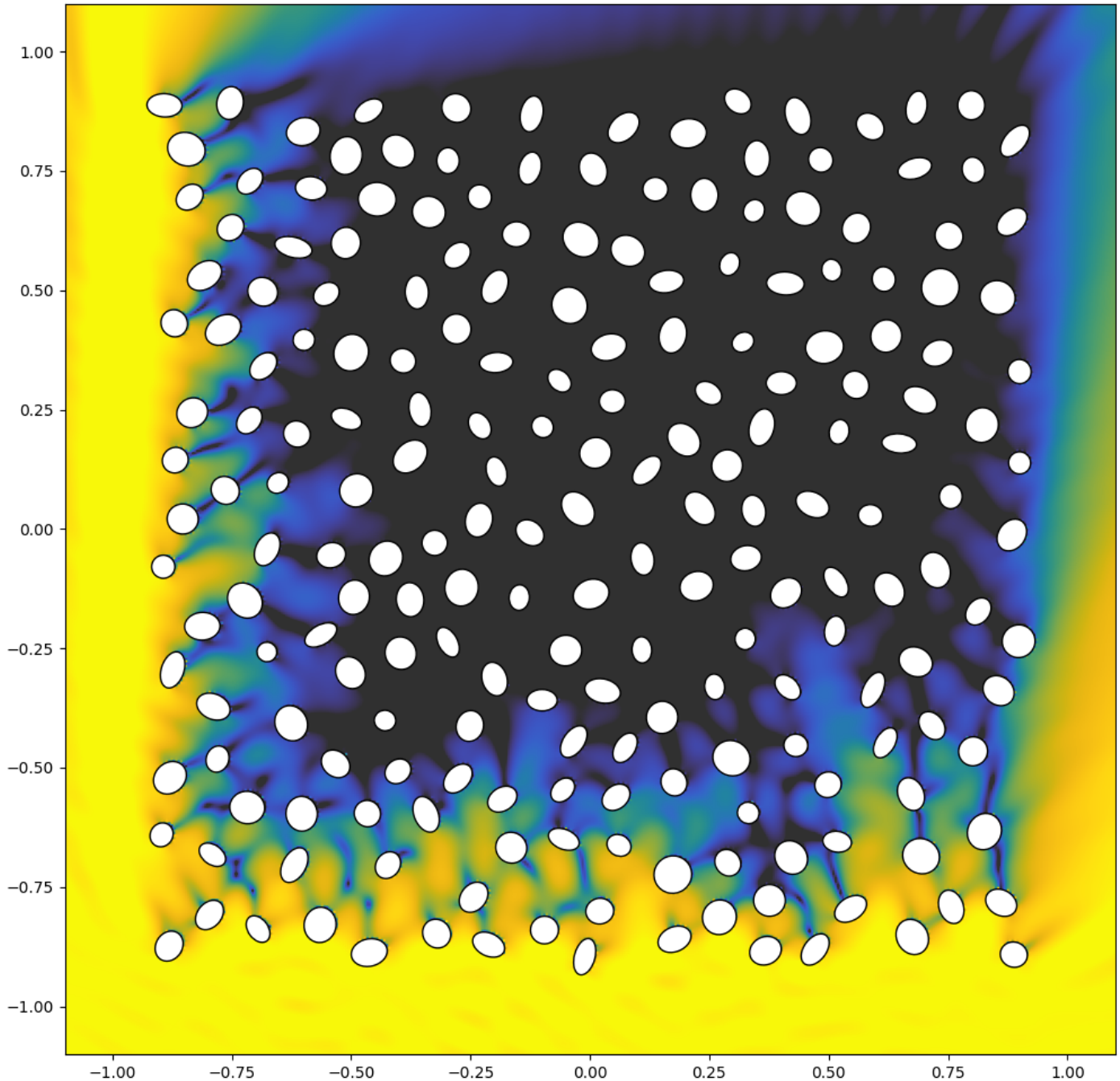
$k = 24$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

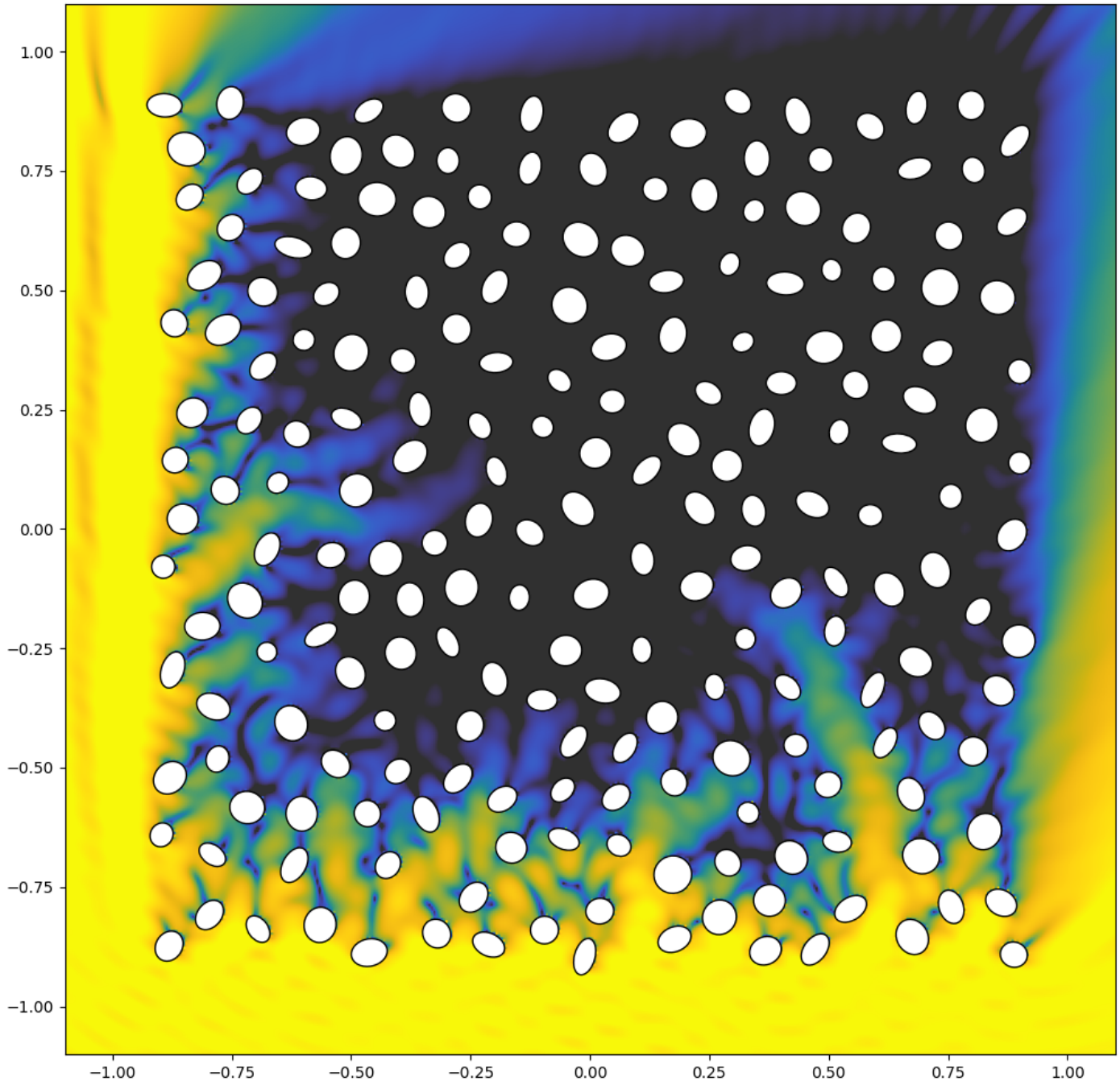
$k = 33$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

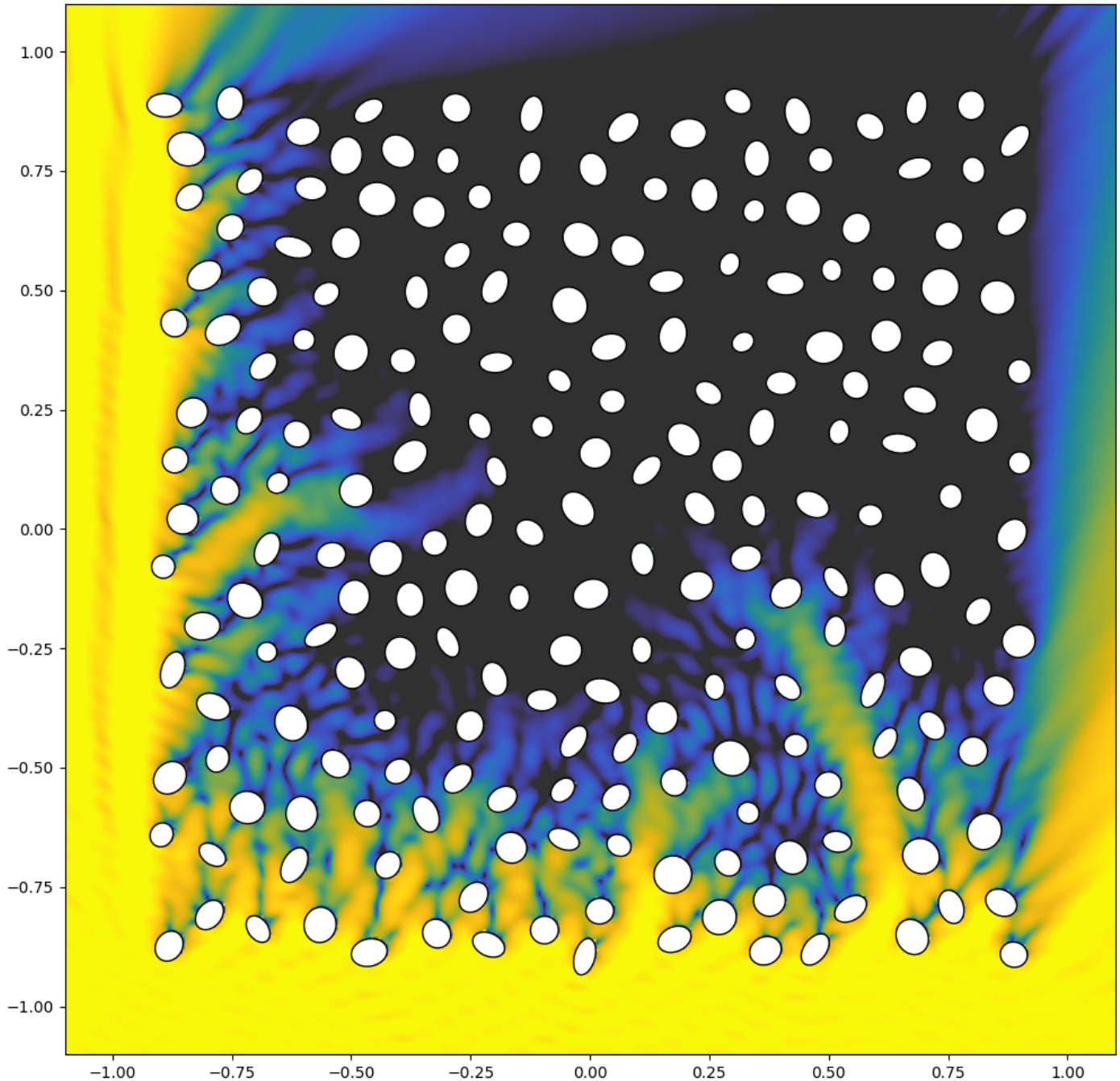
$k = 42$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

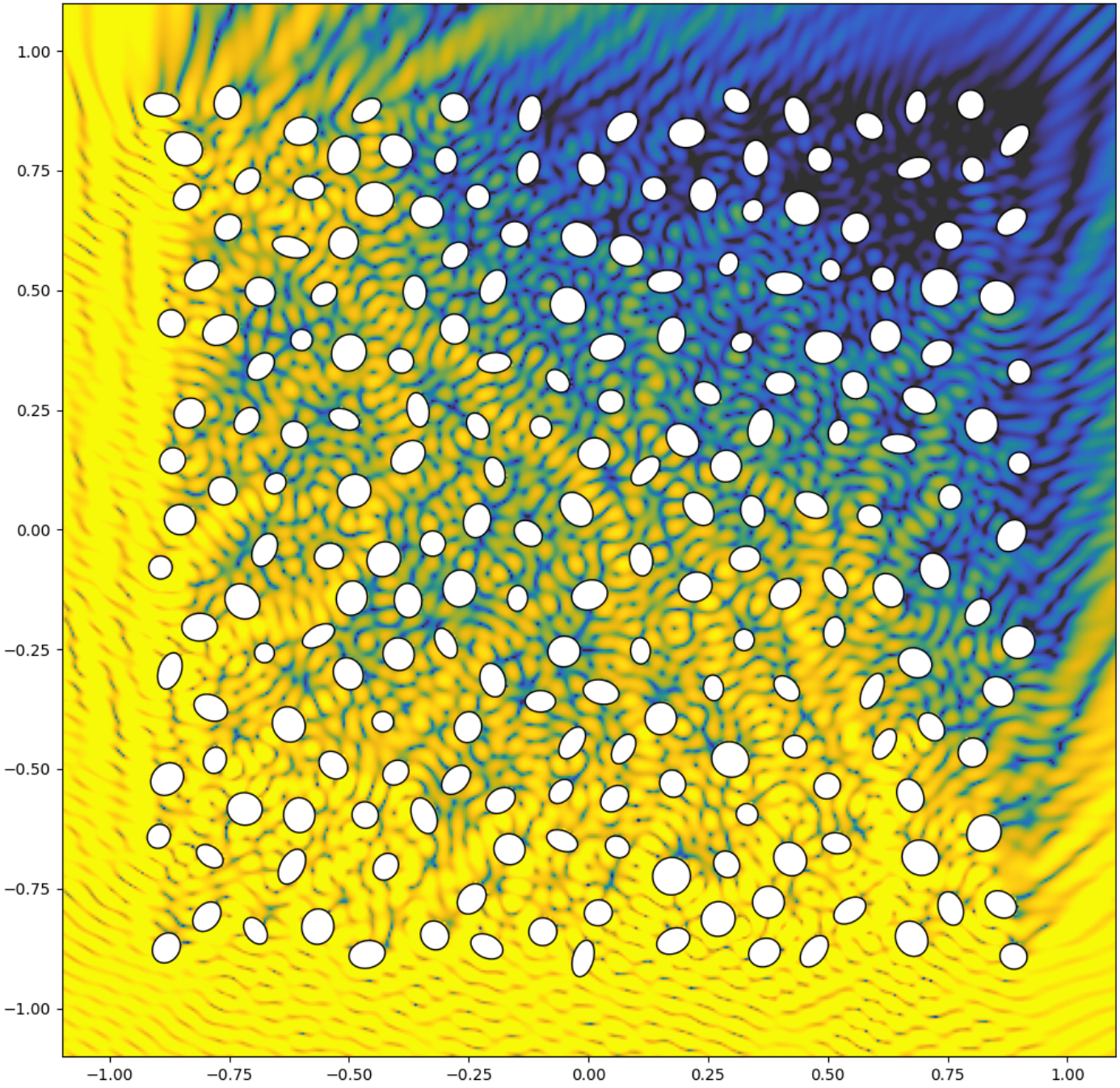
$k = 56$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

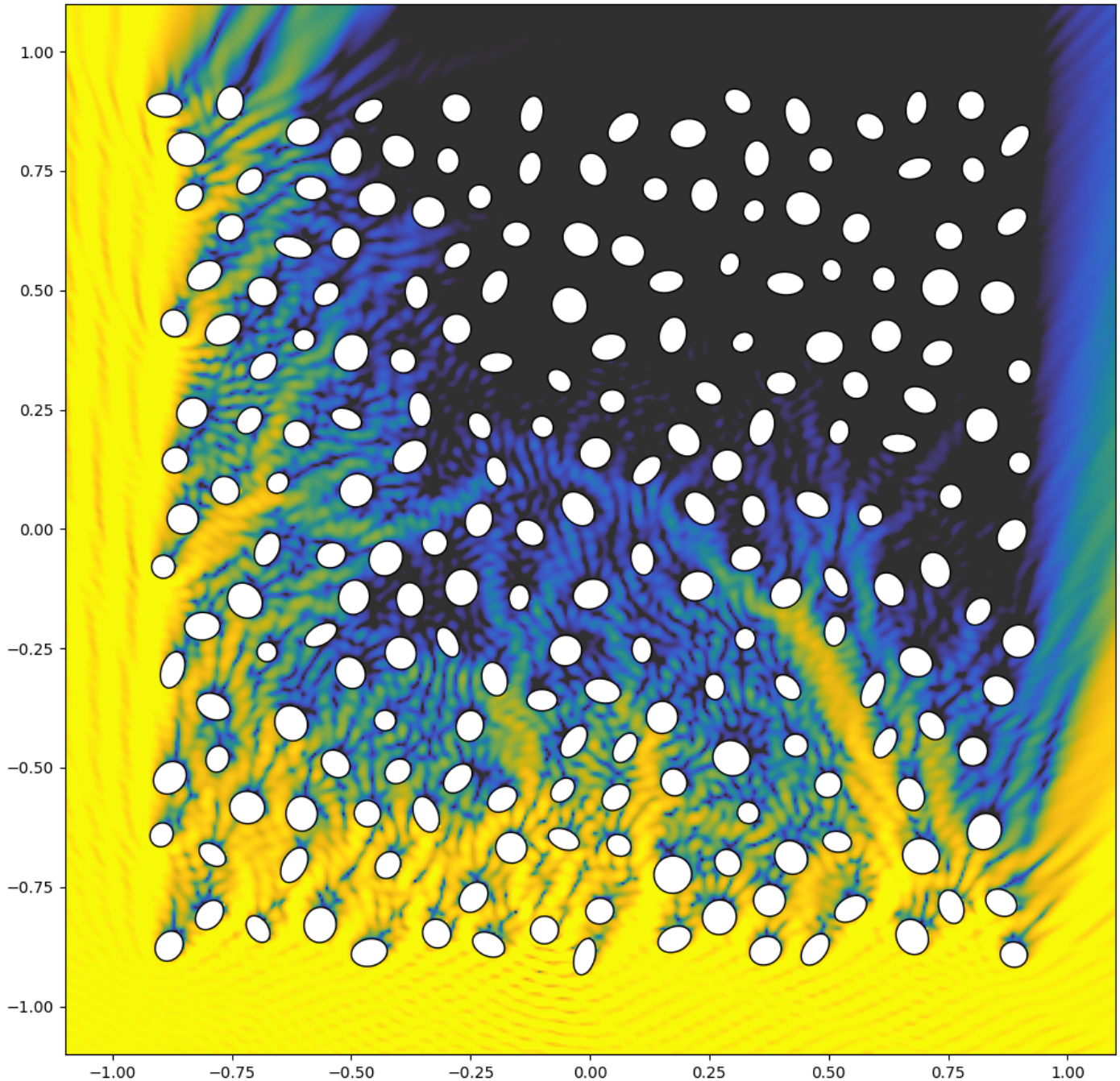
$k = 75$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

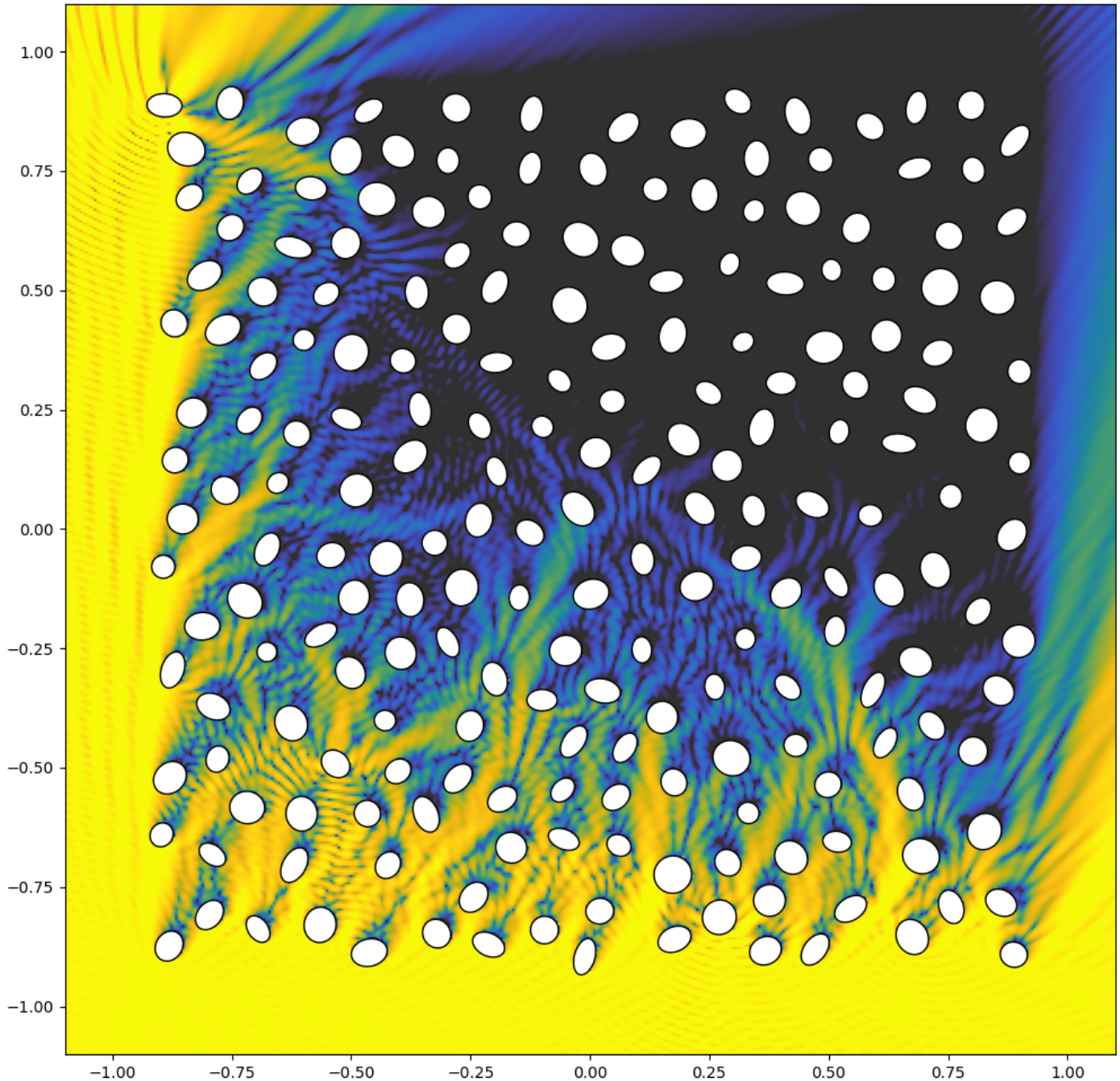
$k = 100$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

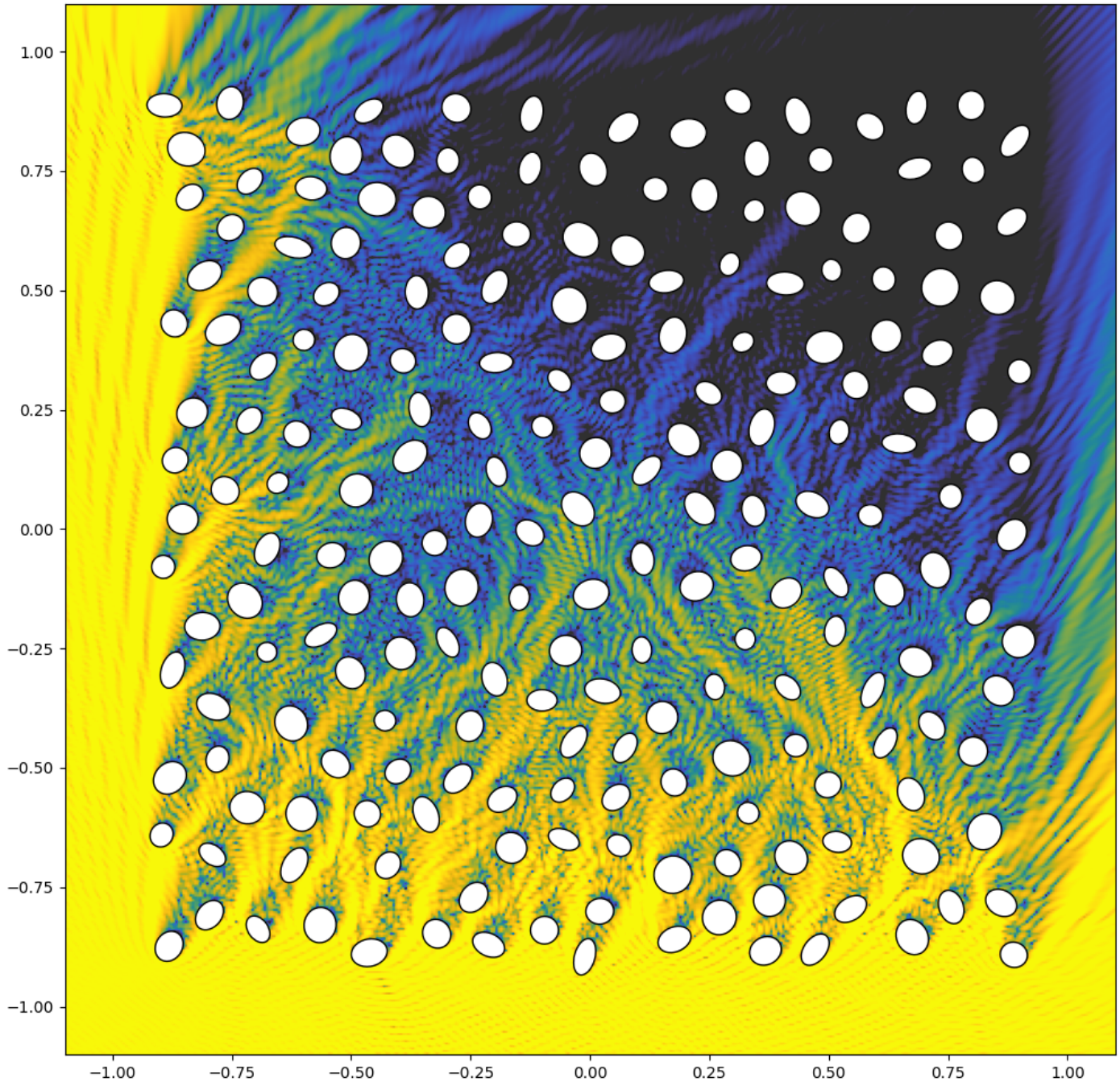
$k = 133$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

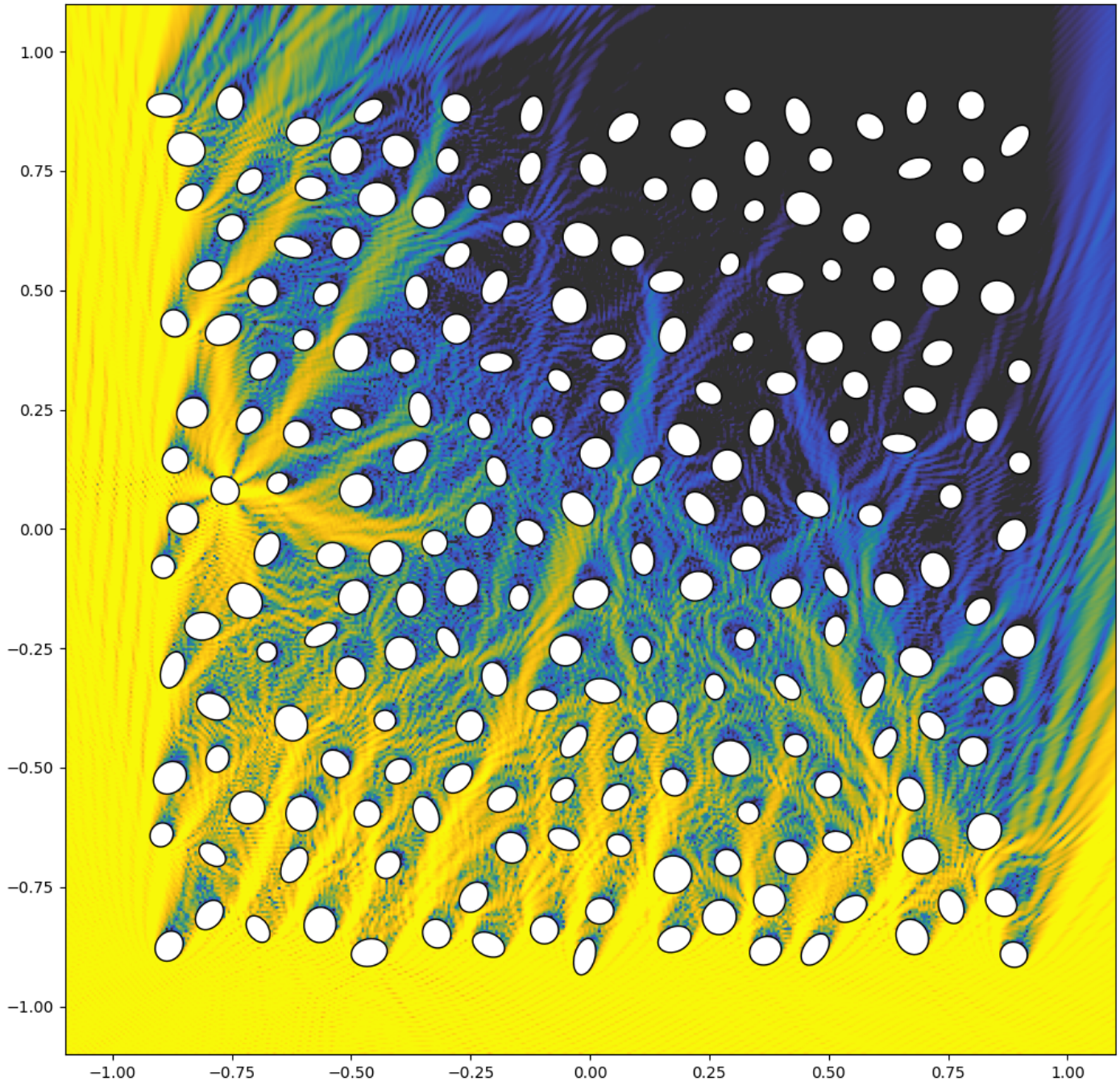
$k = 178$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

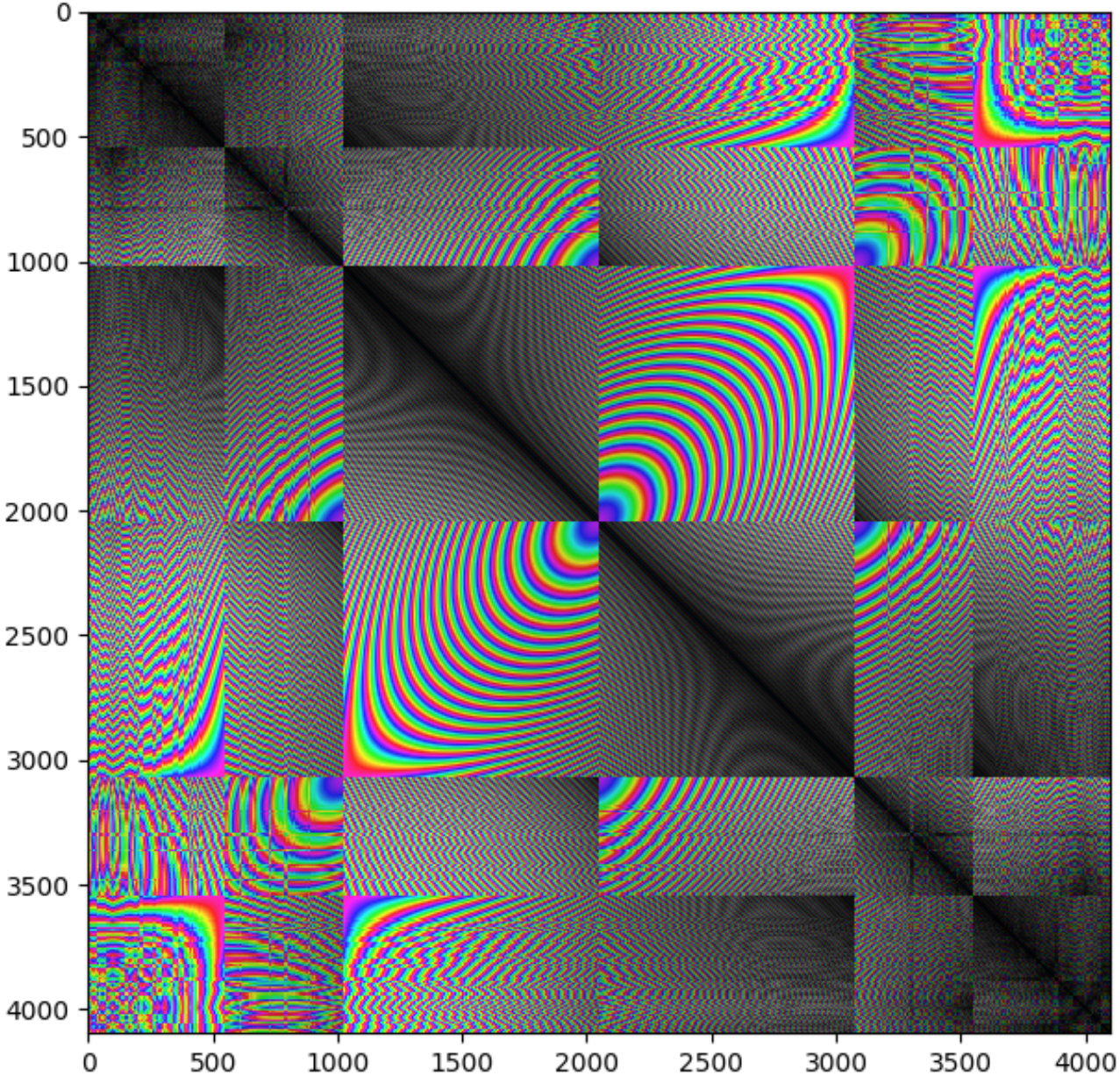
$k = 237$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

$k = 333$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

Can we “butterfly” the Helmholtz kernel matrix?



Key tool: large k rank estimate

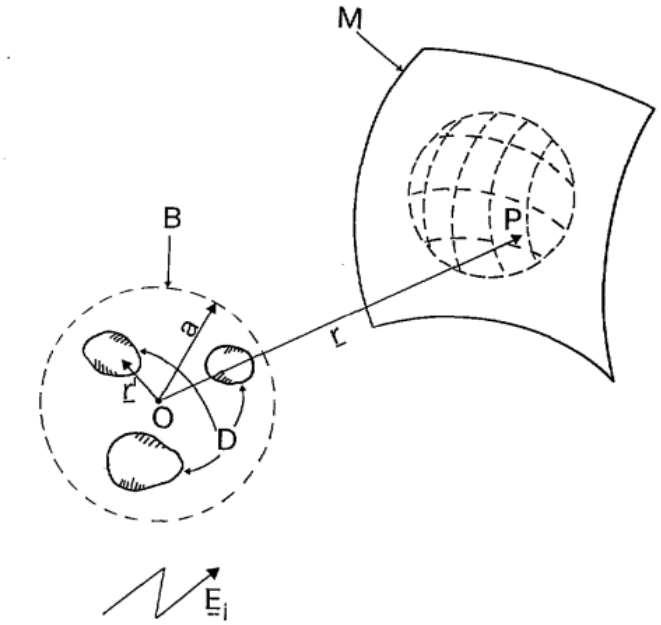


Fig. 1. Geometry of the scattering problem.

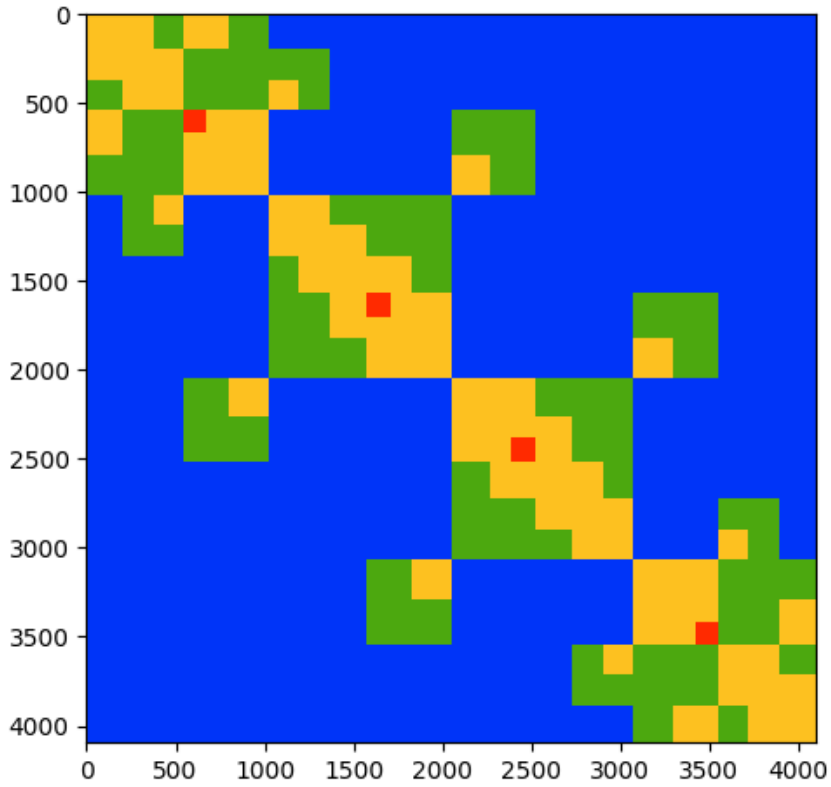
Theorem. Let $k > 0$. For 2D scatterer with radius r_1 and domain with radius r_2, if R is the separation distance, then ϵ-rank of Helmholtz kernel matrix is $kr_1r_2/(R-r_1-r_2)+O(\log(1/\epsilon))$ as $k \to \infty$.[1]

---

[1] Bucci and Franceschetti, "On the Spatial Bandwidth of Scattered Fields", IEEE Transactions on Antennas and Propagation, 1987.

Multilevel butterfly factorization

Can’t butterfly K directly, can only butterfly blocks corresponding to well-separated sets of points. Use a quadtree to generate these sets.

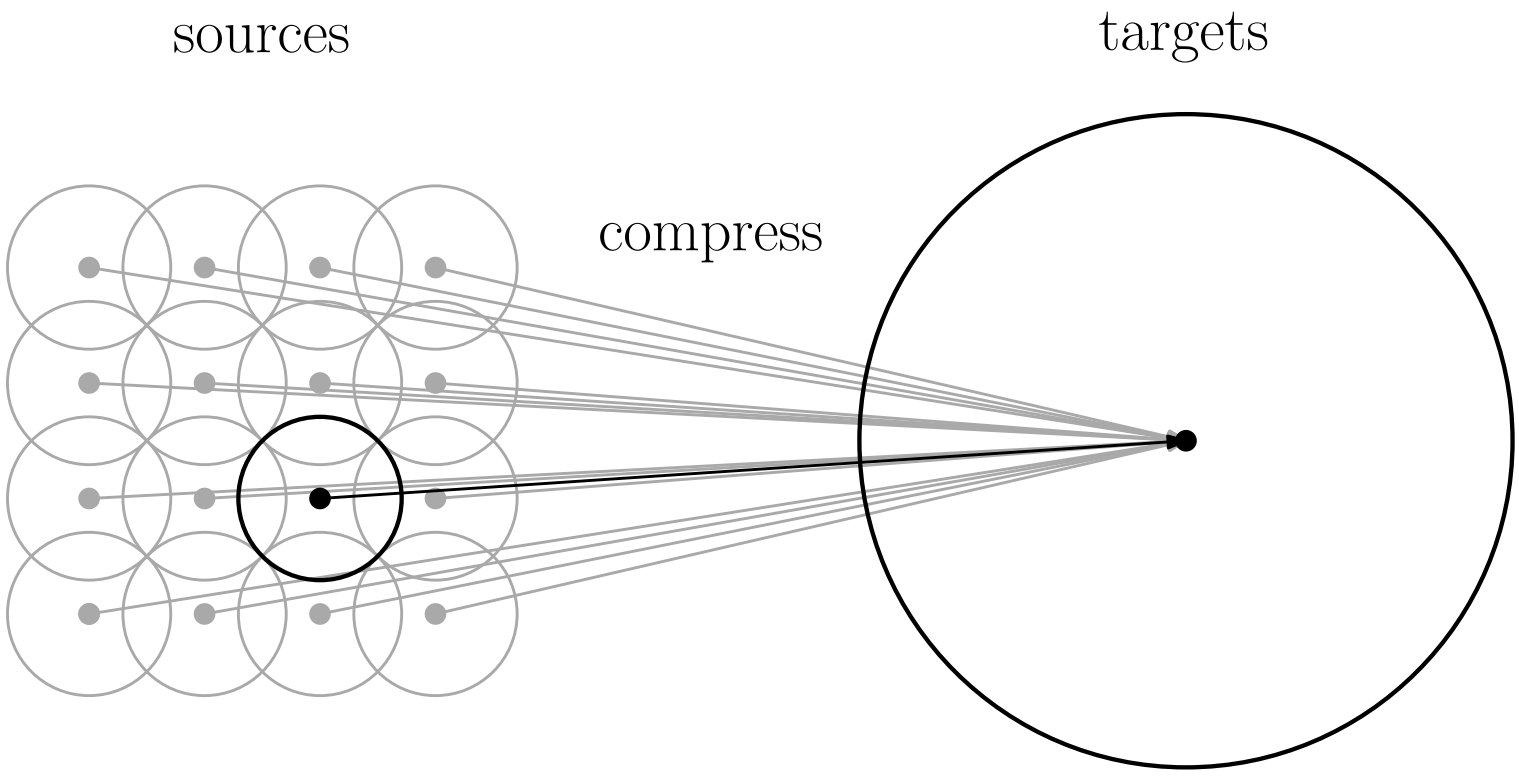


How to butterfly a block of K

- **Input:** well-separated source and target boxes in quadtree.
- Find deepest level of source tree which is 1) complete, and 2) for which the rank between each leaf and the target box is “small”.
- Go up and down source and target trees simultaneously, build butterfly factorization based on equivalent surface principle using proxy points.

This is the simplest thing that works and is efficient.
Lots of room for improvement.

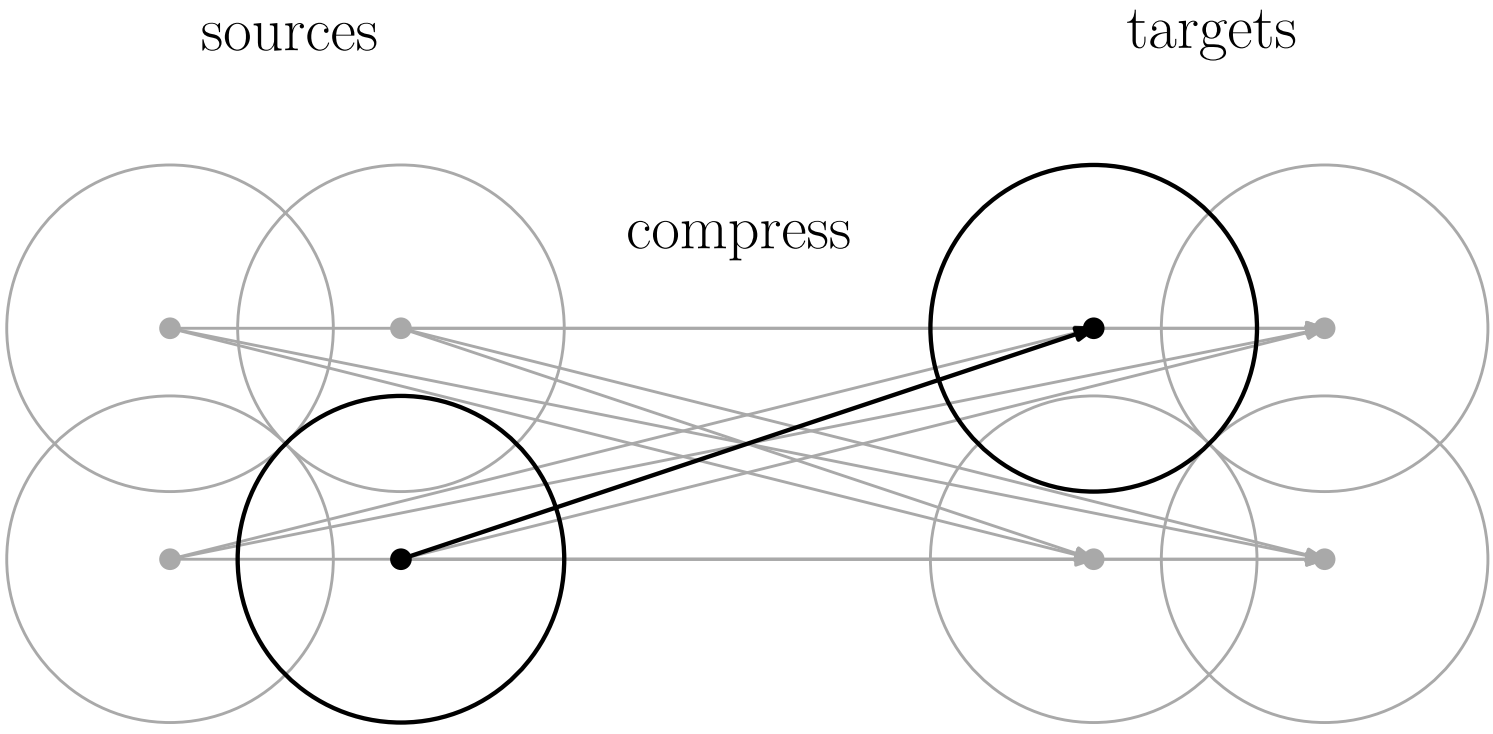
How to butterfly a block of K



equivalent surfaces are nested

observation domains are nested

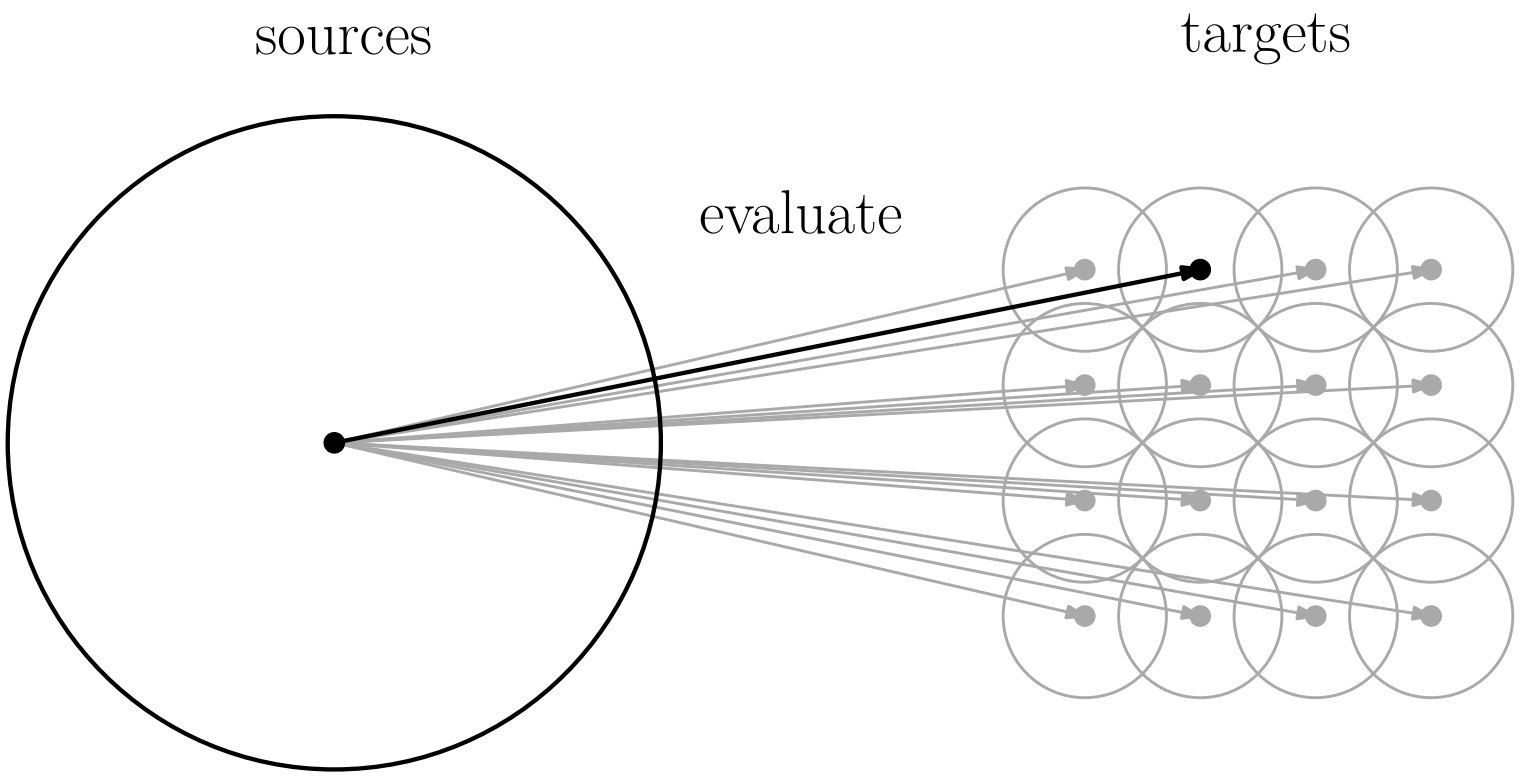
How to butterfly a block of K



equivalent surfaces are nested

observation domains are nested

How to butterfly a block of K



equivalent surfaces are nested

observation domains are nested

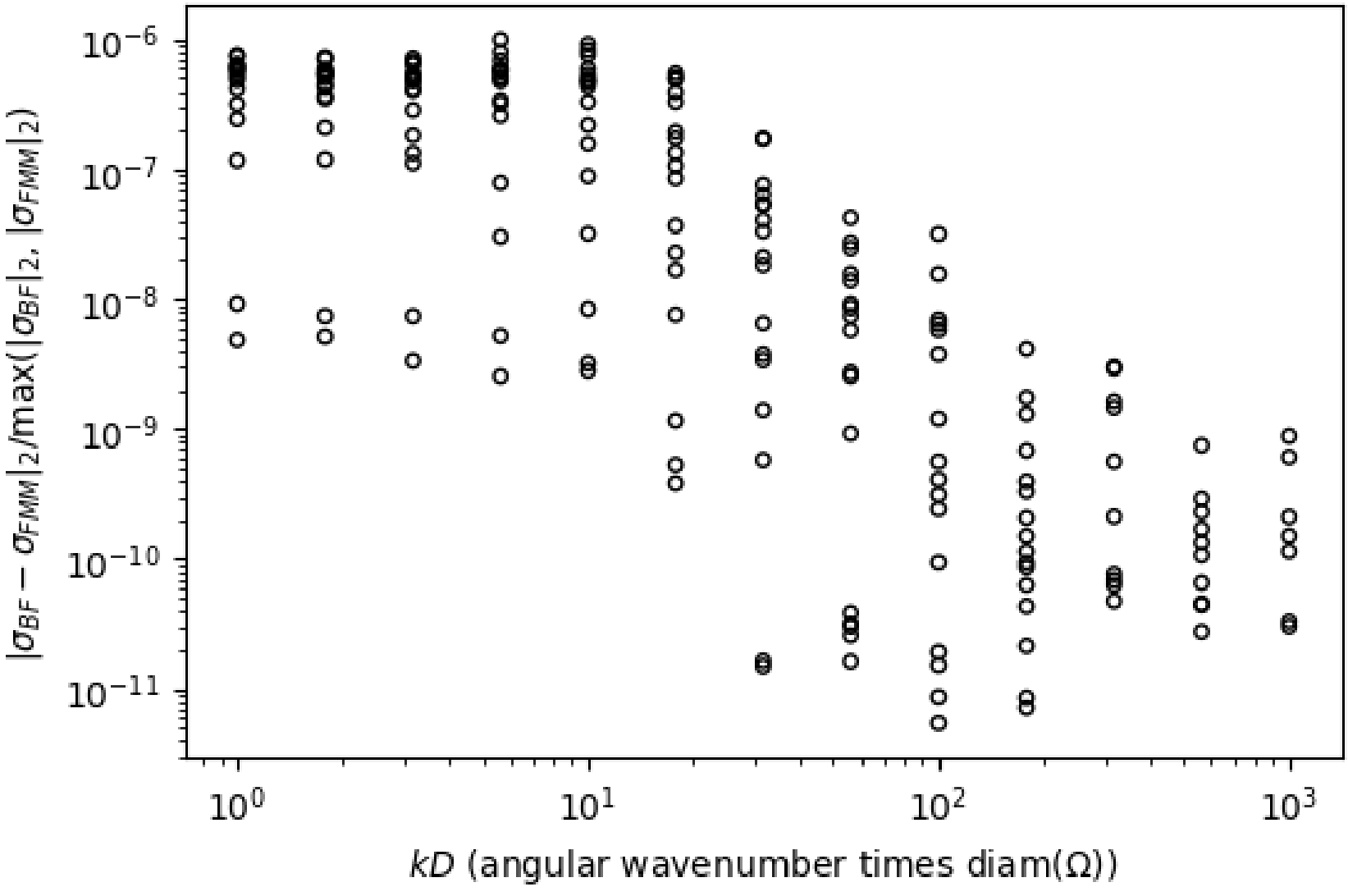
Multilevel butterfly factorization

- Butterfly each kernel subblock corresponding to well-separated nodes in the quadtree
- Time and space complexity of one butterfly factorization: $O(N \log k)$.
- ... of a multilevel butterfly factorization: $O(N \log N \log k)$.
- Same complexity for assembly *and* multiplication, but assembly is 1-2 orders of magnitude slower.
- If $k = \Theta(N)$, then $O(N \log^2 N)$ complexity.

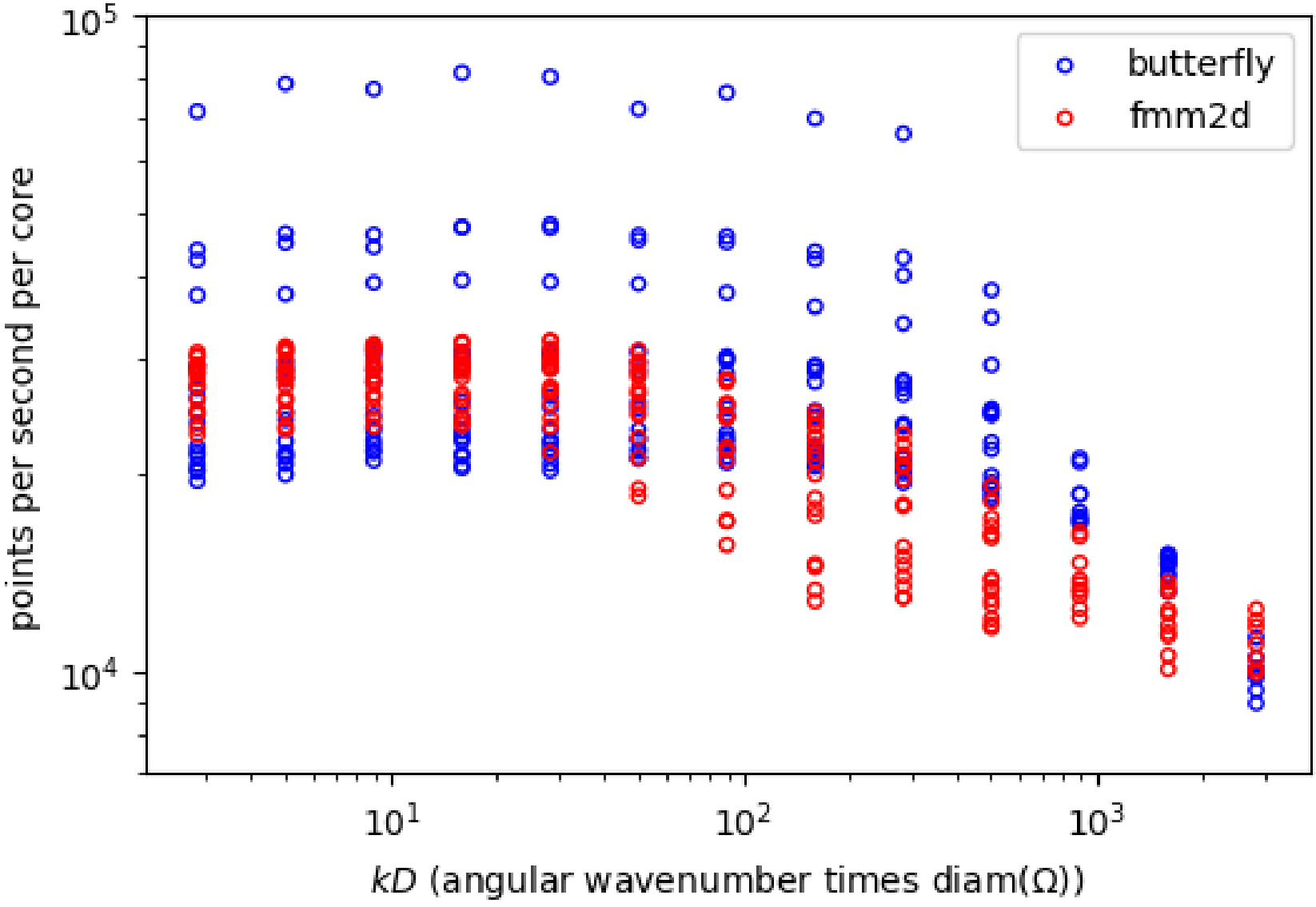
Numerical experiments

- Grid of experiments: k, ellipse radius, discretization fineness (h) and number of points (N)
- Solve scattering problem with ML-BF and HF-FMM (using Flatiron/CCM's `fmm2d`, current state of the art)
- Everything is 100% single-threaded (no multithreaded BLAS, etc.)
- Measure relative accuracy, time taken to solve, # iterations, memory use, etc.

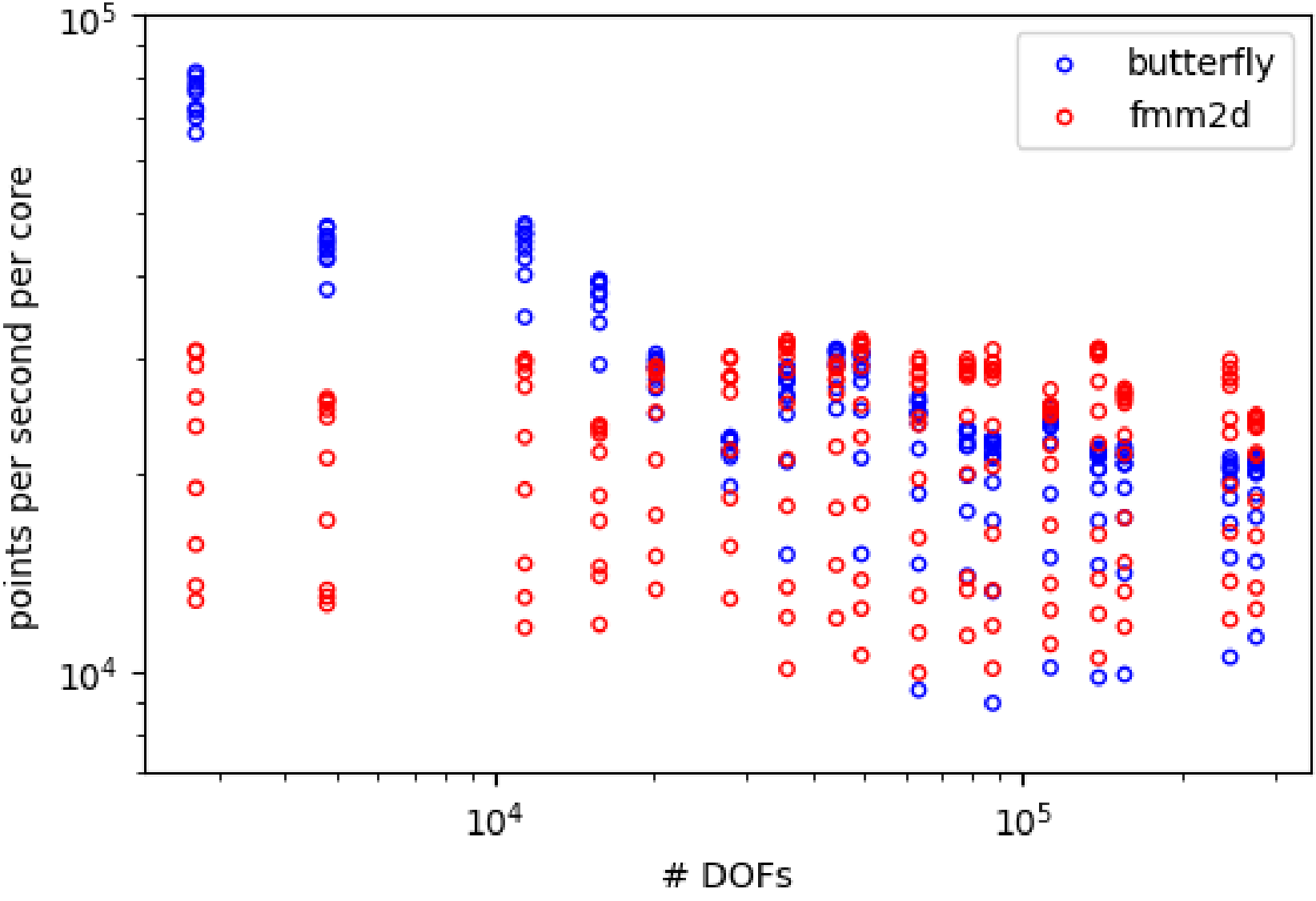
Numerical experiments



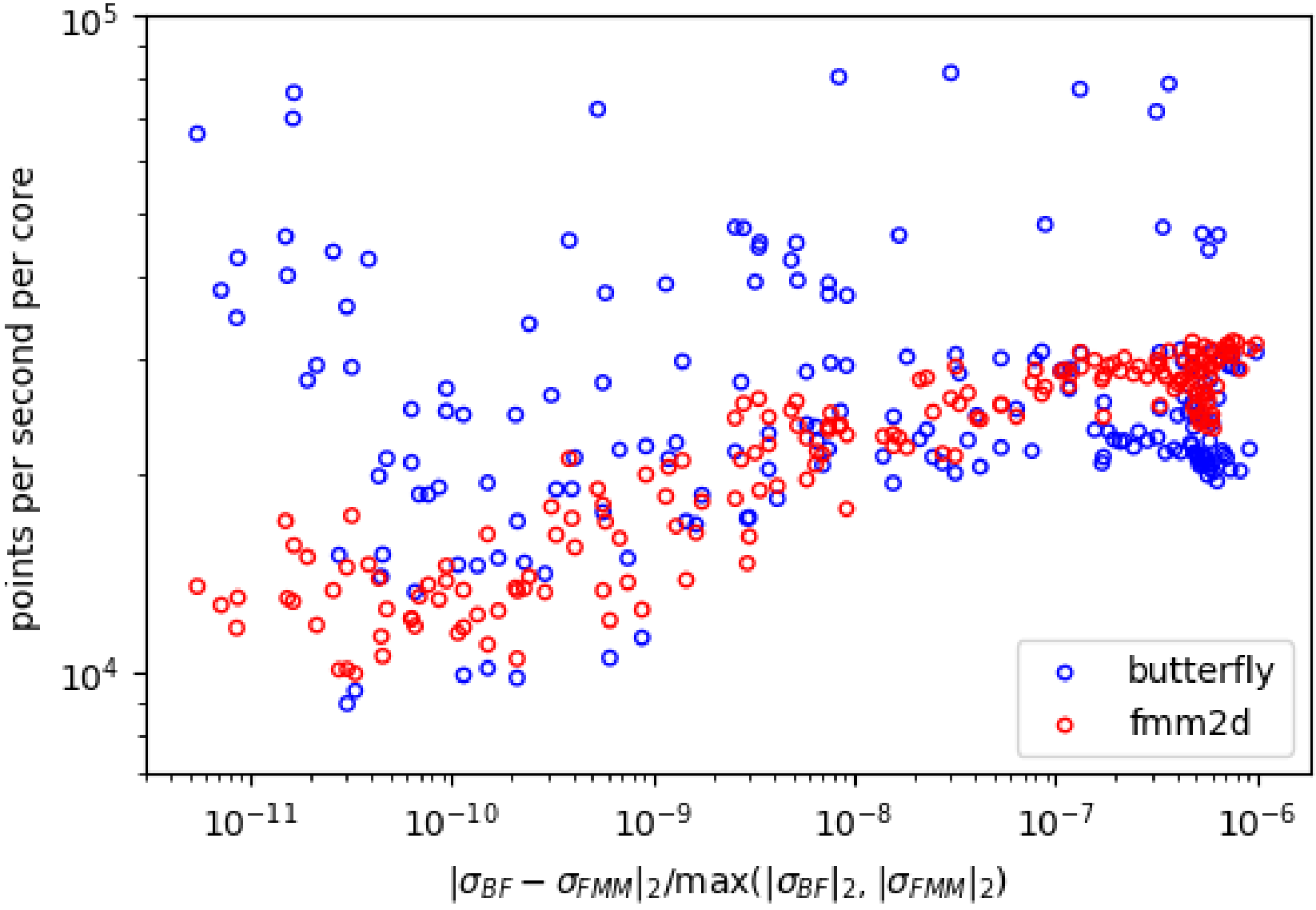
Numerical experiments



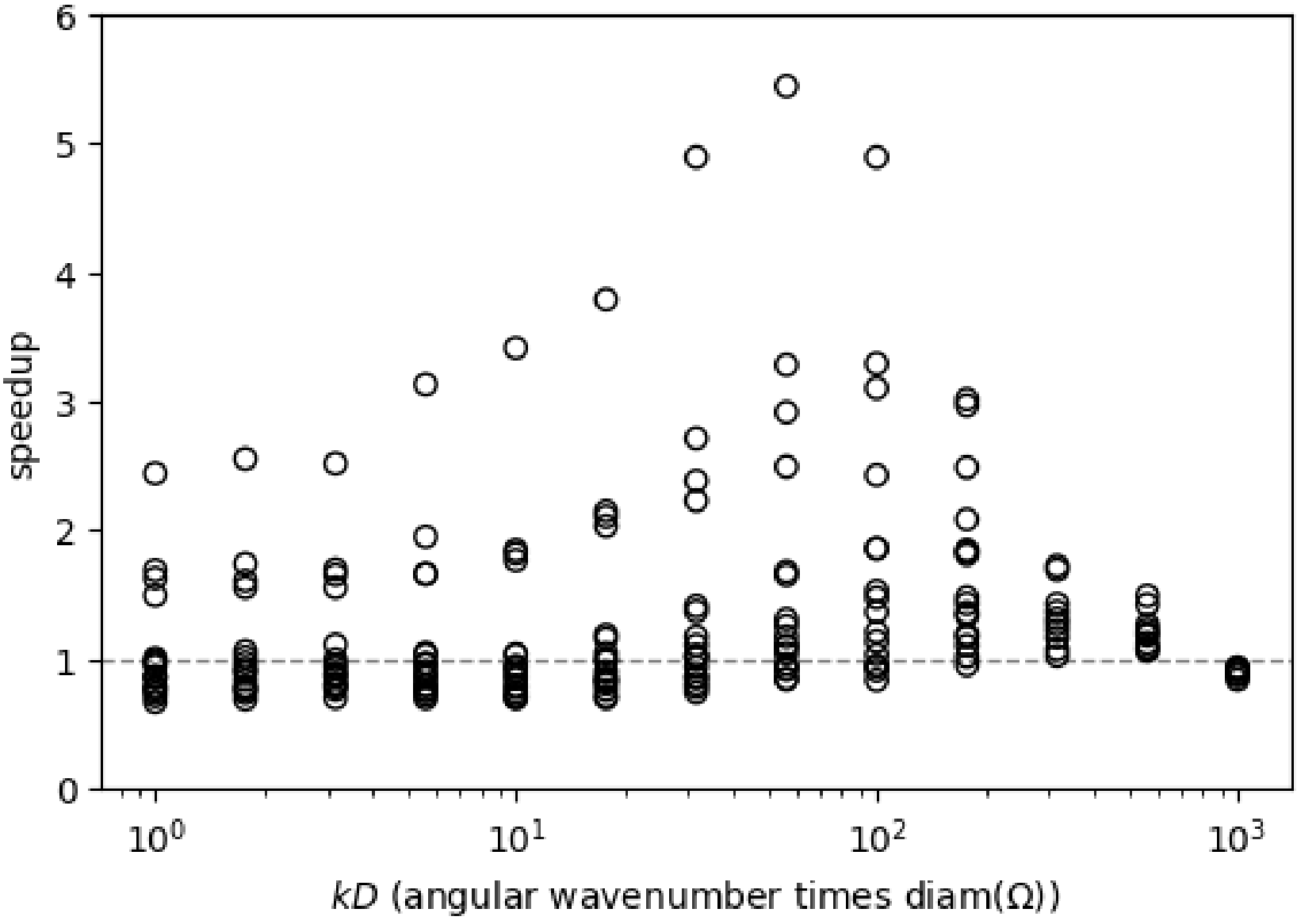
Numerical experiments



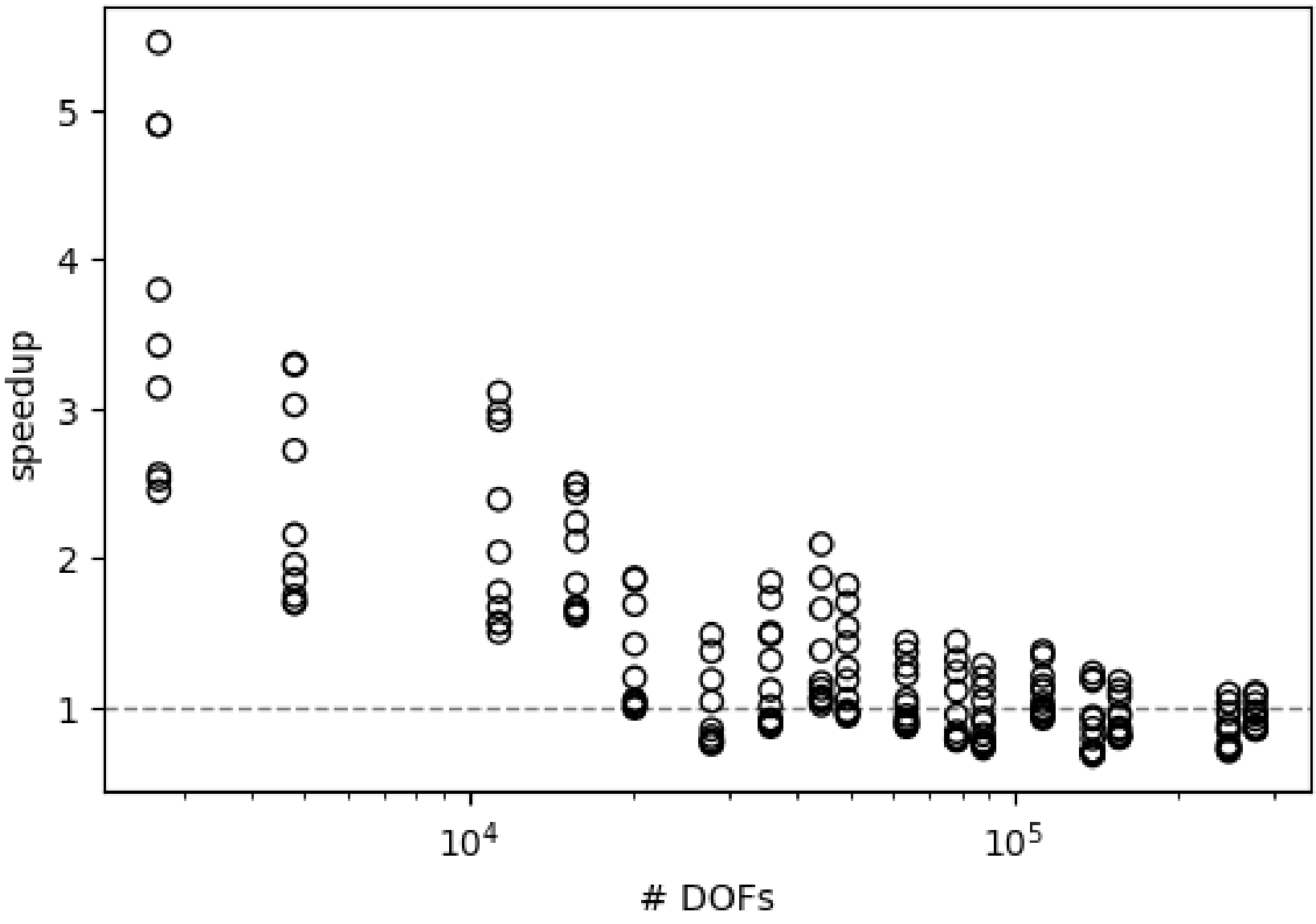
Numerical experiments



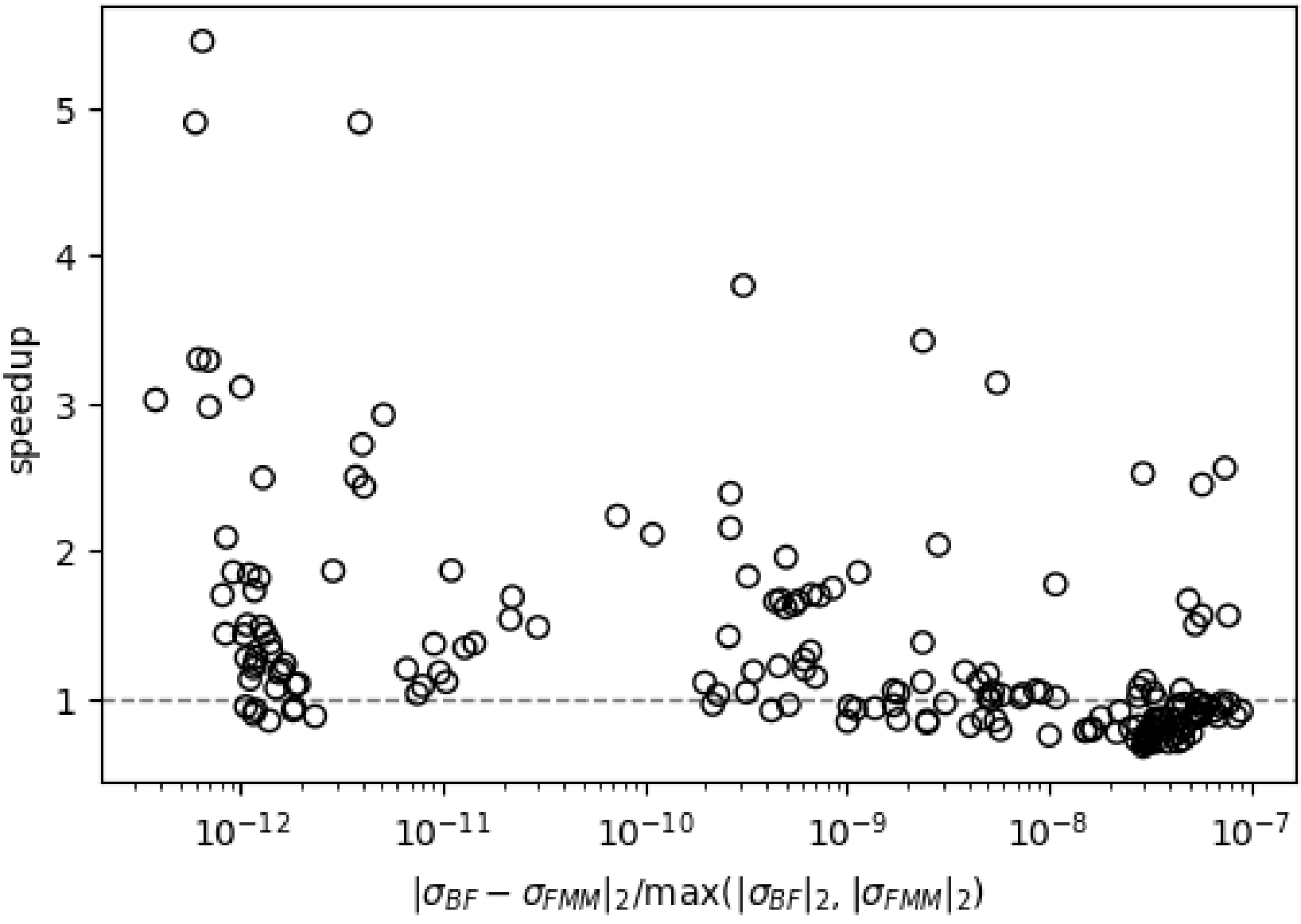
Numerical experiments



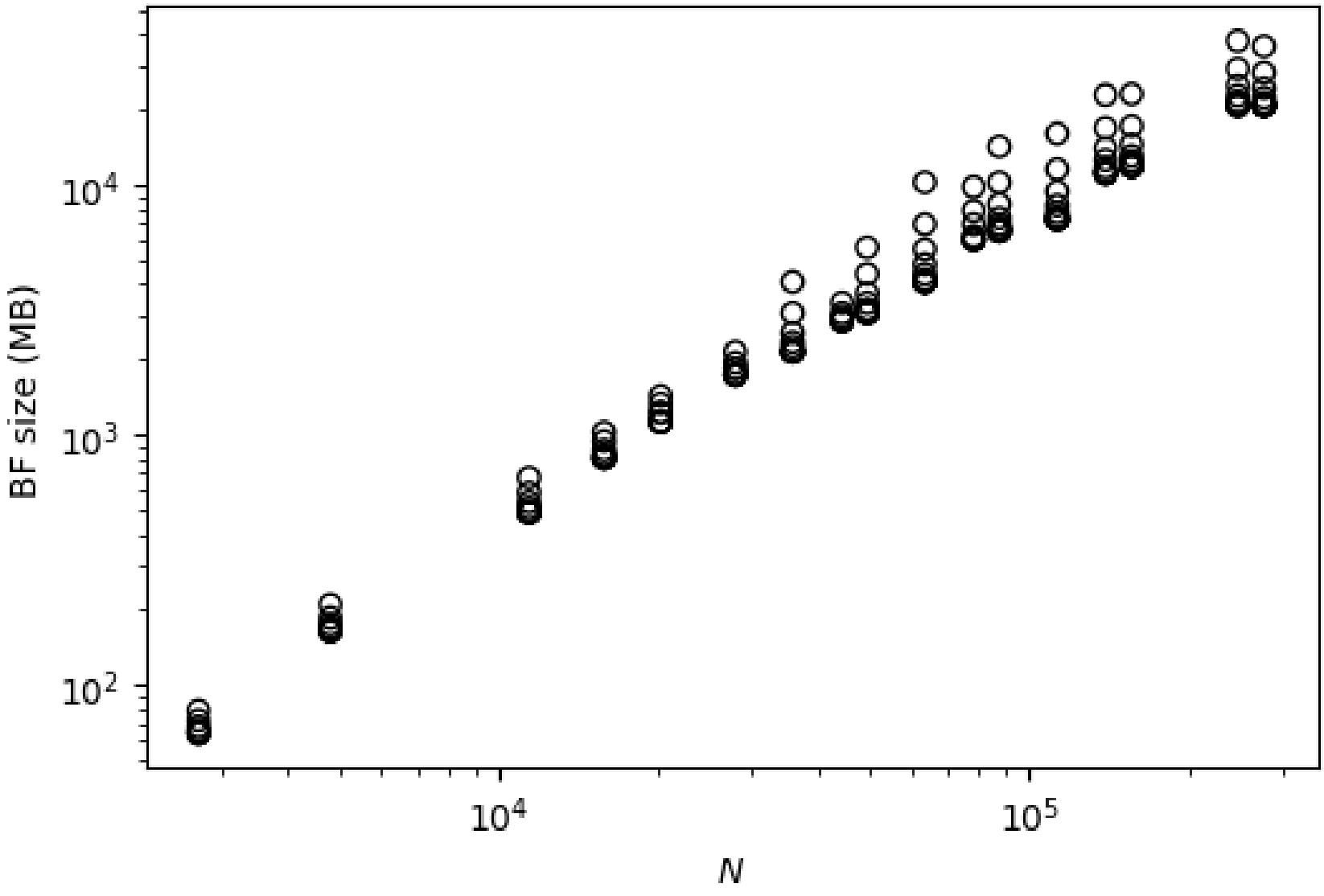
Numerical experiments



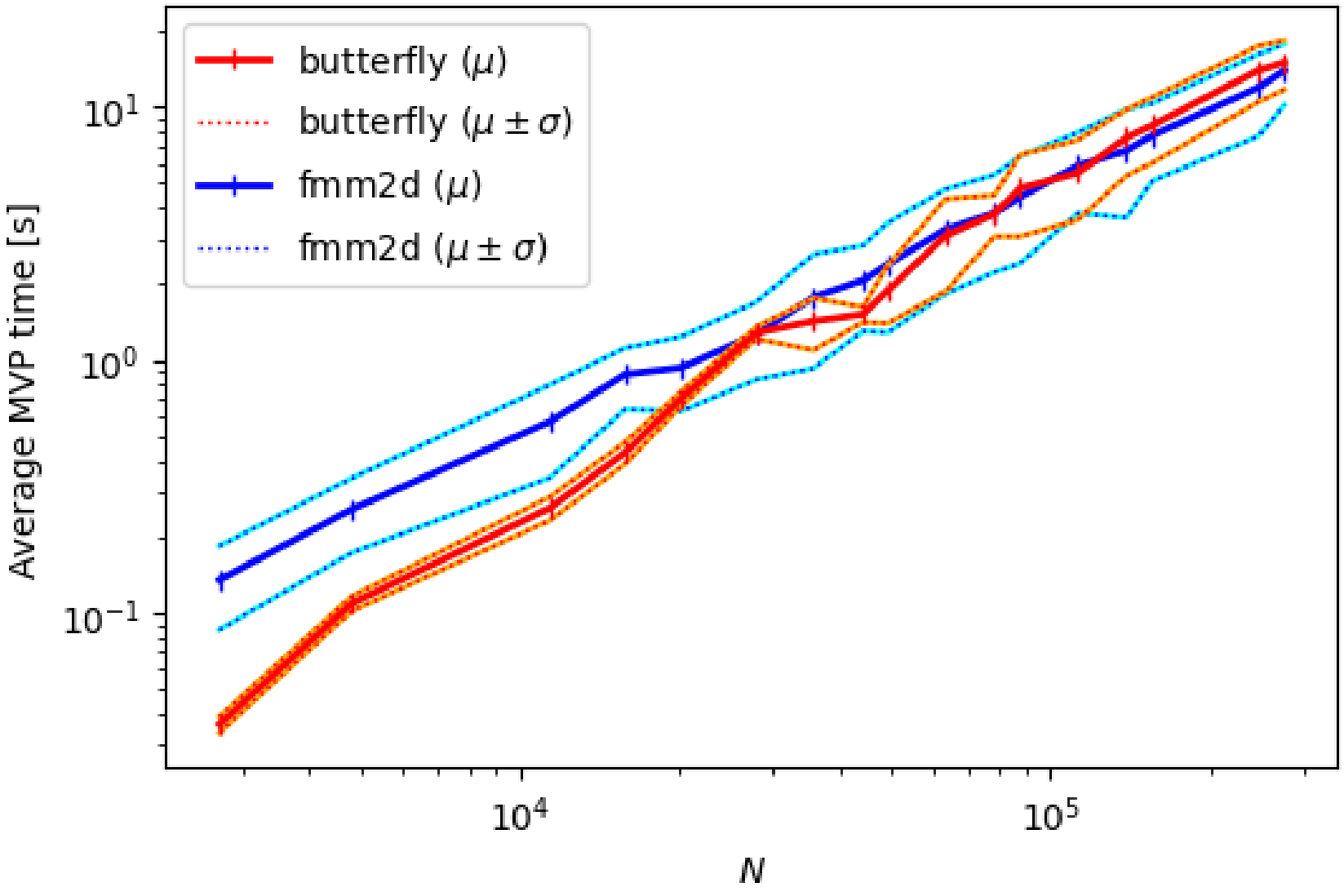
Numerical experiments



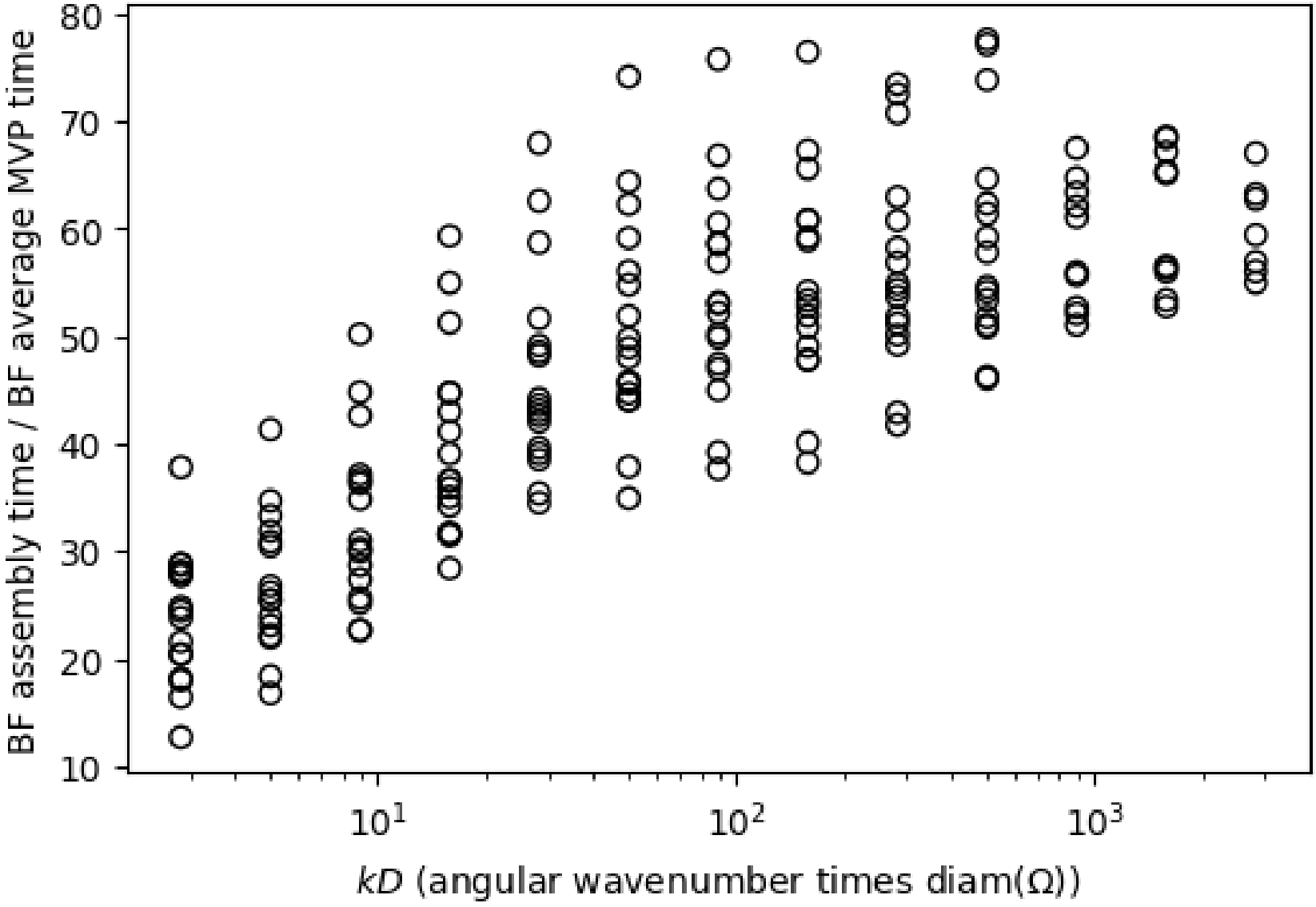
Numerical experiments



Numerical experiments



Numerical experiments



Prospectus

- Swap out OpenBLAS for BLIS
- Combine BF with HF-FMM
- Solve 3D scattering problems
- Even better a priori rank estimate
- Reduce memory use... is an in-place analysis-based algorithm possible? (Zernike-polynomial based butterfly?)
- Parallelism

butterfly: a C library for butterfly factorizations

Motivation

- Butterfly is all about hierarchical block matrices.
- Python is slow, Julia is :-(.
- There is no **usable** hierarchical matrix library.
- Lasting software is written in stable languages (C, C++, Fortran, maybe MATLAB).
- All languages' FFIs consume C (the lingua franca of systems programming).
- We can do better than arrays of floats.

```
https://github.com/sampotter/butterfly
```

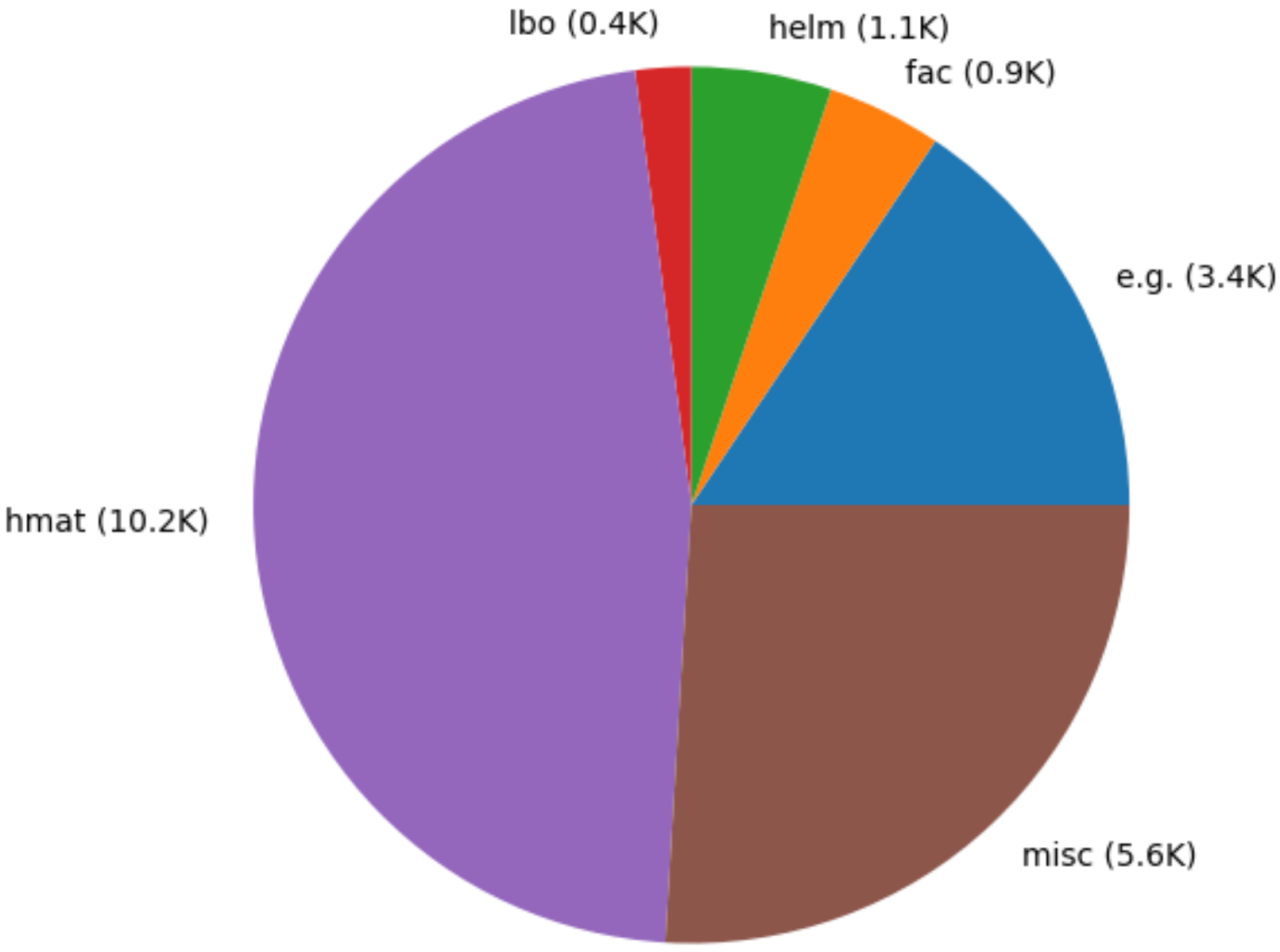
What's in the library?

- `hmat`: hierarchy of matrix types
- `misc`: trees, Chebyshev polynomials, fast tree evaluators, geometry, memory management, other support types
- `fac`: core butterfly factorization types
- `helm`: Helmholtz-specific BF stuff, BIE-specific stuff (layer potentials, BIE-friendly quadrature)
- `lbo`: streaming butterfly factorization and LBO-specific things
- `e.g.`: example programs

More about `hmat`

- Not unlike numpy (but faster!)
- Encode knowledge about structured matrices
- Blocks matrices, dense matrices, sparse matrices, diagonal matrices, matrix factorizations, etc.
- A matrix knows if it's triangular, orthogonal, etc.
- Doesn't *do* any linear algebra itself: delegate to BLAS (BLIS), LAPACK (FLAME), ARPACK, UMFPACK, CHOLMOD, SLEPc, EVSL, whatever...
- **Goal**: encode our knowledge about how linear algebra works in a library (numpy doesn't go far enough)

`cloc` breakdown (total: 21.2K LOC)



Prospectus

`butterfly`:

- Middle-out butterfly factorizations
- Other low-rank approximations (ID, QR, etc.)
- **Framework for implementing any butterfly factorization**

`hmat`:

- Hierarchical matrix algebra
- Task-based parallelism
- Batched linear algebra on GPU?
- A binary file format (“`.hmat`”)

Together: the ingredients for high-performance **fast direct solvers**

Conclusion

- Introduce new algorithm for butterfly factorization of the “manifold harmonic transform”.
- Thorough evaluation of ML-BF indicates that it is competitive with HF-FMM.
- New C library for BFs built on top of...
- ... a new C library for hierarchical matrices.

https://github.com/sampotter/butterfly

Conclusion

- **SFP**, Paul Beckman, Mike O'Neil. "Butterfly-accelerated manifold harmonic transforms." *In preparation.*
- **SFP**, Leslie Greengard, Mike O'Neil. "Butterfly-accelerated multiple scattering." *In preparation.*

Thank you!